EXAM PACK

EEE455

VLSI Design and Testing



Exam Pack

Session 2020–21 *Solutions and formula sheet of the 4th Year 1st Semester Final Examination 2024*

IN THIS PACK

1 PAST PAPER SOLUTIONS
Question Paper Solutions 2020–21 — pages 3--78 — 76 pages

2 FORMULA SHEET
Formula Sheet — pages 79--90 — 12 pages

Tap a document to open it. Each one keeps its own cover, contents and links.

Prepared by Nur Mohammad

November 2026

IN THIS PACK

Exam Pack

Scan to open the website

Tap a heading to open that document, or a line to go straight to that page. Each document keeps its own cover, contents and links; the bookmarks panel lists them all.

1 PAST PAPER SOLUTIONS
Question Paper Solutions 2020–21

pages 3--78

Question Paper 5
1(a) C-V Characteristics of the MOS Gate Capacitance 13
1(b) Pseudo-NMOS Inverter Ratio 16
1(c) Schematic and Stick Diagram of a Complex Gate 19
1(d) Minimum Path Delay by Logical Effort 23
1(e) Diffusion versus Ion Implantation 26
2(a) Elmore Delay of an RC Tree 29
2(b) Effect of the Beta Ratio on the Inverter 31
2(c) Fastest 8-Input AND Gate 33
2(d) Static and Dynamic Power Dissipation 35
2(e) Domino Logic and the Domino OR Gate 36
3(a) Car Alarm in Five Logic Styles 39
3(b) Transistor Sizing for Equal Rise and Fall Resistance 42
3(c) Dynamic Power and Delay versus Supply Voltage 44
3(d) Skewed and Unskewed Inverters 47
4(a) Setup and Hold Time Check 51
4(b) Asynchronous Resettable Flip-Flop 54
4(c) From Verilog Code to a Fabricated Chip 56
4(d) AND Gate with Transmission Gates and Pass Transistors 57
5(a) 2x2 SRAM Array with 6T Cells 59
5(b) Bus Architectures of a 4-Bit Datapath 63
5(c) Masks and Photoresists 65
5(d) Patterning SiO2 by Photolithography 66
6(a) 2-to-1 Multiplexer with Pass Transistors 70
6(b) 7-Segment Display Driver in Verilog 71
6(c) Divide-by-3 Counter as an FSM 72
6(d) Verilog Code for the Divide-by-3 FSM 74
6(e) XOR in a PLA 75
6(f) Truth Table of the Flip-Flop in Figure 6(f) 76

2 FORMULA SHEET
Formula Sheet

pages 79--90

1 MOS transistor 82
2 Inverter levels and noise margins 83
3 Delay 83
4 Transistor sizing 84
5 Power 85
6 Sequential timing 85
7 Memory and logic functions 86
8 Key formulas 87
9 Symbol index 88

PAST PAPER SOLUTIONS

EEE455

VLSI Design and Testing

Question Paper Solutions

Session 2020–21

4th Year 1st Semester Final Examination 2024

QUESTION MAP

Q1	Q2	Q3	Q4	Q5	Q6
25 marks	25 marks	25 marks	25 marks	25 marks	25 marks

EACH ANSWER



Prepared by Nur Mohammad

November 2026

BACK OF THE COVER

The paper at a glance

3 h	100	4 of 6	45 min
Total time	Total marks	Questions to answer	Per question

Time = marks × 1.8 min (180 min for 100 marks). Tick the questions you will answer; each sub-part links to its solution.

Q	Sub-part	Marks	Time	Pick
Group-A · Answer any two questions from Group-A				
Q1	(a) C-V Characteristics of the MOS Gate Capacitance	4	7 min	
	(b) Pseudo-NMOS Inverter Ratio	8	14 min	
	(c) Schematic and Stick Diagram of a Complex Gate	6	11 min	☐
	(d) Minimum Path Delay by Logical Effort	5	9 min	
	(e) Diffusion versus Ion Implantation	2	4 min	
Q2	(a) Elmore Delay of an RC Tree	5	9 min	
	(b) Effect of the Beta Ratio on the Inverter	6	11 min	
	(c) Fastest 8-Input AND Gate	7	13 min	☐
	(d) Static and Dynamic Power Dissipation	4	7 min	
	(e) Domino Logic and the Domino OR Gate	3	5 min	
Q3	(a) Car Alarm in Five Logic Styles	6	11 min	
	(b) Transistor Sizing for Equal Rise and Fall Resistance	4	7 min	☐
	(c) Dynamic Power and Delay versus Supply Voltage	8	14 min	
	(d) Skewed and Unskewed Inverters	7	13 min	
Group-B · Answer any two questions from Group B				
Q4	(a) Setup and Hold Time Check	10	18 min	
	(b) Asynchronous Resettable Flip-Flop	6	11 min	☐
	(c) From Verilog Code to a Fabricated Chip	6	11 min	
	(d) AND Gate with Transmission Gates and Pass Transistors	3	5 min	
Q5	(a) 2x2 SRAM Array with 6T Cells	10	18 min	
	(b) Bus Architectures of a 4-Bit Datapath	7	13 min	☐
	(c) Masks and Photoresists	2	4 min	
	(d) Patterning SiO2 by Photolithography	6	11 min	
Q6	(a) 2-to-1 Multiplexer with Pass Transistors	4	7 min	
	(b) 7-Segment Display Driver in Verilog	5	9 min	
	(c) Divide-by-3 Counter as an FSM	6	11 min	☐
	(d) Verilog Code for the Divide-by-3 FSM	3	5 min	
	(e) XOR in a PLA	3	5 min	
	(f) Truth Table of the Flip-Flop in Figure 6(f)	4	7 min	

Formulas follow Weste and Harris, CMOS VLSI Design (capacitances, logical effort, path design, timing) and Kang and Leblebici, CMOS Digital Integrated Circuits (long-channel MOSFET model).

Shahjalal University of Science and Technology

Department of Electrical and Electronic Engineering
4th Year 1st Semester Final Examination 2024
Course Code: EEE 455, Session: 2020-21
Course Title: VLSI Design and Testing

Full Marks: 100 **Credits: 3.0** **Duration: 3 Hours**

Group-A

[Answer any two questions from Group-A]

1. (a) Explain the C-V characteristics of intrinsic MOS gate capacitance in different regions. 4

   (b) A pseudo NMOS inverter is designed such that the output voltage becomes 0.25 V when the input voltage is 5 V. The following data are given: $\mu_n C_{ox} = 120\ \mu$A/V^2, $\mu_p C_{ox} = 50\ \mu$A/V^2, $V_{ton} = 1$V, $V_{top} = -1$ V, $V_{DD} = 5$ V, $\gamma = 0.5$. Assume that the body of the NMOS transistor is connected with the ground and the body of the PMOS transistor is connected with V_{DD}.

   (i) Draw the circuit diagram of the inverter.
   (ii) Calculate the inverter ratio i.e., the ratio of driver transistor (W/L) to the load transistor (W/L).
   (iii) What will be the output voltage if the input voltage is held at 0.25 V? Explain your answer. 8

   (c) Draw the schematic and stick diagram of the circuit implementing the logic $Y = \overline{(A + B + C) \cdot D}$ 6

   (d) Estimate the minimum delay of the path from A to B in Figure 1(d) and choose transistor sizes to achieve this delay. The initial NAND2 gate may present a load of 8 λ of transistor width on the input and the output load is equivalent to 45 λ of transistor width. 5

   (e) Differentiate between the diffusion process and the ion implantation process in semiconductor fabrication. 2

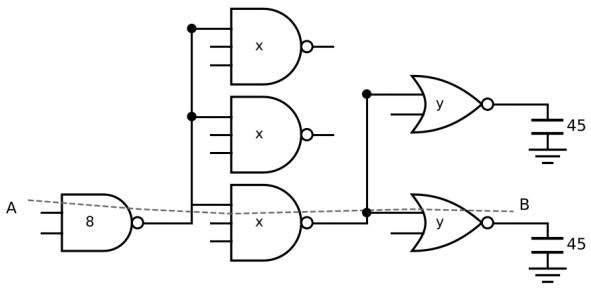


Figure 1(d) of the paper (redrawn)

2. (a) For the RC circuit shown in Figure 2(a), compute the Elmore delays from input node to node 7 and from input node to node 5 of this circuit in terms of R and C component values. 5

   (b) Briefly explain what happens to the transfer characteristics and noise margins when beta ratio is increased in a CMOS inverter pair. 6

   (c) Design an 8-input AND gate using FO4 in different configurations. Determine which configuration provides the fastest performance. 7

Free shared copy · Nur Mohammad

(d) Write short note on –i) Static power dissipation ii) Dynamic power dissipation. 4

(e) What is domino gate? Draw the circuit diagram of a domino OR gate. 3

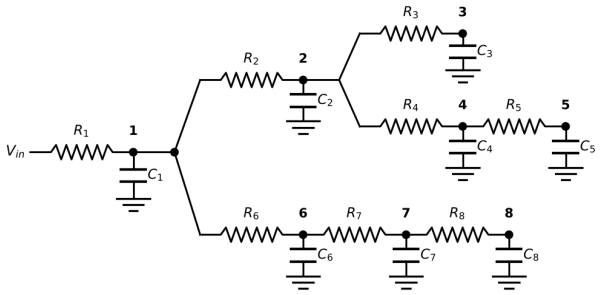


Figure 2(a) of the paper (redrawn)

3. (a) You have to design a car alarm system in which the alarm will ring (Y= 1) if anyone of the following condition is satisfied.

(i) The key is in (K= 1), The Seat belt is fastened (B=1) and the door is open (D=0) or
(ii) The key is in, Door is closed and Seat belt is not fastened.

Write the Truth table, the Boolean equations and show the implementation of the circuit in (i) static CMOS, (ii) NMOS, (iii) Pseudo NMOS, (iv) Footed dynamic CMOS and (v) Un-footed dynamic CMOS. 6

(b) For the logic function $Y = \overline{(A + BC) \cdot D}$, choose transistor widths to achieve equal rise and fall resistance as a unit inverter. Assume, $\mu_n = 3\mu_p$ 4

(c) Show that the dynamic power dissipation of a CMOS inverter varies directly with square of the power supply voltage whereas the delay of the circuit varies inversely with the power supply voltage. 8

(d) Explain the inverter width sizing for favoring the rising edge, falling edge, or achieving an unskewed inverter. 7

Free shared copy · Nur Mohammad

Group-B

[Answer any two questions from Group B]

4. (a) In the following circuit of Figure 4(a), determine whether there is any set up or hold time violation. 10

   (b) Show the circuit diagram of an asynchronous resettable Flip-Flop and explain its operation utilizing a timing diagram. 6

   (c) Suppose you have written Verilog code to operate a microprocessor. What will be the upcoming steps till its fabrication? Briefly explain your answer. 6

   (d) Implement a 2-input AND gate using both transmission gates and pass-transistor logic. 3

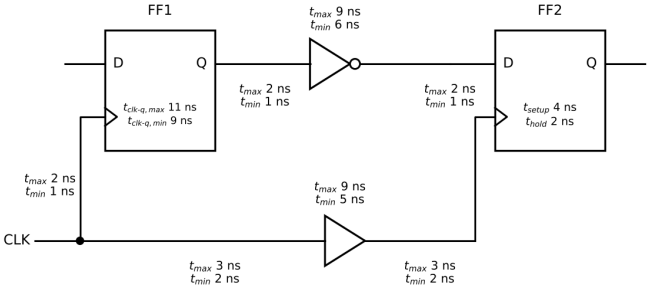


Figure 4(a) of the paper (redrawn)

5. (a) (i) Show the circuit diagram of a 2 x 2 SRAM array which uses 6-transistor memory cell as basic storage cell. Clearly show the row select, column select, pre-charge and sense signal in your circuit.

   (ii) Explain how the READ and WRITE operation are performed showing the second row and first column cell as an example.

   (iii) Explain how sense amplifier reduces the READ time of the cell. 10

   (b) A 4-bit datapath consists of register, ALU and a shifter. Show two possible bus architectures of the system such that an addition operation of two operands stored in the register and storing the result back in the register can be computed in at most two clock cycles. 7

   (c) Explain the following: (i) positive mask, (ii) negative mask, (iii) positive photoresist and (iv) negative photoresist. 2

   (d) Describe the process sequence for patterning SiO_2 as illustrated in the figure 5(d). 6

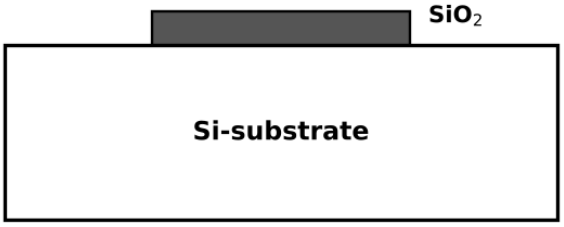


Figure 5(d) of the paper (redrawn)

6. (a) Design a 2-to-1 multiplexer using pass transistor logic. Mention its working procedure. 4

   (b) Design a 7-segment display driver in Verilog. 5

Free shared copy · Nur Mohammad

(c) You have to design a divide-by-3 counter using FSM.

i) Draw the state transition diagram of the finite state machine described.
ii) Derive the state transition table and equations for the next-state logic.
iii) Draw the complete FSM circuit. 6

(d) Write a Verilog code for the FSM described in Question 6(c). 3

(e) Sketch a dot diagram for a 2-input XOR using a PLA. 3

(f) Fill in the truth table for the flip flop shown in Figure 6(f). Is this a positive edge triggered flip flop? 4

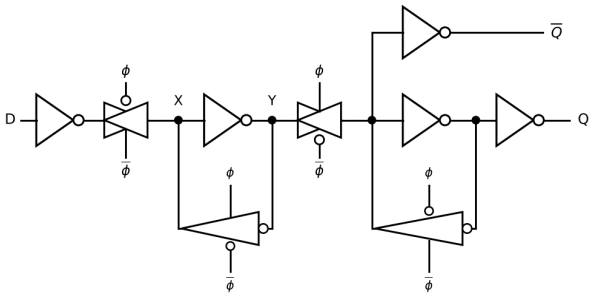


Figure 6(f) of the paper (redrawn)

ϕ	$\overline{\phi}$	D	X	Y	$\overline{Q}$	Q
0	1		1			1
1	0	1				
0	1	1				
1	0	1				

The table of Figure 6(f), as printed

Free shared copy · Nur Mohammad

Contents

1(a) C-V Characteristics of the MOS Gate Capacitance **11**
1 Theory . . . 11
2 Answer . . . 13
3 Keyword Explanation . . . 14
4 Notes and Remarks . . . 14

1(b) Pseudo-NMOS Inverter Ratio **14**
1 Theory . . . 14
2 Answer . . . 16
3 Keyword Explanation . . . 17
4 Notes and Remarks . . . 17

1(c) Schematic and Stick Diagram of a Complex Gate **17**
1 Theory . . . 17
2 Answer . . . 18
3 Keyword Explanation . . . 20
4 Notes and Remarks . . . 20

1(d) Minimum Path Delay by Logical Effort **21**
1 Theory . . . 21
2 Answer . . . 22
3 Keyword Explanation . . . 24
4 Notes and Remarks . . . 24

1(e) Diffusion versus Ion Implantation **24**
1 Theory . . . 24
2 Answer . . . 25
3 Notes and Remarks . . . 25
4 Final Concise Answer . . . 25

2(a) Elmore Delay of an RC Tree **27**
1 Theory . . . 27
2 Answer . . . 28
3 Keyword Explanation . . . 28
4 Notes and Remarks . . . 29

2(b) Effect of the Beta Ratio on the Inverter **29**
1 Theory . . . 29
2 Answer . . . 30
3 Keyword Explanation . . . 30
4 Notes and Remarks . . . 30

2(c) Fastest 8-Input AND Gate **31**
1 Theory . . . 31
2 Answer . . . 31
3 Keyword Explanation . . . 33
4 Notes and Remarks . . . 33

2(d) Static and Dynamic Power Dissipation **33**
1 Theory . . . 33
2 Answer . . . 34

Free shared copy · Nur Mohammad

3 Keyword Explanation . . . 34

2(e) Domino Logic and the Domino OR Gate **34**
1 Theory . . . 34
2 Answer . . . 36
3 Keyword Explanation . . . 36
4 Notes and Remarks . . . 36
5 Final Concise Answer . . . 36

3(a) Car Alarm in Five Logic Styles **37**
1 Theory . . . 37
2 Answer . . . 38
3 Keyword Explanation . . . 40
4 Notes and Remarks . . . 40

3(b) Transistor Sizing for Equal Rise and Fall Resistance **40**
1 Theory . . . 41
2 Answer . . . 41
3 Keyword Explanation . . . 42
4 Notes and Remarks . . . 42

3(c) Dynamic Power and Delay versus Supply Voltage **42**
1 Theory . . . 42
2 Answer . . . 43
3 Keyword Explanation . . . 44
4 Notes and Remarks . . . 45

3(d) Skewed and Unskewed Inverters **45**
1 Theory . . . 45
2 Answer . . . 45
3 Keyword Explanation . . . 46
4 Notes and Remarks . . . 46
5 Final Concise Answer . . . 46

4(a) Setup and Hold Time Check **49**
1 Theory . . . 49
2 Answer . . . 50
3 Keyword Explanation . . . 52
4 Notes and Remarks . . . 52

4(b) Asynchronous Resettable Flip-Flop **52**
1 Theory . . . 52
2 Answer . . . 53
3 Keyword Explanation . . . 54
4 Notes and Remarks . . . 54

4(c) From Verilog Code to a Fabricated Chip **54**
1 Theory . . . 54
2 Answer . . . 54
3 Keyword Explanation . . . 55

4(d) AND Gate with Transmission Gates and Pass Transistors **55**
1 Theory . . . 55

Free shared copy · Nur Mohammad

2 Answer . . . 56
3 Final Concise Answer . . . 56

5(a) 2x2 SRAM Array with 6T Cells **57**
1 Theory . . . 57
2 Answer . . . 58
3 Keyword Explanation . . . 61
4 Notes and Remarks . . . 61

5(b) Bus Architectures of a 4-Bit Datapath **61**
1 Theory . . . 62
2 Answer . . . 62
3 Keyword Explanation . . . 63
4 Notes and Remarks . . . 63

5(c) Masks and Photoresists **63**
1 Theory . . . 63
2 Answer . . . 64
3 Notes and Remarks . . . 64

5(d) Patterning SiO2 by Photolithography **64**
1 Theory . . . 64
2 Answer . . . 65
3 Keyword Explanation . . . 66
4 Final Concise Answer . . . 66

6(a) 2-to-1 Multiplexer with Pass Transistors **68**
1 Theory . . . 68
2 Answer . . . 68

6(b) 7-Segment Display Driver in Verilog **69**
1 Theory . . . 69
2 Answer . . . 69
3 Notes and Remarks . . . 70

6(c) Divide-by-3 Counter as an FSM **70**
1 Theory . . . 70
2 Answer . . . 70
3 Keyword Explanation . . . 72
4 Notes and Remarks . . . 72

6(d) Verilog Code for the Divide-by-3 FSM **72**
1 Theory . . . 72
2 Answer . . . 73

6(e) XOR in a PLA **73**
1 Theory . . . 73
2 Answer . . . 74

6(f) Truth Table of the Flip-Flop in Figure 6(f) **74**
1 Theory . . . 74
2 Answer . . . 75
3 Keyword Explanation . . . 76

Free shared copy · Nur Mohammad

4 Final Concise Answer 76

Free shared copy · Nur Mohammad

1. (a) Explain the C-V characteristics of intrinsic MOS gate capacitance in different regions. Marks: 4

   (b) A pseudo NMOS inverter is designed such that the output voltage becomes 0.25 V when the input voltage is 5 V. The following data are given: $\mu_n C_{ox} = 120\ \mu A/V^2$, $\mu_p C_{ox} = 50\ \mu A/V^2$, $V_{ton} = 1V$, $V_{top} = -1$ V, $V_{DD} = 5$ V, $\gamma = 0.5$. Assume that the body of the NMOS transistor is connected with the ground and the body of the PMOS transistor is connected with V_{DD}.

   (i) Draw the circuit diagram of the inverter.

   (ii) Calculate the inverter ratio i.e., the ratio of driver transistor (W/L) to the load transistor (W/L).

   (iii) What will be the output voltage if the input voltage is held at 0.25 V? Explain your answer. Marks: 8

   (c) Draw the schematic and stick diagram of the circuit implementing the logic $Y = \overline{(A+B+C)\cdot D}$ Marks: 6

   (d) Estimate the minimum delay of the path from A to B in Figure 1(d) and choose transistor sizes to achieve this delay. The initial NAND2 gate may present a load of 8 λ of transistor width on the input and the output load is equivalent to 45 λ of transistor width. Marks: 5

   (e) Differentiate between the diffusion process and the ion implantation process in semiconductor fabrication. Marks: 2

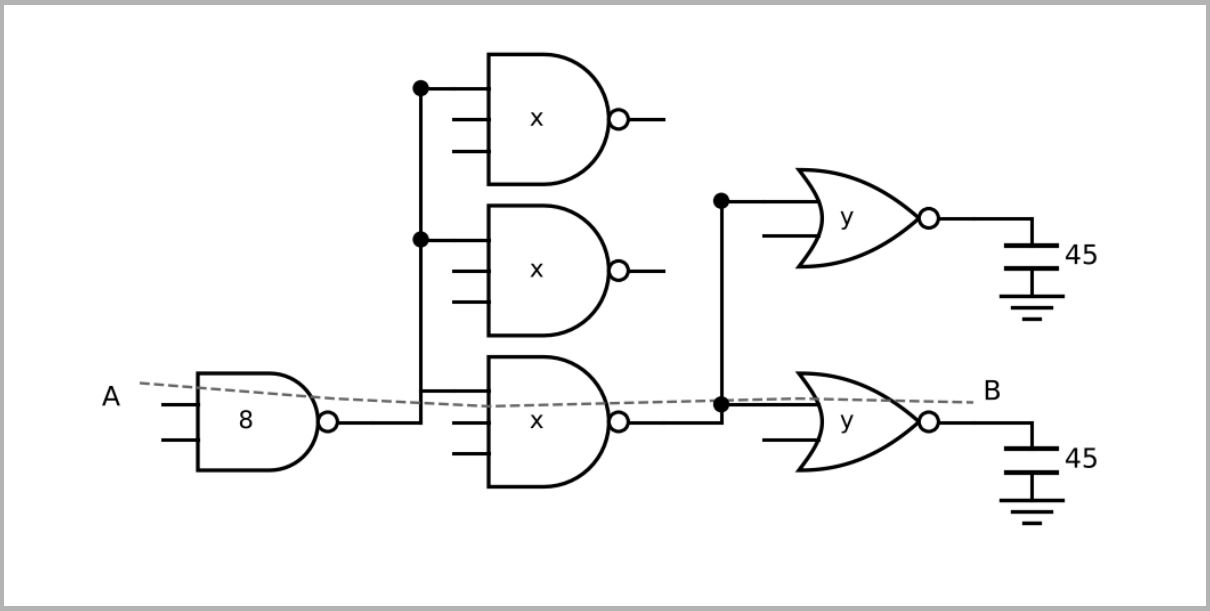


Figure 1: Figure 1(d) of the paper (redrawn)

1.(a) Explain the C-V characteristics of intrinsic MOS gate capacitance in different regions. Marks: 4

1. Theory

1. What is a MOS transistor made of? (Core Idea)

A MOS transistor is a sandwich of three layers, stacked from top to bottom:

- **Metal (gate):** today it is polysilicon, which conducts like a metal.
- **Oxide:** a very thin layer of silicon dioxide (SiO_2), which is a perfect insulator.
- **Semiconductor (body):** the silicon underneath. For an nMOS transistor the body is p-type, so it is full of free **holes** (positive carriers).

Two heavily doped n-type regions, the **source** and the **drain**, sit in the body on either side of the gate.

Free shared copy · Nur Mohammad

Because the oxide does not conduct, the gate and the body form a **capacitor**: two conducting plates (gate and body) with an insulator between them. Its value, when the "plates" are the gate and the silicon just under the oxide, is

$$C_0 = C_{ox}\, W L, \qquad C_{ox} = \frac{\varepsilon_{ox}}{t_{ox}}$$

where

- C_{ox} is the oxide capacitance per unit area (F/m^2)
- $\varepsilon_{ox} = 3.9\,\varepsilon_0$ is the permittivity of SiO_2 and t_{ox} its thickness
- W and L are the width and length of the gate

2. Why does the capacitance change with the gate voltage?

A capacitor stores charge on its two plates. The gate plate is always a conductor. The "plate" on the silicon side is not fixed: it is wherever the silicon puts its charge, and that depends on the gate voltage V_g. So the capacitance changes with V_g. A plot of capacitance against voltage is called the **C-V characteristic**.

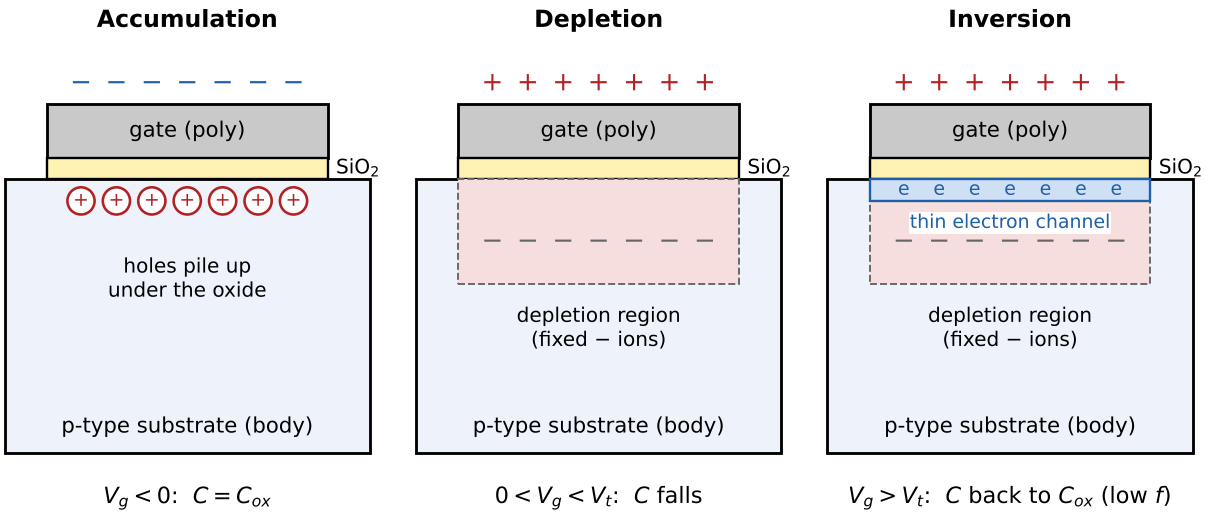


Figure 2: The three regions of a MOS capacitor on p-type silicon: accumulation, depletion and inversion

Reading the figure. Each panel is a cross-section through the gate. The signs above the gate show the charge on the gate.

- **Accumulation ($V_g < 0$):** the negative gate attracts the holes of the p-type body. They pile up right under the oxide. The silicon-side plate is therefore at the oxide surface, only t_{ox} away from the gate, so $C = C_{ox}$, the largest value.
- **Depletion ($0 < V_g < V_t$):** a positive gate pushes the holes away. A layer with no free carriers, the **depletion region**, forms under the oxide. It holds only the fixed negative ions of the acceptor atoms. The charge now sits deeper, so the plates are further apart. The depletion capacitance C_{dep} is in series with C_{ox}, and the total falls.
- **Inversion ($V_g > V_t$):** the gate is positive enough to pull electrons up to the surface. A thin layer of electrons, the **channel**, forms under the oxide; the surface has "inverted" from p-type to n-type. The plate is back at the oxide surface, so C returns to about C_{ox}.

Here V_t is the **threshold voltage**, the gate voltage at which the channel forms.

Free shared copy · Nur Mohammad

3. The C-V curve of the MOS capacitor

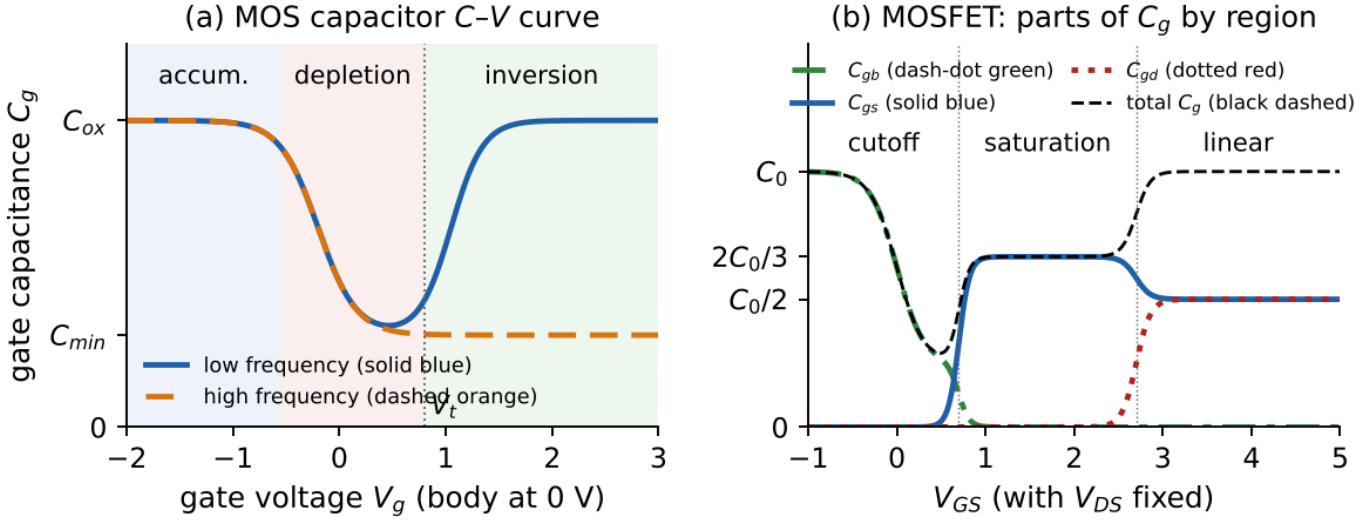


Figure 3: C-V characteristics: (a) MOS capacitor, (b) how the gate capacitance of a MOSFET is shared between body, source and drain in each region

Reading the figure. In panel (a), the curve is flat at C_{ox} in accumulation, falls through depletion to a minimum C_{min}, and rises back to C_{ox} in inversion. The solid blue curve is measured at **low frequency**: the channel electrons have time to follow the a.c. test signal. At **high frequency** (dashed orange) they cannot, because electrons are made slowly in p-type silicon, so the capacitance stays at C_{min}. In a MOSFET the source and drain supply the electrons quickly, so the gate capacitance behaves like the low-frequency curve.

4. Where does the capacitance go in a MOSFET?

In a transistor the gate charge has three partners: the body (b), the source (s) and the drain (d). So the gate capacitance is split into three parts, C_{gb}, C_{gs} and C_{gd}. Which one is large depends on the region of operation (Weste & Harris):

- **Cutoff ($V_{gs} < V_t$):** there is no channel. The gate sees only the body: $C_{gb} \approx C_0$ (less in depletion), $C_{gs} = C_{gd} = 0$.
- **Linear ($V_{gs} > V_t$, small V_{ds}):** a channel connects source and drain. It shields the body from the gate, so $C_{gb} = 0$. The channel is shared equally: $C_{gs} = C_{gd} = C_0/2$.
- **Saturation ($V_{gs} > V_t$, $V_{ds} > V_{gs} - V_t$):** the channel is pinched off near the drain, so the drain loses its share: $C_{gd} = 0$, $C_{gs} \approx \frac{2}{3}C_0$, $C_{gb} = 0$.

Panel (b) of the figure above shows these three parts as V_{gs} rises at a fixed V_{ds}: cutoff, then saturation, then linear.

2. Answer

Definition: The intrinsic gate capacitance of a MOS transistor is the capacitance of the gate–oxide–silicon sandwich. Its full value is $C_0 = C_{ox}WL$ with $C_{ox} = \varepsilon_{ox}/t_{ox}$. How large it is, and to which terminal it connects, depends on the gate voltage.

MOS capacitor C-V curve (p-type body) (figure in the Theory):

- **Accumulation, $V_g < 0$:** holes gather under the oxide; $C_g = C_{ox}WL = C_0$.
- **Depletion, $0 < V_g < V_t$:** holes are pushed away and a depletion layer forms; C_{dep} is in series with C_{ox}, so C_g falls to a minimum C_{min}.
- **Inversion, $V_g > V_t$:** an electron channel forms at the surface; C_g rises back to C_0 (at low frequency, and always in a MOSFET, where source and drain supply the electrons).

Free shared copy · Nur Mohammad

Split of the gate capacitance in a MOSFET:

Region	C_{gb}	C_{gs}	C_{gd}	Total C_g
Cutoff	$\leq C_0$	0	0	$\approx C_0$
Linear	0	$C_0/2$	$C_0/2$	C_0
Saturation	0	$\frac{2}{3}C_0$	0	$\frac{2}{3}C_0$

Table 1: Intrinsic gate capacitance of a MOSFET in each region

Interpretation: for delay estimates the gate is treated as a capacitor of about C_0 (in W&H units, C_g per µm of gate width), whatever the region.

3. Keyword Explanation

1. Threshold voltage V_t

↩ back

The gate voltage at which a conducting channel of electrons (nMOS) forms under the oxide.

2. Depletion region

↩ back

A layer of silicon with no free carriers, only fixed ionised dopant atoms. It acts like extra insulator thickness.

3. Intrinsic capacitance

↩ back

The capacitance that comes from the transistor action itself (gate to channel and body), as opposed to the **extrinsic** (parasitic) overlap and junction capacitances.

4. Notes and Remarks

- **Extrinsic parts are extra:** the gate also overlaps the source and drain a little ($C_{gs,ol}$, $C_{gd,ol}$), and the source and drain junctions have their own capacitance to the body. These do not change much with region.
- **Why 2/3 in saturation:** in saturation the channel charge is not uniform; it thins out towards the drain. Integrating it along the channel gives $\frac{2}{3}$ of the full value.

1.(b) A pseudo NMOS inverter is designed such that the output voltage becomes 0.25 V when the input voltage is 5 V. The following data are given: $\mu_n C_{ox}$ = 120 μA/V^2, $\mu_p C_{ox}$ = 50 μA/V^2, V_{ton} = 1V, V_{top} = -1 V, V_{DD} = 5 V, γ = 0.5. Assume that the body of the NMOS transistor is connected with the ground and the body of the PMOS transistor is connected with V_{DD}. (i) Draw the circuit diagram of the inverter. (ii) Calculate the inverter ratio i.e., the ratio of driver transistor (W/L) to the load transistor (W/L). (iii) What will be the output voltage if the input voltage is held at 0.25 V? Explain your answer. Marks: 8

1. Theory

1. What is a pseudo-NMOS inverter? (Core Idea)

A normal CMOS inverter has a pMOS on top and an nMOS below, and their gates are tied together, so exactly one of them is on at a time. A **pseudo-NMOS** inverter keeps the nMOS (the **driver**), but the pMOS (the **load**) has its gate tied to ground, so it is **always on**. It acts like a pull-up resistor.

- When the input is low, the nMOS is off; the always-on pMOS pulls the output up to V_{DD}.
- When the input is high, the nMOS is on and fights the pMOS. The output goes low, but not all the way to 0 V. It settles where the two currents are equal.

Because the low output depends on the strength ratio of the two transistors, pseudo-NMOS is called **ratioed logic**. Its advantage is fewer transistors (one pMOS for any gate); its cost is a static current whenever the output is low.

2. How much current does a MOSFET carry?

For the long-channel (Shockley) model (Kang & Leblebici):

$$I_D = \frac{k'}{2}\frac{W}{L}\left[2(V_{GS} - V_T)V_{DS} - V_{DS}^2\right] \quad \text{(linear: } V_{DS} < V_{GS} - V_T\text{)}$$

$$I_D = \frac{k'}{2}\frac{W}{L}(V_{GS} - V_T)^2 \quad \text{(saturation: } V_{DS} \geq V_{GS} - V_T\text{)}$$

where

- $k' = \mu C_{ox}$ is the process transconductance (A/V^2): $k'_n = \mu_n C_{ox}$ for nMOS, $k'_p = \mu_p C_{ox}$ for pMOS
- W/L is the aspect ratio of the transistor, chosen by the designer
- V_T is the threshold voltage. For a pMOS, all voltages and V_T are negative; the same formulas hold with signs kept

We write $k_n = k'_n(W/L)_n$ and $k_p = k'_p(W/L)_p$.

3. The body effect

If the source of a transistor is at a different voltage from its body, the threshold voltage rises:

$$V_T = V_{T0} + \gamma\left(\sqrt{|2\phi_F| + V_{SB}} - \sqrt{|2\phi_F|}\right)$$

where V_{T0} is the threshold with $V_{SB} = 0$, γ the body-effect coefficient and ϕ_F the Fermi potential. Here the nMOS source is at ground with its body, and the pMOS source is at V_{DD} with its body. So $V_{SB} = 0$ for both, and $V_T = V_{T0}$: **the given γ has no effect**.

4. Finding the output low voltage

With $V_{in} = V_{DD}$ and the output low:

- the nMOS has a large V_{GS} and a small V_{DS}, so it is in the **linear** region;
- the pMOS has $V_{GS} = 0 - V_{DD}$ and a large $|V_{DS}|$, so it is in **saturation**.

Setting the two currents equal gives the ratio k_n/k_p needed for a chosen V_{OL}.

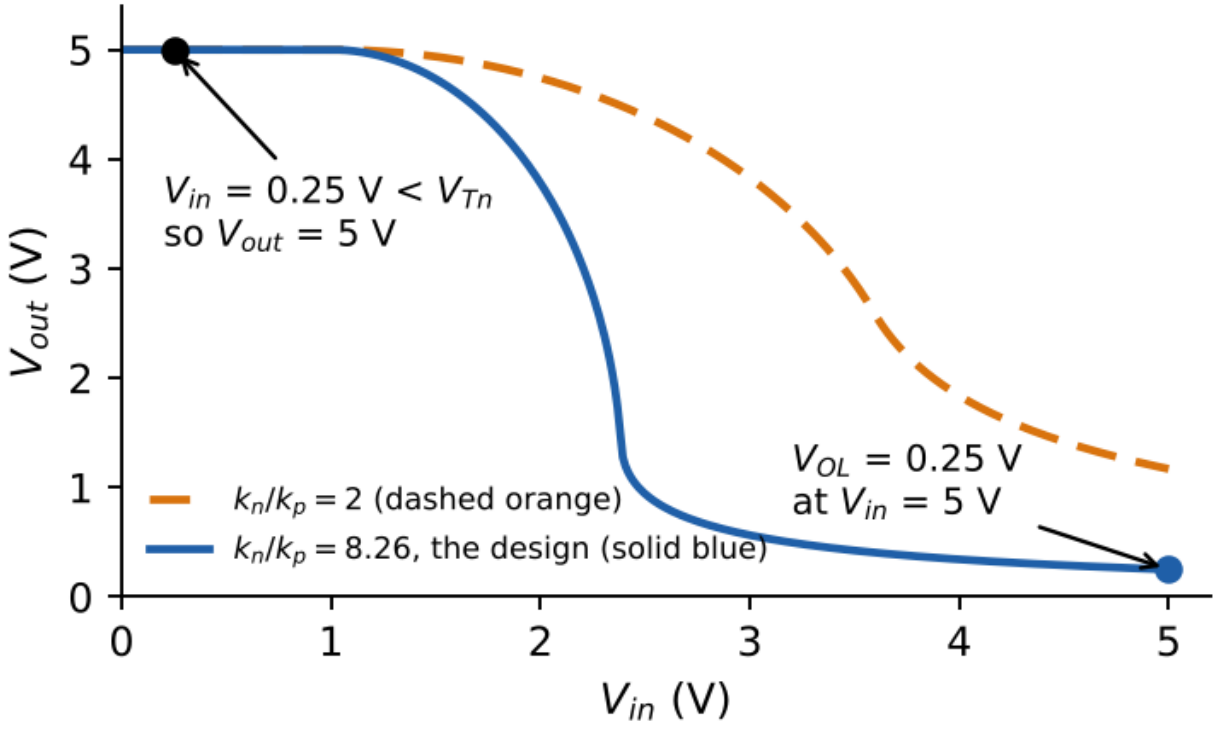


Figure 4: Pseudo-NMOS voltage transfer curves: the design ratio of 8.26 (solid blue) gives V_OL = 0.25 V; a weaker driver (dashed orange) cannot pull the output low

Free shared copy · Nur Mohammad

Reading the figure. Both curves start at $V_{OH} = 5$ V for a low input, because nothing fights the pMOS then. With the designed ratio (solid blue) the output falls to 0.25 V at $V_{in} = 5$ V. With a weaker driver ($k_n/k_p = 2$, dashed orange) the output stays above 1 V: the logic 0 is poor. The bigger the ratio, the lower V_{OL}.

2. Answer

Given:

- $k'_n = \mu_n C_{ox} = 120\ \mu\text{A/V}^2$, $k'_p = \mu_p C_{ox} = 50\ \mu\text{A/V}^2$
- $V_{T0,n} = 1$ V, $V_{T0,p} = -1$ V, $V_{DD} = 5$ V, $\gamma = 0.5$
- Design point: $V_{in} = 5$ V gives $V_{out} = V_{OL} = 0.25$ V

(i) Circuit diagram

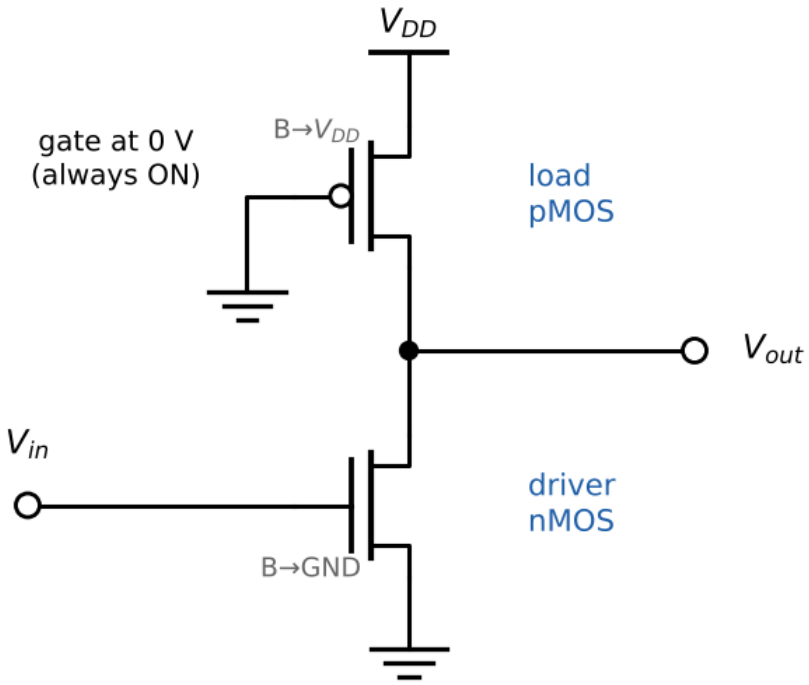


Figure 5: Pseudo-NMOS inverter: pMOS load with its gate at ground, nMOS driver; nMOS body at ground, pMOS body at VDD

Body effect: the nMOS source and body are both at ground, and the pMOS source and body are both at V_{DD}. So $V_{SB} = 0$ for both transistors and $V_{Tn} = 1$ V, $V_{Tp} = -1$ V; γ does not enter the answer.

(ii) Inverter ratio

Step 1: Regions. nMOS: $V_{GS} = 5$ V, $V_{DS} = 0.25$ V $< V_{GS} - V_{Tn} = 4$ V, so **linear**. pMOS: $V_{GS} = 0 - 5 = -5$ V, $V_{DS} = 0.25 - 5 = -4.75$ V; $|V_{DS}| = 4.75 > |V_{GS} - V_{Tp}| = 4$ V, so **saturation**.

Step 2: Equal currents.

$$\frac{k_n}{2}\left[2(V_{in} - V_{Tn})V_{OL} - V_{OL}^2\right] = \frac{k_p}{2}\left(V_{GS,p} - V_{Tp}\right)^2$$

$$\frac{k_n}{2}\left[2(5-1)(0.25) - 0.25^2\right] = \frac{k_p}{2}\left(-5+1\right)^2$$

$$k_n(1.9375) = k_p(16) \quad \Rightarrow \quad \frac{k_n}{k_p} = \frac{16}{1.9375} = 8.258$$

Free shared copy · Nur Mohammad

Step 3: Convert to W/L. Since $k = k'(W/L)$:

$$\frac{(W/L)_n}{(W/L)_p} = \frac{k_n}{k_p} \cdot \frac{\mu_p C_{ox}}{\mu_n C_{ox}} = 8.258 \times \frac{50}{120} = 3.44$$

$$\boxed{\frac{(W/L)_{driver}}{(W/L)_{load}} = 3.44}$$

The driver nMOS must be about 3.44 times "wider" than the pMOS load. **Check:** with $(W/L)_p = 1$ and $(W/L)_n = 3.441$, both currents are 400 µA.

(iii) Output for $V_{in} = 0.25$ V

$V_{in} = 0.25$ V is below $V_{Tn} = 1$ V, so the nMOS is **off** and carries no current. The pMOS is still on (its gate is at 0 V). With no current to carry, its V_{DS} falls to zero, so the output rises all the way to the supply:

$$\boxed{V_{out} = V_{OH} = V_{DD} = 5 \text{ V}}$$

No static current flows in this state; the static current of a pseudo-NMOS gate flows only when its output is low.

3. Keyword Explanation

1. Ratioed logic

↩ back

Logic in which the low output level is set by the ratio of pull-down to pull-up strength, not by switching the pull-up off.

2. Inverter ratio

↩ back

Here $(W/L)_{driver}/(W/L)_{load}$; a larger ratio gives a lower V_{OL} but a slower rising edge and more area.

4. Notes and Remarks

- **Static power:** while the output is low, 400 µA (with $(W/L)_p = 1$) flows from V_{DD} to ground, which is $5 \times 400\ \mu\text{A} = 2$ mW per gate. This is why pseudo-NMOS is used only in special places (wide NOR gates, PLAs).
- **If the nMOS body were not at its source** (for example in a series stack), $V_{SB} > 0$ and γ would raise V_{Tn}; that is why the paper gives γ.

1.(c) Draw the schematic and stick diagram of the circuit implementing the logic $Y = \overline{(A + B + C) \cdot D}$ Marks: 6

1. Theory

1. How a static CMOS gate is built (Core Idea)

Every static CMOS gate has two networks:

- a **pull-down network** of nMOS transistors between the output and ground. It conducts (pulls the output to 0) exactly when the output should be 0;
- a **pull-up network** of pMOS transistors between V_{DD} and the output. It conducts (pulls the output to 1) exactly when the output should be 1.

An nMOS is on when its gate is 1; a pMOS is on when its gate is 0. Two rules build the networks:

- **nMOS in series** make AND, **nMOS in parallel** make OR. The pull-down network implements the function inside the bar of $Y = \overline{f}$.
- The pull-up network is the **dual**: every series connection becomes parallel and every parallel connection becomes series.

So a single CMOS stage always gives an **inverting** function, such as $\overline{(A+B+C)D}$, which is why this one-stage gate is called an AOI-type **complex gate**.

2. What is a stick diagram?

A stick diagram is a simple sketch of the **layout** (the drawing of layers on the chip) that shows which layer goes where, but not exact sizes. Each layer is drawn as a coloured line:

- **green, thick dashed:** n-diffusion (nMOS source and drain)
- **yellow, thick solid:** p-diffusion (pMOS source and drain, inside an n-well)
- **red, thin dash-dot:** polysilicon (gates). Wherever a poly line (red, dash-dot) crosses a diffusion strip (thick), there is a transistor
- **blue, solid:** metal (wires, V_{DD} and GND rails)
- **×:** a contact, a connection between metal and diffusion

3. The Euler path trick

To make a compact layout, all transistors of a network should sit on **one unbroken strip** of diffusion, with the polysilicon gates crossing it in order. This is possible when we can walk through every transistor of the network once without lifting the pen: an **Euler path**. If the same order of inputs works for both the nMOS and the pMOS networks, one vertical poly line per input serves both strips.

2. Answer

Function: $Y = \overline{(A+B+C)\cdot D}$.

- **Pull-down (nMOS):** $(A+B+C)$ is OR, so A, B, C are in **parallel**; "$\cdot D$" is AND, so this group is in **series** with D.
- **Pull-up (pMOS, dual):** A, B, C in **series**, and this chain in **parallel** with D.

That is 4 nMOS + 4 pMOS = 8 transistors.

Free shared copy · Nur Mohammad

Schematic

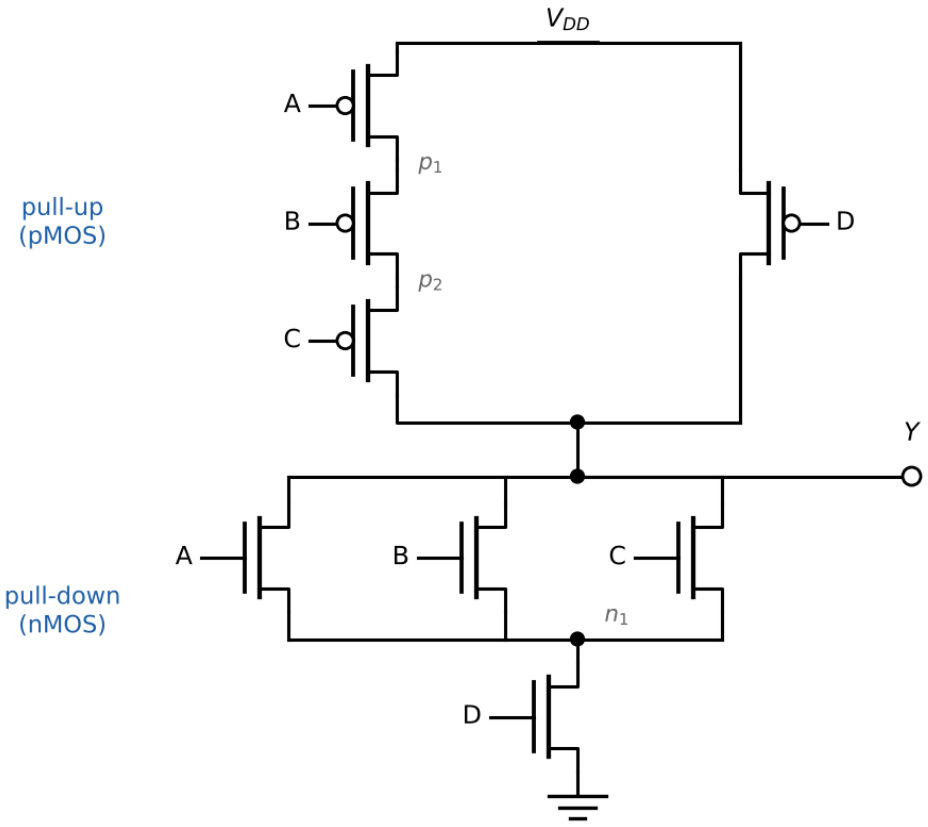


Figure 6: Schematic of Y = NOT((A + B + C)D): pMOS A-B-C series in parallel with D on top; nMOS A, B, C in parallel, in series with D below

Check: if $D = 0$, the pMOS D is on and the nMOS D is off, so $Y = 1$ ✔. If $D = 1$ and any of A, B, C is 1, a pull-down path exists and $Y = 0$ ✔. If $D = 1$ and $A = B = C = 0$, the series pMOS chain conducts and $Y = 1$ ✔.

Stick diagram

Euler path (gate order A, B, C, D):

- nMOS: $Y \xrightarrow{A} n_1 \xrightarrow{B} Y \xrightarrow{C} n_1 \xrightarrow{D} \text{GND}$
- pMOS: $V_{DD} \xrightarrow{A} p_1 \xrightarrow{B} p_2 \xrightarrow{C} Y \xrightarrow{D} V_{DD}$

The same order works for both, so four vertical poly lines are enough.

Free shared copy · Nur Mohammad

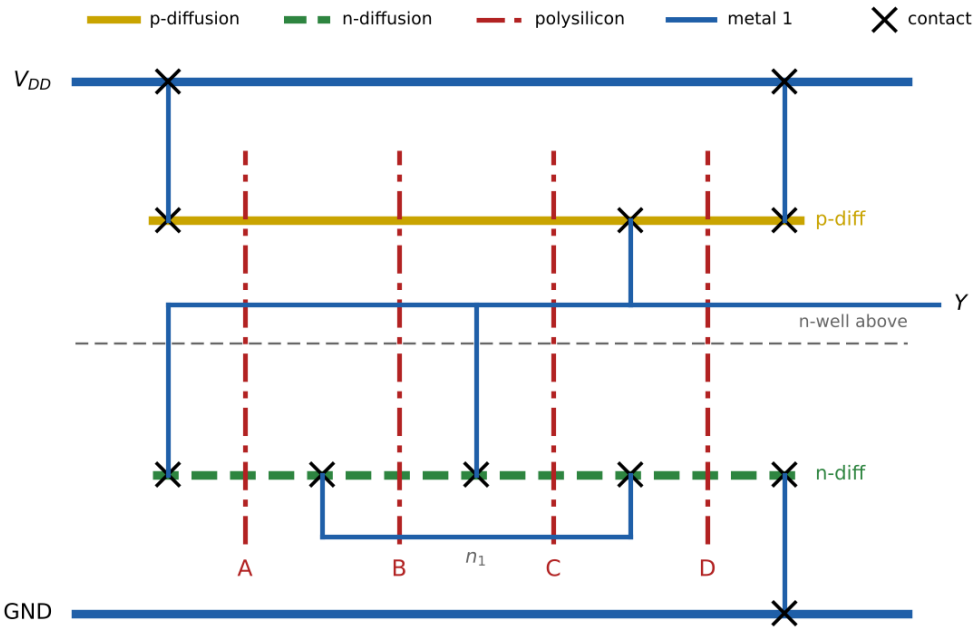


Figure 7: Stick diagram with gate order A, B, C, D: p-diffusion on top, n-diffusion below, metal joins the two Y contacts and the two n1 contacts

Reading the figure. The upper thick solid strip (yellow) is the pMOS row: V_{DD} contacts at both ends, and the output contact between C and D. The lower thick dashed strip (green) is the nMOS row: the output (Y) contacts are at the left end and between B and C; the internal node n_1 contacts (between A–B and between C–D) are joined by a metal strap; the right end goes to GND. One metal line joins all Y contacts and leaves the cell on the right.

3. Keyword Explanation

1. Complex (AOI/OAI) gate

↩ back

One CMOS stage that computes an inverted AND-OR (or OR-AND) expression, instead of several separate gates.

2. Dual network

↩ back

The network obtained by exchanging series and parallel connections; the pMOS network of a static CMOS gate is the dual of its nMOS network.

3. Euler path

↩ back

A path through a graph that uses every edge (transistor) exactly once; it lets one diffusion strip hold all transistors of a network.

4. Notes and Remarks

- **Lambda rules:** a full layout would use design rules in units of λ (for example poly width 2λ, contact $2\lambda \times 2\lambda$). The stick diagram leaves these out on purpose.
- **Shared diffusion:** in the nMOS row the contacts between A–B and C–D are the same node n_1, and the pMOS between C and D shares the Y contact. Sharing diffusion this way lowers capacitance and area.

Free shared copy · Nur Mohammad

1.(d) Estimate the minimum delay of the path from A to B in Figure 1(d) and choose transistor sizes to achieve this delay. The initial NAND2 gate may present a load of 8 λ of transistor width on the input and the output load is equivalent to 45 λ of transistor width. Marks: 5

1. Theory

1. The delay of one gate (Core Idea)

Logical effort is a quick hand method (Weste & Harris) to find the delay of a path of gates and the best size of each gate. It writes the delay of one gate, in units of τ (the delay of an ideal inverter driving an identical inverter with no parasitics), as

$$d = f + p = g\,h + p$$

where

- $f = gh$ is the **stage effort**, the part of the delay due to the load
- g is the **logical effort**: how much worse the gate is than an inverter at delivering current, for the same input capacitance
- $h = C_{out}/C_{in}$ is the **electrical effort** (fanout): load capacitance over the gate's own input capacitance
- p is the **parasitic delay**: the delay of the gate driving nothing, due to its own output capacitance

2. Where do g and p come from?

Size every gate so its worst pull-down and pull-up match a unit inverter (nMOS width 1, pMOS width 2, because holes move about half as fast as electrons). Then the input capacitance of the gate, divided by the inverter's 3, is g.

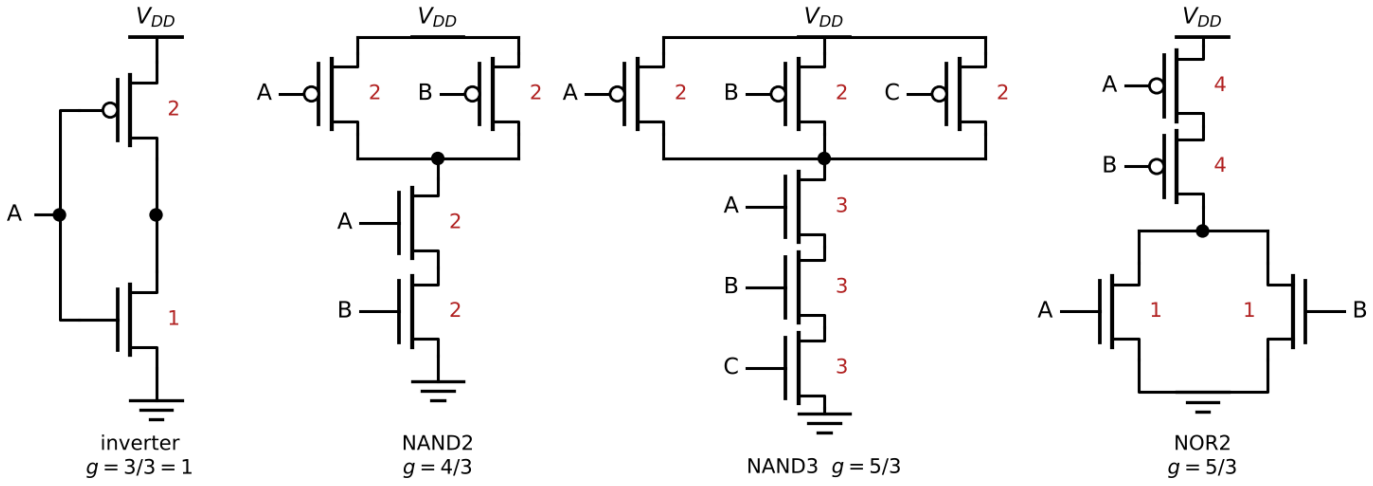


Figure 8: Unit-sized gates (widths labelled in red beside each transistor): an n-input NAND has n series nMOS of width n; a NOR has series pMOS of width 2n

Reading the figure. In the NAND2, the two nMOS are in series, so each is twice as wide (2) to keep the same resistance; the pMOS are in parallel and stay at 2. One input sees $2 + 2 = 4$, so $g = 4/3$. In the NAND3, one input sees $3 + 2 = 5$, so $g = 5/3$. In the NOR2 the pMOS are in series (width 4 each), so one input sees $4 + 1 = 5$ and $g = 5/3$. In general:

Free shared copy · Nur Mohammad

Gate	g	p
Inverter	1	1
NAND-n	$(n+2)/3$	n
NOR-n	$(2n+1)/3$	n

Table 2: Logical effort and parasitic delay

3. A whole path

For a path of N stages, multiply the efforts of the stages:

$$G = \prod g_i, \quad H = \frac{C_{out(path)}}{C_{in(path)}}, \quad B = \prod b_i, \quad F = GBH, \quad P = \sum p_i$$

where b is the **branching effort** of a stage: (capacitance on the path + capacitance off the path) / capacitance on the path. If a gate drives three identical gates and only one is on our path, $b = 3$.
The total delay is smallest when every stage carries the *same* effort:

$$\hat{f} = F^{1/N}, \qquad \hat{D} = N\,F^{1/N} + P$$

Then each gate is sized by working backwards from the load:

$$C_{in,i} = \frac{g_i\, C_{out,i}}{\hat{f}}$$

2. Answer

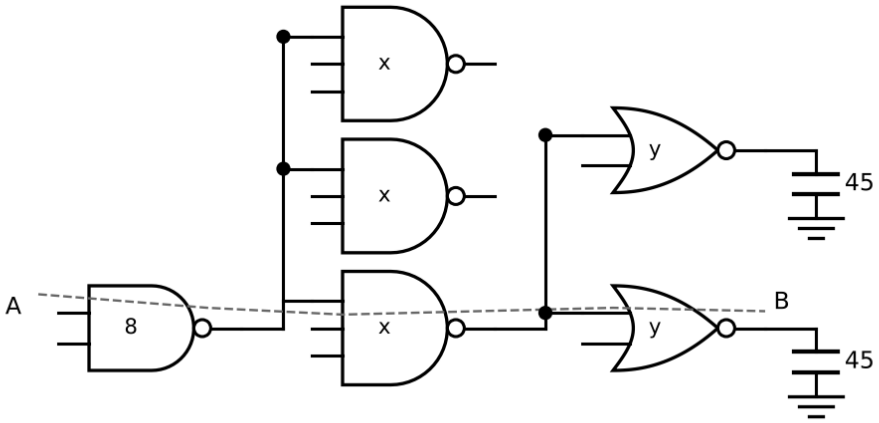


Figure 9: Repeated from the question: Figure 1(d) of the paper (redrawn)

Given: a NAND2 (input capacitance 8) drives three NAND3 gates (x); the NAND3 on the path drives two NOR2 gates (y); the NOR2 on the path drives a load of 45. Units: λ of transistor width.

Step 1: Path efforts

$$G = g_{NAND2}\, g_{NAND3}\, g_{NOR2} = \frac{4}{3}\cdot\frac{5}{3}\cdot\frac{5}{3} = \frac{100}{27} = 3.70$$

$$B = 3 \times 2 = 6, \qquad H = \frac{45}{8} = 5.625$$

$$F = GBH = \frac{100}{27} \times 6 \times 5.625 = 125$$

Free shared copy · Nur Mohammad

Step 2: Best stage effort and minimum delay

$$\hat{f} = F^{1/3} = 125^{1/3} = 5$$

$$P = p_{NAND2} + p_{NAND3} + p_{NOR2} = 2 + 3 + 2 = 7$$

$$\hat{D} = 3\hat{f} + P = 3(5) + 7 = 22\ \tau$$

$$\boxed{\hat{D} = 22\,\tau \approx 4.4 \text{ FO4 delays}}$$

(One FO4 inverter delay is $4 + 1 = 5\tau$.)

Step 3: Gate sizes, from the load backwards

$$y = \frac{g_{NOR2} \times 45}{\hat{f}} = \frac{(5/3)(45)}{5} = 15$$

$$x = \frac{g_{NAND3} \times (2y)}{\hat{f}} = \frac{(5/3)(30)}{5} = 10$$

Check: the NAND2 input is $\dfrac{(4/3)(3x)}{5} = \dfrac{(4/3)(30)}{5} = 8$ ✔, which is the given 8.

$$\boxed{x = 10\lambda, \quad y = 15\lambda}$$

Step 4: Transistor widths

Each gate is the unit gate scaled to its input capacitance:

Gate	Input capacitance	Unit gate (n, p)	nMOS width	pMOS width
NAND2	8	2, 2 (sum 4)	4λ	4λ
NAND3 (x)	10	3, 2 (sum 5)	6λ	4λ
NOR2 (y)	15	1, 4 (sum 5)	3λ	12λ

Table 3: Transistor widths for the minimum delay

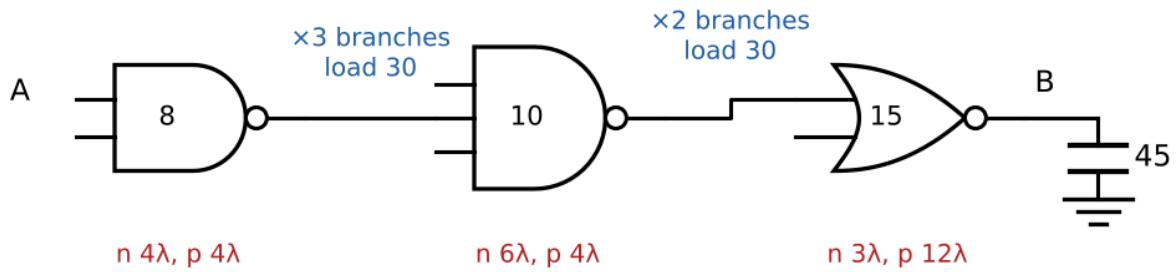


Figure 10: The path from A to B with the chosen input capacitances and transistor widths

Final Answer: minimum delay $\hat{D} = 22\tau$ (about 4.4 FO4), with $x = 10\lambda$ and $y = 15\lambda$ of input capacitance per input; each stage then has effort 5.

Free shared copy · Nur Mohammad

3. Keyword Explanation

1. FO4 delay

↩ back

The delay of an inverter driving four copies of itself: $d = 1 \cdot 4 + 1 = 5\tau$. It is a process-independent yardstick of speed.

2. Branching effort

↩ back

The factor by which a stage's output current is split among gates off the path; it multiplies the effort.

4. Notes and Remarks

- **Every stage has the same effort:** NAND2: $\frac{4}{3} \cdot \frac{30}{8} = 5$; NAND3: $\frac{5}{3} \cdot \frac{30}{10} = 5$; NOR2: $\frac{5}{3} \cdot \frac{45}{15} = 5$. This is the sign of an optimally sized path.
- **Best number of stages:** a stage effort near 4 is ideal. Here $\hat{f} = 5$ with 3 stages, which is close; adding inverters would not help much.

1.(e) Differentiate between the diffusion process and the ion implantation process in semiconductor fabrication. Marks: 2

1. Theory

1. What is doping? (Core Idea)

Pure silicon conducts poorly. To make n-type or p-type regions (sources, drains, wells), we add a small amount of impurity atoms: phosphorus or arsenic for n-type, boron for p-type. This is **doping**. There are two ways to put the atoms into the wafer.

2. Diffusion

The wafer is heated to about 900–1200 °C in a furnace with a gas (or a deposited layer) that contains the dopant. At this temperature the dopant atoms wander into the silicon, from high concentration at the surface to low concentration inside, just like a drop of ink spreads in water. The depth is set by the **time and temperature**.

3. Ion implantation

Dopant atoms are ionised, accelerated by a high voltage (10–200 keV) and shot into the wafer like bullets. They stop at a depth set by their **energy**. The number of ions (the **dose**) is counted exactly by measuring the beam current. The crystal is damaged by the impacts, so the wafer is then **annealed** (heated briefly) to repair it and activate the dopants.

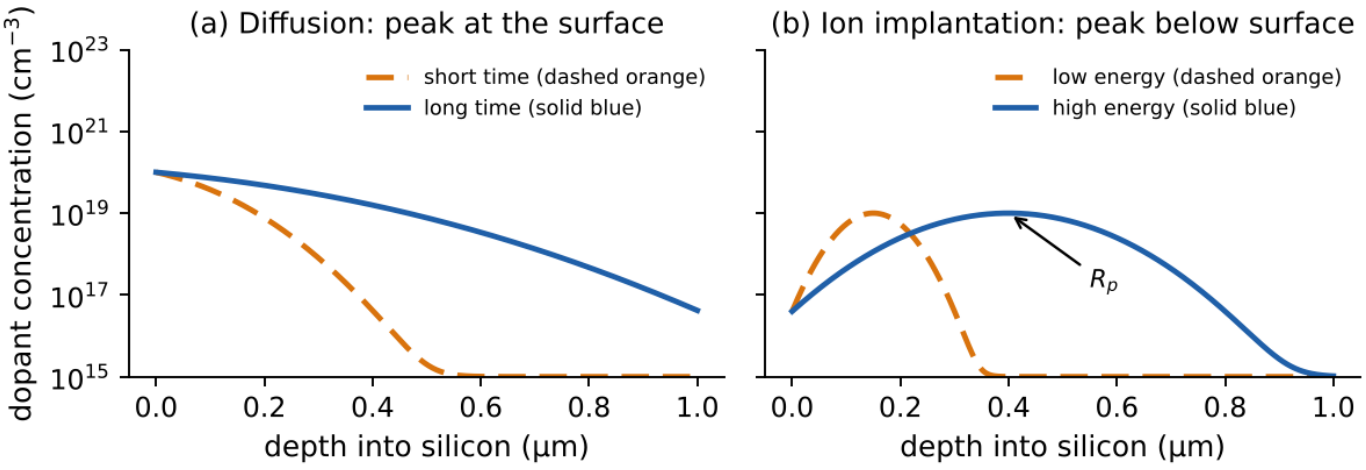


Figure 11: Dopant profiles: (a) diffusion, peak at the surface; (b) ion implantation, peak below the surface at the projected range Rp

Reading the figure. In diffusion (a) the concentration is highest at the surface and falls with depth; a longer time (solid blue) pushes the dopant deeper. In implantation (b) the profile is a bell shape

whose peak lies below the surface, at the **projected range** R_p; higher energy (solid blue) puts the peak deeper.

2. Answer

Point	Diffusion	Ion implantation
How	Dopant atoms move in by heat (high temperature, 900–1200 °C)	Ionised dopant atoms are accelerated and shot in (near room temperature)
Depth set by	Time and temperature	Ion energy
Profile	Peak at the surface, falls with depth	Peak below the surface (at R_p)
Dose control	Less precise	Very precise (beam current is measured)
Lateral spread	Large (spreads sideways under the mask)	Small (ions travel straight)
Mask	Needs oxide (high temperature)	Photoresist, oxide or nitride
Crystal damage	None	Damage, needs annealing
Cost	Simple, many wafers at once	Expensive equipment

Table 4: Diffusion versus ion implantation

3. Notes and Remarks

- Modern CMOS uses ion implantation for almost all doping steps (wells, threshold adjust, source and drain) because of its precise dose and shallow, sharp junctions; a short diffusion (drive-in) during annealing still happens.

4. Final Concise Answer

(a) The gate–oxide–body sandwich is a capacitor $C_0 = C_{ox}WL$. On p-type silicon it is C_{ox} in **accumulation** ($V_g < 0$), falls in **depletion** ($0 < V_g < V_t$) because the depletion layer adds in series, and returns to C_{ox} in **inversion** ($V_g > V_t$, low frequency or in a MOSFET). In a MOSFET: cutoff $C_{gb} \approx C_0$; linear $C_{gs} = C_{gd} = C_0/2$; saturation $C_{gs} = \frac{2}{3}C_0$, $C_{gd} = 0$.

(b) (i) nMOS driver with pMOS load whose gate is grounded; both bodies tied to their sources, so $V_{SB} = 0$ and γ has no effect. (ii) nMOS linear, pMOS saturated: $k_n/k_p = 16/1.9375 = 8.26$, so $(W/L)_n/(W/L)_p = 8.26 \times 50/120 = 3.44$. (iii) $V_{in} = 0.25$ V $< V_{Tn}$: nMOS off, $V_{out} = V_{DD} = 5$ V.

(c) nMOS: A, B, C in parallel, in series with D; pMOS: A–B–C in series, in parallel with D. Stick diagram with the common Euler path A, B, C, D.

(d) $G = 100/27$, $B = 6$, $H = 45/8$, $F = 125$, $\hat{f} = 5$, $P = 7$: $\hat{D} = 22\tau$ (4.4 FO4). Sizes $x = 10\lambda$, $y = 15\lambda$; NAND2 n 4/p 4, NAND3 n 6/p 4, NOR2 n 3/p 12.

(e) Diffusion: dopant enters by heat, peak at the surface, more sideways spread, less precise. Ion implantation: accelerated ions, peak below the surface, precise dose and depth, low temperature, needs annealing.

Free shared copy · Nur Mohammad

2. (a) For the RC circuit shown in Figure 2(a), compute the Elmore delays from input node to node 7 and from input node to node 5 of this circuit in terms of R and C component values. Marks: 5

   (b) Briefly explain what happens to the transfer characteristics and noise margins when beta ratio is increased in a CMOS inverter pair. Marks: 6

   (c) Design an 8-input AND gate using FO4 in different configurations. Determine which configuration provides the fastest performance. Marks: 7

   (d) Write short note on –i) Static power dissipation ii) Dynamic power dissipation. Marks: 4

   (e) What is domino gate? Draw the circuit diagram of a domino OR gate. Marks: 3

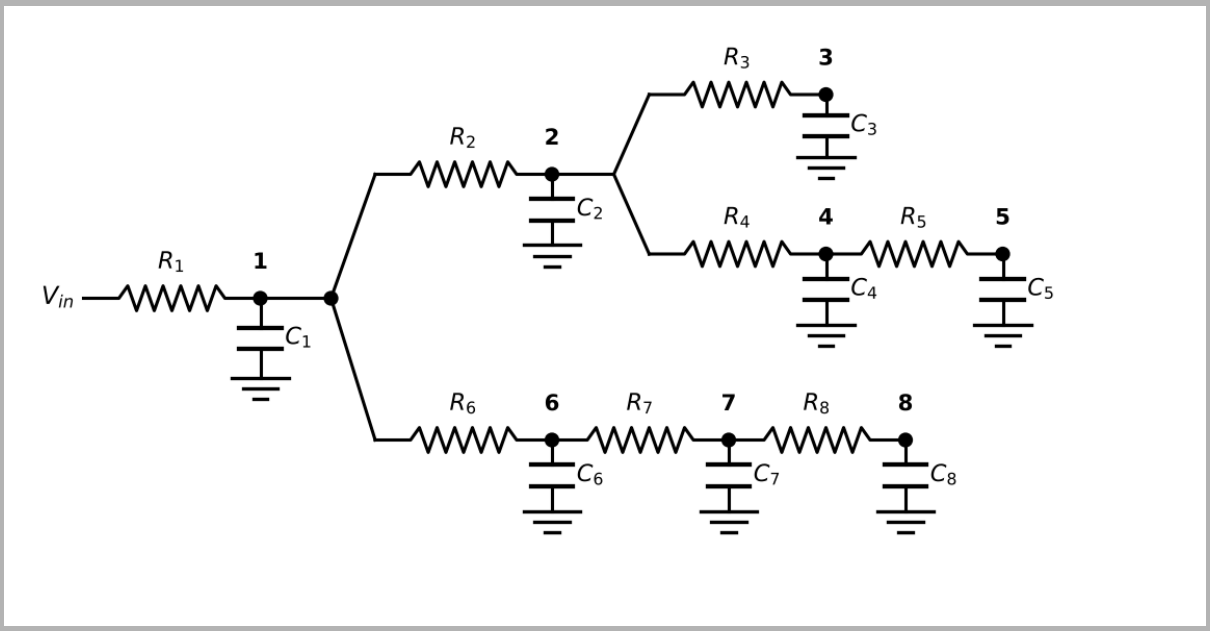


Figure 12: Figure 2(a) of the paper (redrawn)

Free shared copy · Nur Mohammad

2.(a) For the RC circuit shown in Figure 2(a), compute the Elmore delays from input node to node 7 and from input node to node 5 of this circuit in terms of R and C component values. Marks: 5

1. Theory

1. Why do we model wires and transistors as R and C? (Core Idea)

An "on" transistor behaves roughly like a **resistor**, and every wire, gate input and diffusion region has **capacitance** to ground. So the path a signal takes on a chip looks like a network of resistors with capacitors hanging from the nodes. When the input switches, each capacitor must charge through the resistors in front of it, and this takes time: the **delay**.

2. One resistor and one capacitor

For a single R charging a single C, the output rises as $1 - e^{-t/RC}$. The time constant RC is a good measure of its delay (the 50 % point is $0.69RC$).

3. Many resistors and capacitors: the Elmore delay

A network with many R and C has many time constants and no simple exact answer. Elmore's idea is to approximate it by one time constant, which is easy to compute for an **RC tree** (a network with no loops, all capacitors to ground):

$$t_{D,i} = \sum_k R_{ik}\, C_k$$

where

- $t_{D,i}$ is the Elmore delay from the input to node i
- C_k is the capacitor at node k; the sum runs over *every* capacitor in the tree
- R_{ik} is the resistance that the path input $\to i$ and the path input $\to k$ have **in common**

In words: each capacitor is charged by the current that flows through the shared resistors, so it adds (shared resistance) $\times$ (its capacitance) to the delay of node i.

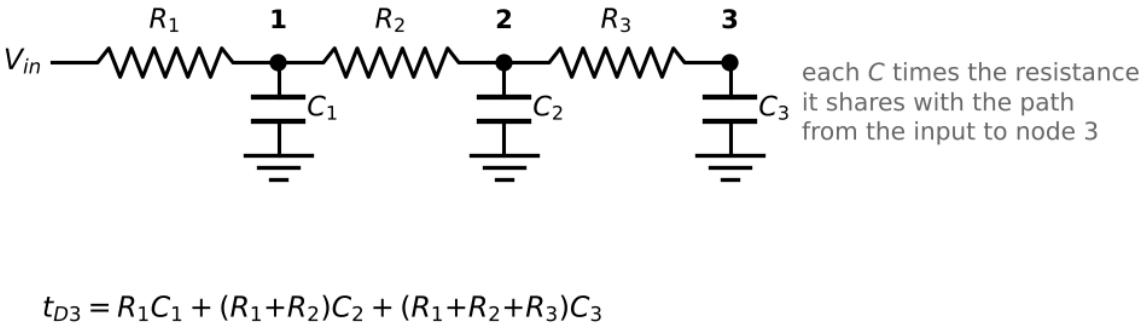


Figure 13: Elmore delay of a simple RC ladder

Reading the figure. For node 3 of a ladder, C_1 shares only R_1 with the path to node 3; C_2 shares $R_1 + R_2$; C_3 shares all three. A capacitor on a **side branch** shares only the resistance up to the point where its branch leaves the path.

Free shared copy · Nur Mohammad

2. Answer

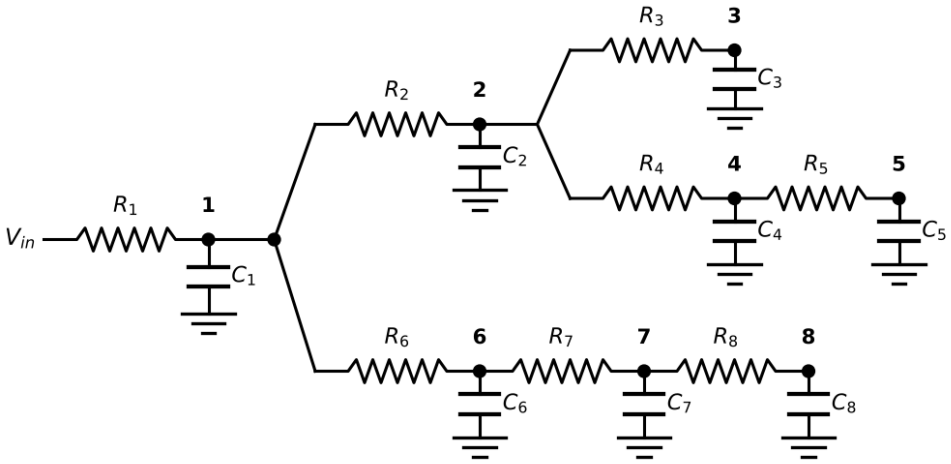


Figure 14: Repeated from the question: Figure 2(a) of the paper (redrawn)

Method: for each capacitor, find the resistance it shares with the path from V_{in} to the target node, multiply, and add.

Step 1: Delay to node 7

Path from the input to node 7: $R_1 \to R_6 \to R_7$.

- C_1, C_2, C_3, C_4, C_5: they share only R_1 with this path (their branch leaves at node 1)
- C_6: shares $R_1 + R_6$
- C_7 and C_8: share $R_1 + R_6 + R_7$ (C_8 hangs beyond node 7, so it shares the whole path to node 7)

$$t_{D7} = R_1C_1 + R_1(C_2 + C_3 + C_4 + C_5) + (R_1 + R_6)C_6 + (R_1 + R_6 + R_7)(C_7 + C_8)$$

Collecting by resistor:

$$\boxed{t_{D7} = R_1(C_1 + C_2 + C_3 + C_4 + C_5 + C_6 + C_7 + C_8) + R_6(C_6 + C_7 + C_8) + R_7(C_7 + C_8)}$$

Step 2: Delay to node 5

Path from the input to node 5: $R_1 \to R_2 \to R_4 \to R_5$.

- C_1, C_6, C_7, C_8: share only R_1
- C_2 and C_3: share $R_1 + R_2$ (C_3's branch leaves after node 2)
- C_4: shares $R_1 + R_2 + R_4$
- C_5: shares the whole path $R_1 + R_2 + R_4 + R_5$

$$t_{D5} = R_1(C_1 + C_6 + C_7 + C_8) + (R_1 + R_2)(C_2 + C_3) + (R_1 + R_2 + R_4)C_4 + (R_1 + R_2 + R_4 + R_5)C_5$$

$$\boxed{t_{D5} = R_1(C_1 + \cdots + C_8) + R_2(C_2 + C_3 + C_4 + C_5) + R_4(C_4 + C_5) + R_5C_5}$$

Check: with every R and every C equal, $t_{D7} = 8 + 3 + 2 = 13RC$ and $t_{D5} = 8 + 4 + 2 + 1 = 15RC$. Node 5 is further from the input, so its delay is larger, as expected.

3. Keyword Explanation

1. RC tree

↩ back

A network of resistors with capacitors from each node to ground, with exactly one resistive path from the input to every node (no loops).

Free shared copy · Nur Mohammad

2. Shared resistance R_{ik}

↩ back

The sum of resistances lying on both the path input $\to i$ and the path input $\to k$.

4. Notes and Remarks

- **Equivalent form:** $t_{D,i} = \sum_j R_j \times$ (total capacitance downstream of R_j), summed over the resistors on the path to i. This is the "collected" form in the boxes.
- The Elmore delay is an upper bound of the 50 % delay and is accurate enough for comparing designs.

2.(b) Briefly explain what happens to the transfer characteristics and noise margins when beta ratio is increased in a CMOS inverter pair. Marks: 6

1. Theory

1. The voltage transfer characteristic (Core Idea)

The **voltage transfer characteristic** (VTC) of an inverter is the plot of V_{out} against V_{in} for slowly changing input. For a CMOS inverter it stays at V_{DD} for a low input, falls steeply in the middle and stays at 0 V for a high input. Two numbers describe it:

- **Switching threshold** V_M**:** the input voltage at which $V_{out} = V_{in}$ (the curve crosses the 45° line).
- **Noise margins:** how much noise a logic level can carry before the next gate misreads it.

2. Noise margins

The **unity-gain points** V_{IL} and V_{IH} are the two inputs where the slope of the VTC is -1. Below V_{IL} the input is read as a clean 0, above V_{IH} as a clean 1, and in between the gate amplifies noise. With CMOS outputs at the rails ($V_{OL} = 0$, $V_{OH} = V_{DD}$):

$$NM_L = V_{IL} - V_{OL}, \qquad NM_H = V_{OH} - V_{IH}$$

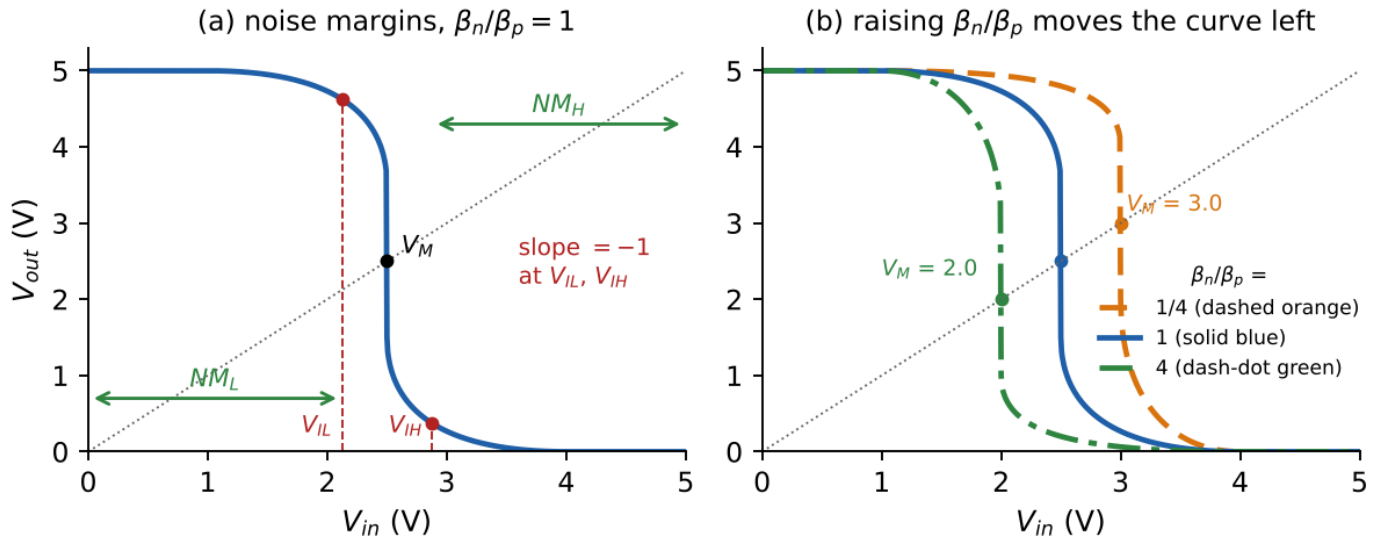


Figure 15: (a) Unity-gain points and noise margins of a symmetric inverter; (b) the VTC for three beta ratios

Reading the figure. In (a) the red dots are the unity-gain points $V_{IL} = 2.125$ V and $V_{IH} = 2.875$ V ($V_{DD} = 5$ V, $V_t = 1$ V), and the green arrows are the two noise margins, both 2.125 V. In (b), a larger β_n/β_p (dash-dot green) moves the whole falling edge to the left; a smaller one (dashed orange) moves it to the right.

Free shared copy · Nur Mohammad

3. What is the beta ratio?

Each transistor has a gain factor $\beta = \mu C_{ox} W/L$. The **beta ratio** is β_n/β_p. Since $\mu_n \approx 2\mu_p$, an inverter with $W_p = 2W_n$ has $\beta_n/\beta_p \approx 1$ (symmetric). Raising β_n/β_p means a relatively **stronger nMOS** (or weaker pMOS).

4. The switching threshold

At V_M both transistors are in saturation with equal currents:

$$\frac{\beta_n}{2}(V_M - V_{tn})^2 = \frac{\beta_p}{2}(V_{DD} - V_M - |V_{tp}|)^2$$

$$V_M = \frac{V_{DD} - |V_{tp}| + V_{tn}\sqrt{\beta_n/\beta_p}}{1 + \sqrt{\beta_n/\beta_p}}$$

A larger β_n/β_p makes the denominator grow faster than the numerator, so V_M goes down.

2. Answer

Effect of increasing β_n/β_p (stronger nMOS):

1. **Transfer characteristic:** the nMOS can pull the output down at a lower input, so the falling edge of the VTC **shifts to the left** and the switching threshold V_M **decreases** (formula above). The shape of the steep part hardly changes.
2. **Low noise margin $NM_L = V_{IL}$ decreases:** V_{IL} moves left with the curve, so a smaller noise voltage on a logic 0 can already flip the gate.
3. **High noise margin $NM_H = V_{DD} - V_{IH}$ increases:** V_{IH} also moves left, so a logic 1 can tolerate more noise.
4. The margins are equal (largest worst-case margin) at $\beta_n/\beta_p \approx 1$ ($V_M = V_{DD}/2$). Skewing the ratio trades one margin for the other.

Values from the long-channel model ($V_{DD} = 5$ V, $|V_t| = 1$ V):

β_n/β_p	V_M (V)	V_{IL} (V)	V_{IH} (V)	NM_L (V)	NM_H (V)
1/4	3.00	2.78	3.49	2.78	1.51
1	2.50	2.13	2.88	2.13	2.13
4	2.00	1.51	2.22	1.51	2.78

Table 5: Switching threshold and noise margins against beta ratio

Practical Note: a skewed ratio is used on purpose in skewed gates (see 3(d)) to speed up one edge; the price is a smaller noise margin on the other side.

3. Keyword Explanation

1. Switching threshold V_M

↩ back

The input voltage at which the inverter output equals its input; the "decision point" of the gate.

2. Unity-gain point

↩ back

A point of the VTC with slope -1; it marks the edge of the region where the gate amplifies noise.

4. Notes and Remarks

- **Weak dependence:** V_M changes only with $\sqrt{\beta_n/\beta_p}$, so a factor of 4 in the ratio moves V_M by only 0.5 V here. The VTC is robust against process variations for this reason.
- **The symmetric case:** for $\beta_n = \beta_p$ and $V_{tn} = |V_{tp}|$, $V_{IL} = (3V_{DD} + 2V_t)/8 = 2.125$ V, matching the table.

2.(c) Design an 8-input AND gate using FO4 in different configurations. Determine which configuration provides the fastest performance. Marks: 7

1. Theory

1. Why not one big gate? (Core Idea)

An 8-input AND could be one NAND8 followed by an inverter. But a NAND8 has 8 nMOS in series, so each must be 8 times wider; its logical effort $(8+2)/3 = 3.33$ and its parasitic delay 8 are large. Splitting the function into smaller gates lowers the effort per stage, but adds stages. Logical effort tells us which split is fastest.

The method, $\hat{D} = NF^{1/N} + P$, and the values of g and p, are built up in 1(d).

2. "Using FO4"

An **FO4** load is a load of four times the gate's own input capacitance, the standard load of a gate inside a real path. So the AND gate is designed for an electrical effort

$$H = \frac{C_{out}}{C_{in}} = 4$$

and delays are also given in FO4 units (1 FO4 $= 5\tau$).

3. Using De Morgan to split an AND

$A_1A_2\cdots A_8$ can be built as:

- NAND8 then INV
- two NAND4s, then a NOR2: $\overline{\overline{A_1..A_4} + \overline{A_5..A_8}} = A_1..A_8$
- four NAND2s, then a NOR4
- NAND2s, NOR2s, NAND2 and INV in four levels, and so on

There is no branching ($B = 1$), so $F = GH = 4G$.

2. Answer

Given: $H = 4$ (FO4 load), $B = 1$. For each design: $G = \prod g$, $F = 4G$, $P = \sum p$, $\hat{D} = NF^{1/N} + P$.

Free shared copy · Nur Mohammad

Design	N	G	F	P	$\hat{f}$	$\hat{D}$ (τ)	$\hat{D}$ (FO4)
NAND8 + INV	2	$\frac{10}{3} \cdot 1 = 3.33$	13.33	9	3.65	16.30	3.26
NAND4 + NOR2	2	$2 \cdot \frac{5}{3} = 3.33$	13.33	6	3.65	**13.30**	**2.66**
NAND2 + NOR4	2	$\frac{4}{3} \cdot 3 = 4$	16	6	4.00	14.00	2.80
NAND2 + NOR2 + NAND2 + INV	4	2.96	11.85	7	1.86	14.42	2.88
NAND4 + INV + NAND2 + INV	4	2.67	10.67	8	1.81	15.23	3.05
(NAND2 + INV) × 3	6	2.37	9.48	9	1.46	17.73	3.55

Table 6: 8-input AND designs with an FO4 load

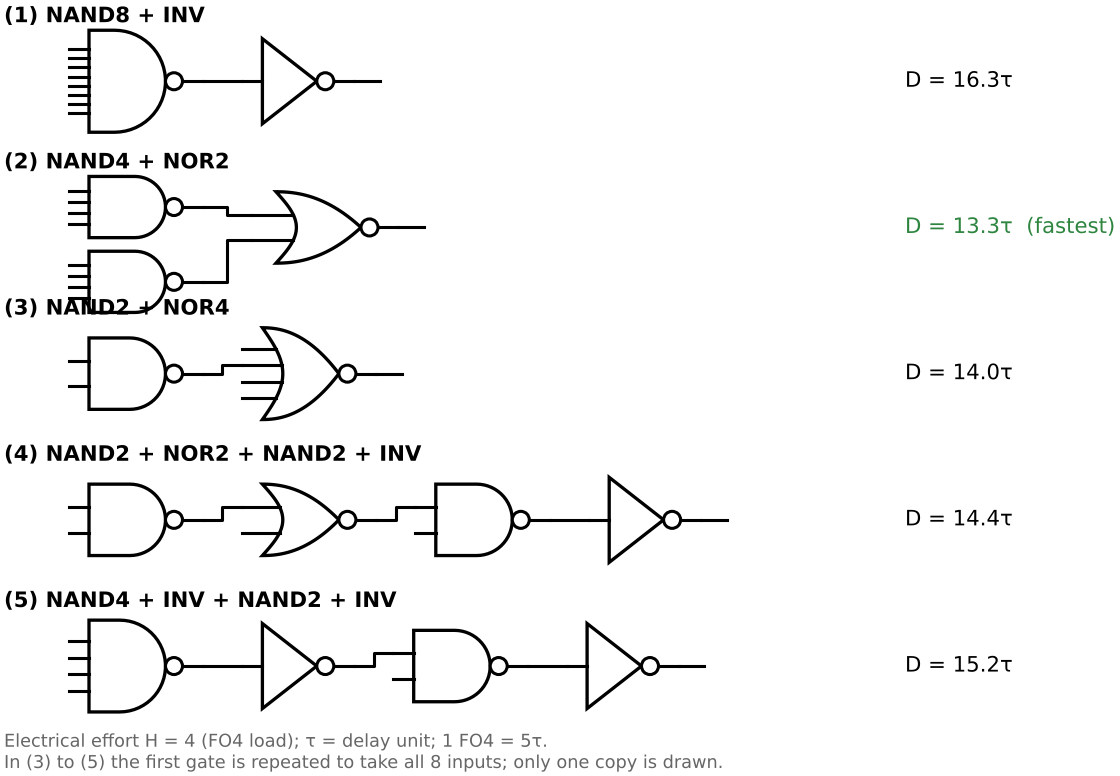


Figure 16: Five of the configurations with their delays

Working for the fastest design (NAND4 + NOR2):

$$G = g_{NAND4}\, g_{NOR2} = \frac{4+2}{3} \cdot \frac{2 \cdot 2 + 1}{3} = 2 \times \frac{5}{3} = \frac{10}{3}$$

$$F = GH = \frac{10}{3} \times 4 = 13.33, \qquad \hat{f} = \sqrt{13.33} = 3.65$$

$$P = 4 + 2 = 6, \qquad \hat{D} = 2(3.65) + 6 = 13.3\,\tau$$

$$\boxed{\text{NAND4 + NOR2 is fastest: } \hat{D} = 13.3\,\tau \approx 2.66 \text{ FO4}}$$

Why: with a load of only 4, the path effort F is small (about 10 to 16). The best number of stages is $\log_4 F \approx 1.9$, so **two stages** are right. Among the two-stage designs, NAND4 + NOR2 has the lowest parasitic delay (6) and the same G as NAND8 + INV, whose NAND8 costs 8 units of parasitic delay. Four- and six-stage designs waste time in parasitic delay because the load is too small to need the extra gain.

Free shared copy · Nur Mohammad

3. Keyword Explanation

1. Electrical effort H

↩ back

Output load capacitance divided by the input capacitance of the path.

2. Parasitic delay p

↩ back

The delay of a gate driving no load, caused by the diffusion capacitance at its own output; for NAND-n and NOR-n, $p \approx n$.

4. Notes and Remarks

- **With a large load the answer changes:** for $H = 12$ (the Weste & Harris example) the same designs give 21.7, 18.7, 19.9, 16.8 (NAND2 + NOR2 + NAND2 + INV, fastest) and 17.5 τ. A larger F needs more stages, so the four-stage design wins there.

2.(d) Write short note on –i) Static power dissipation ii) Dynamic power dissipation. Marks: 4

1. Theory

1. Where does a chip's power go? (Core Idea)

Power is energy per second taken from the supply: $P = V_{DD}\, I_{DD}$. In a CMOS chip the current I_{DD} has two kinds of parts:

- currents that flow **all the time**, even when nothing switches: they give **static power**;
- currents that flow **only when a signal switches**: they give **dynamic power**.

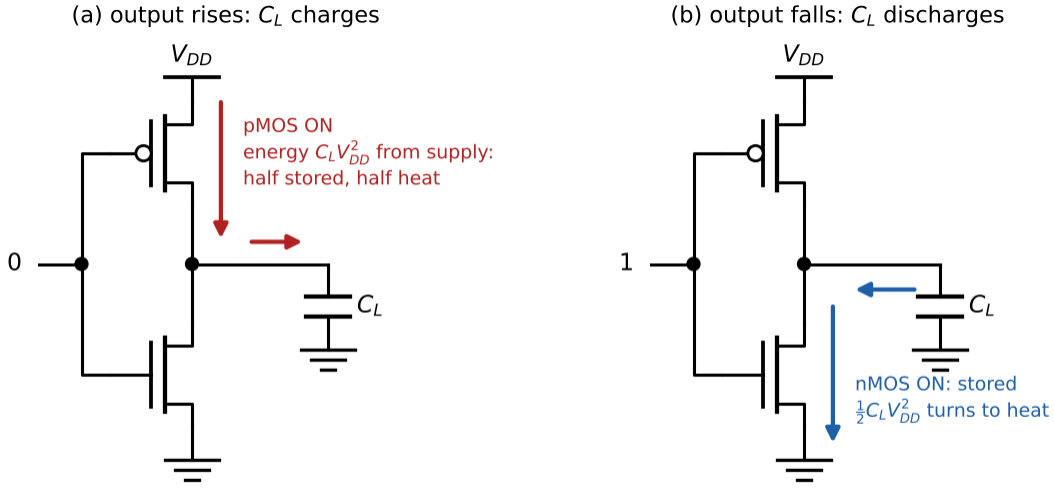


Figure 17: Dynamic power: (a) charging the load capacitance through the pMOS, (b) discharging it through the nMOS

Reading the figure. In (a) the input falls, the pMOS turns on and current flows from V_{DD} into the load capacitance C_L (red arrows). In (b) the input rises, the nMOS turns on and C_L empties into ground (blue arrows). Each full cycle (charge then discharge) takes energy $C_L V_{DD}^2$ from the supply: half is lost as heat in the pMOS while charging, the other half is stored in C_L and lost in the nMOS while discharging.

2. Answer

i) Static power dissipation

Static power is drawn even when no input changes:

$$P_{static} = I_{static} V_{DD}$$

Its sources:

- **Subthreshold leakage:** an "off" transistor still passes a small current, which grows quickly as V_t is lowered. This is the main part in modern processes.
- **Gate leakage:** electrons tunnel through a very thin gate oxide.
- **Junction leakage:** the reverse-biased source/drain diodes pass a small current.
- **Contention current:** in ratioed circuits such as pseudo-NMOS (1(b)), the pull-up and pull-down are both on while the output is low, so a large d.c. current flows.

An ideal static CMOS gate has no d.c. path from V_{DD} to ground, so its static power is only leakage. Ways to reduce it: higher-V_t transistors where speed is not needed, power gating (switching off idle blocks), lower V_{DD}.

ii) Dynamic power dissipation

Dynamic power is drawn when nodes switch:

$$P_{dynamic} = \alpha C_L V_{DD}^2 f$$

where α is the **activity factor** (the fraction of clock cycles in which the node makes a 0→1 transition), C_L the switched capacitance and f the clock frequency. Its sources:

- **Switching power:** charging and discharging the load capacitances (figure above), the main part.
- **Short-circuit power:** while the input passes through the middle, both transistors are briefly on and a current flows straight from V_{DD} to ground; it is small if input edges are fast.

Ways to reduce it: lower V_{DD} (strongest, since $P \propto V_{DD}^2$; derived in 3(c)), clock gating (lower α), smaller capacitance, lower f.

3. Keyword Explanation

1. Activity factor α

↩ back

The probability that a node rises from 0 to 1 in a clock cycle; 1 for a clock, about 0.1 for typical logic.

2. Subthreshold leakage

↩ back

The current of a transistor whose V_{GS} is below V_t; it falls by about 10 times for every 100 mV of gate voltage below V_t.

2.(e) What is domino gate? Draw the circuit diagram of a domino OR gate. Marks: 3

1. Theory

1. Dynamic logic (Core Idea)

A static CMOS gate needs both a pull-up and a pull-down network, so every input drives two transistors, one of them a slow pMOS. **Dynamic logic** removes the pMOS network. It uses a clock ϕ and works in two phases:

- **Precharge** ($\phi = 0$)**:** a single pMOS charges the output node X to V_{DD}.
- **Evaluate** ($\phi = 1$)**:** the pMOS turns off and an nMOS "foot" transistor turns on. If the nMOS network conducts, X discharges to 0; otherwise X stays high, held only by its capacitance.

The gate is fast and small, but has a problem: once X has fallen, it cannot rise again until the next precharge. If one dynamic gate drives another, the second gate may start evaluating while its input is still high from precharge and wrongly discharge.

Free shared copy · Nur Mohammad

2. Domino logic

The fix is to put a static inverter after every dynamic gate. During precharge X is high, so the inverter output is **low**. During evaluation the output can only **rise** ($0 \to 1$). So all inputs to the next gate start low and can only rise: a rising output makes the next gate fall in turn, like a row of falling dominoes. This is **domino logic**.

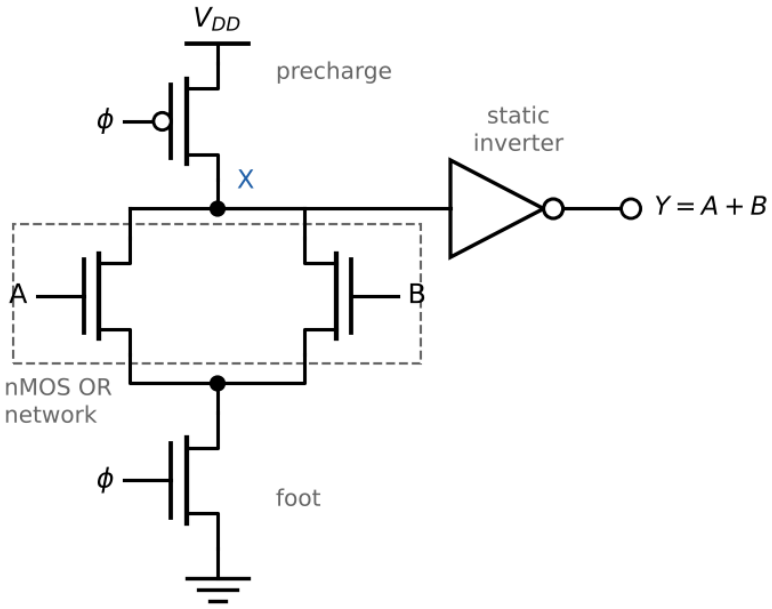


Figure 18: Domino OR gate: precharge pMOS, parallel nMOS for A and B, clocked foot, and the static output inverter

Reading the figure. A and B are nMOS in parallel (OR). During evaluation, if A or B is 1, X discharges, and the inverter makes $Y = 1$. So $Y = A + B$: domino gates are **non-inverting**.

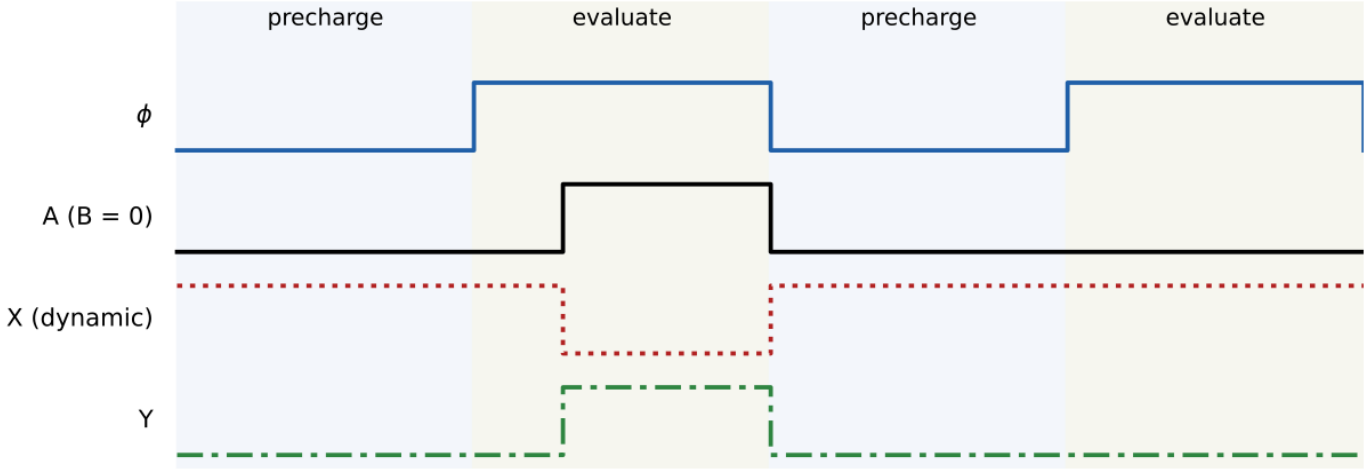


Figure 19: Waveforms: precharge sets X high and Y low; in evaluate, A rising discharges X and raises Y

Free shared copy · Nur Mohammad

2. Answer

Domino gate: a dynamic CMOS gate (clocked precharge pMOS, nMOS logic network, clocked nMOS foot) followed by a **static CMOS inverter**. In the precharge phase ($\phi = 0$) the dynamic node X is charged to V_{DD} and the output is 0; in the evaluate phase ($\phi = 1$) X discharges if the nMOS network conducts, and the output rises to 1. Outputs only make $0 \to 1$ transitions during evaluation, so domino gates can be cascaded safely. The gate is non-inverting.

Domino OR gate ($Y = A + B$): figure above. Precharge pMOS (gate ϕ) from V_{DD} to X; nMOS A and B in parallel from X to the foot; foot nMOS (gate ϕ) to ground; inverter from X to Y.

3. Keyword Explanation

1. Precharge and evaluate

↩ back

The two clock phases of dynamic logic: first set the node high, then conditionally discharge it.

2. Monotonic

↩ back

A signal that changes in one direction only during a phase (here: only rises), which domino requires of its inputs.

4. Notes and Remarks

- **Keeper:** a weak pMOS from V_{DD} to X, driven by the output, is usually added so that leakage cannot slowly discharge X when it should stay high.
- **Limitation:** domino gates can only build non-inverting functions (AND, OR); inverted signals need dual-rail domino.

5. Final Concise Answer

(a) $t_{D7} = R_1(C_1 + \cdots + C_8) + R_6(C_6 + C_7 + C_8) + R_7(C_7 + C_8)$; $t_{D5} = R_1(C_1 + \cdots + C_8) + R_2(C_2 + C_3 + C_4 + C_5) + R_4(C_4 + C_5) + R_5C_5$.

(b) Increasing β_n/β_p strengthens the nMOS: the VTC moves left, V_M falls ($V_M = \frac{V_{DD} - |V_{tp}| + V_{tn}\sqrt{r}}{1+\sqrt{r}}$), NM_L falls and NM_H rises; they are equal at $\beta_n/\beta_p = 1$.

(c) With $H = 4$: NAND8+INV 16.3τ, **NAND4+NOR2 13.3τ (2.66 FO4), fastest**, NAND2+NOR4 14.0τ, four-stage designs 14.4–15.2τ. Two stages suit the small path effort.

(d) Static: $P = I_{static}V_{DD}$ from subthreshold, gate and junction leakage and from contention in ratioed logic. Dynamic: $P = \alpha C_L V_{DD}^2 f$ from charging load capacitance, plus short-circuit current during switching.

(e) A domino gate is a dynamic gate (precharge pMOS, nMOS network, clocked foot) followed by a static inverter; it is non-inverting and safely cascadable. Domino OR: A and B nMOS in parallel under the precharged node X, inverter gives $Y = A + B$.

Free shared copy · Nur Mohammad

3. (a) You have to design a car alarm system in which the alarm will ring (Y= 1) if anyone of the following condition is satisfied.

   (i) The key is in (K= 1), The Seat belt is fastened (B=1) and the door is open (D=0) or
   (ii) The key is in, Door is closed and Seat belt is not fastened.

   Write the Truth table, the Boolean equations and show the implementation of the circuit in (i) static CMOS, (ii) NMOS, (iii) Pseudo NMOS, (iv) Footed dynamic CMOS and (v) Un-footed dynamic CMOS. Marks: 6

   (b) For the logic function $Y = \overline{(A + BC) \cdot D}$, choose transistor widths to achieve equal rise and fall resistance as a unit inverter. Assume, $\mu_n = 3\mu_p$ Marks: 4

   (c) Show that the dynamic power dissipation of a CMOS inverter varies directly with square of the power supply voltage whereas the delay of the circuit varies inversely with the power supply voltage. Marks: 8

   (d) Explain the inverter width sizing for favoring the rising edge, falling edge, or achieving an unskewed inverter. Marks: 7

3.(a) You have to design a car alarm system in which the alarm will ring (Y= 1) if anyone of the following condition is satisfied. (i) The key is in (K= 1), The Seat belt is fastened (B=1) and the door is open (D=0) or (ii) The key is in, Door is closed and Seat belt is not fastened. Write the Truth table, the Boolean equations and show the implementation of the circuit in (i) static CMOS, (ii) NMOS, (iii) Pseudo NMOS, (iv) Footed dynamic CMOS and (v) Un-footed dynamic CMOS. Marks: 6

1. Theory

1. From words to a truth table (Core Idea)

A digital design starts by turning the words into **binary variables**:

- $K = 1$: the key is in
- $B = 1$: the seat belt is fastened
- $D = 1$: the door is closed ($D = 0$: open)
- $Y = 1$: the alarm rings

A **truth table** lists every combination of the inputs (here $2^3 = 8$ rows) and the output for each. Each row with $Y = 1$ gives one AND term (a **minterm**); OR-ing them gives the **sum-of-products** equation.

2. The five logic styles

All five circuits share one idea: an **nMOS pull-down network** that conducts when the gate's output should be 0. They differ in how the output is pulled *up*:

- **Static CMOS:** a full pMOS network, the dual of the nMOS network (1(c)). No static current; $2n$ transistors for n inputs.
- **NMOS (depletion load):** the old all-nMOS technology. The pull-up is a **depletion-mode nMOS** (a transistor that conducts even at $V_{GS} = 0$) with its gate tied to its source, acting as a resistor. Ratioed: $V_{OL} > 0$ and static current when the output is low.
- **Pseudo-NMOS:** the same, but the load is a pMOS with its gate grounded (1(b)).
- **Footed dynamic:** a clocked pMOS precharges the output; a clocked nMOS "foot" at the bottom lets the network evaluate only when $\phi = 1$ (2(e)).
- **Un-footed dynamic:** the same without the foot. It is faster (one fewer series transistor) but the inputs must be 0 during precharge, otherwise there is a path from V_{DD} to ground. This holds when its inputs come from other domino gates, whose outputs are 0 during precharge.

Every one of these gates computes the **inverse** of its pull-down function at its first node, so an inverter at the output gives the true (non-inverted) Y.

Free shared copy · Nur Mohammad

2. Answer

Truth table

K	B	D	Y	Condition
0	0	0	0	key out
0	0	1	0	key out
0	1	0	0	key out
0	1	1	0	key out
1	0	0	0	belt off, door open
1	0	1	**1**	(ii) key in, door closed, belt off
1	1	0	**1**	(i) key in, belt on, door open
1	1	1	0	belt on, door closed

Table 7: Truth table of the car alarm

Boolean equations

$$Y = K\,B\,\overline{D} + K\,\overline{B}\,D = K\left(B\overline{D} + \overline{B}D\right) = K\,(B \oplus D)$$

The pull-down network (nMOS) implements $K \cdot (B\overline{D} + \overline{B}D)$: **K in series** with two parallel branches, B–$\overline{D}$ in series and $\overline{B}$–D in series. The node above it is $Z = \overline{Y}$, and an output inverter gives Y. The complements $\overline{B}$ and $\overline{D}$ come from input inverters.

(i) Static CMOS

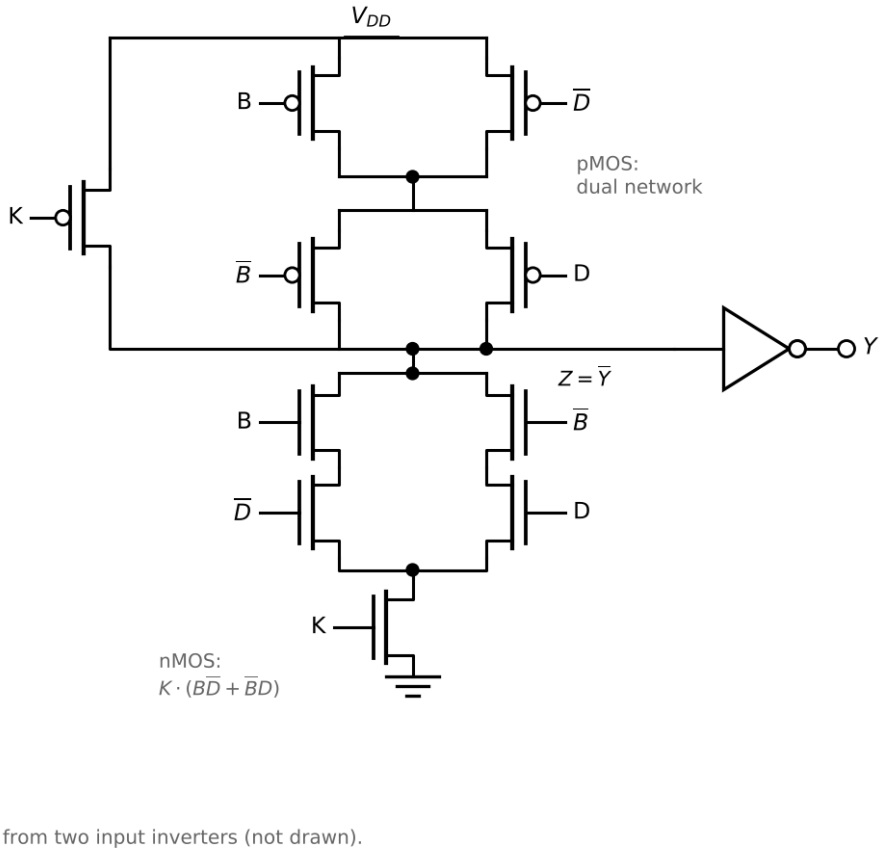


Figure 20: Static CMOS: pMOS dual network over the nMOS network K(B D' + B' D), then an inverter

Free shared copy · Nur Mohammad

The pMOS network is the dual: K_p in **parallel** with $[(B_p \parallel \overline{D}_p)$ in series with $(\overline{B}_p \parallel D_p)]$. Total: 5 + 5 transistors in the complex gate, 2 in the output inverter, 4 in the two input inverters.

(ii) NMOS and (iii) pseudo-NMOS

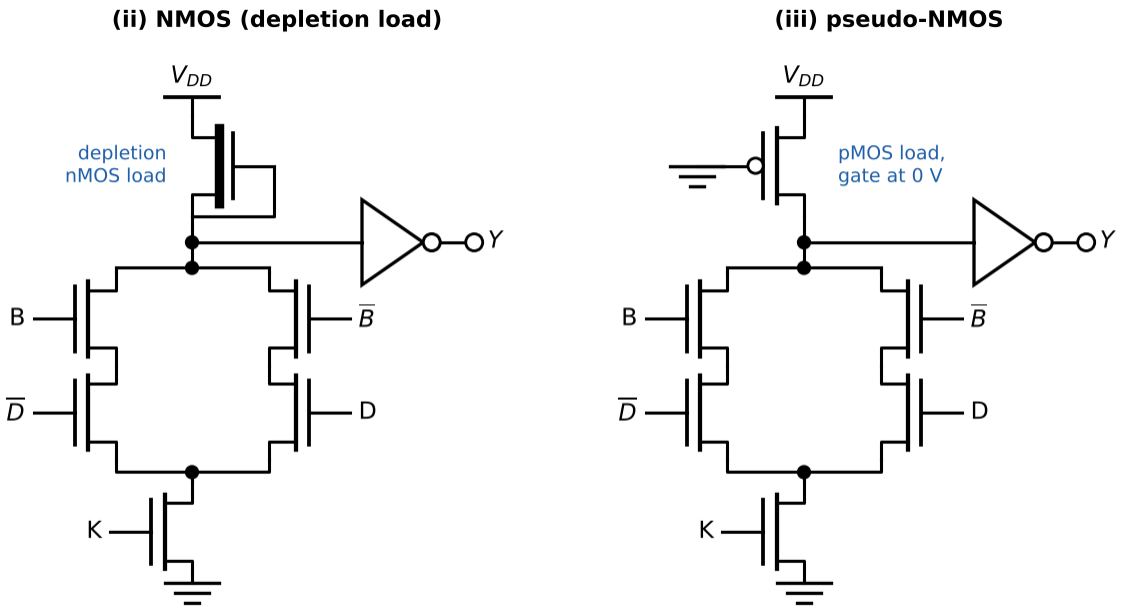


Figure 21: NMOS with a depletion load, and pseudo-NMOS with a grounded-gate pMOS load; same pull-down network

Only the load changes: a depletion nMOS with gate tied to the output node (ii), or a pMOS with gate at 0 V (iii). The output inverter is built in the same style (load + one nMOS). The pull-down transistors must be strong enough (ratio) to give a good V_{OL}.

Free shared copy · Nur Mohammad

(iv) Footed and (v) un-footed dynamic CMOS

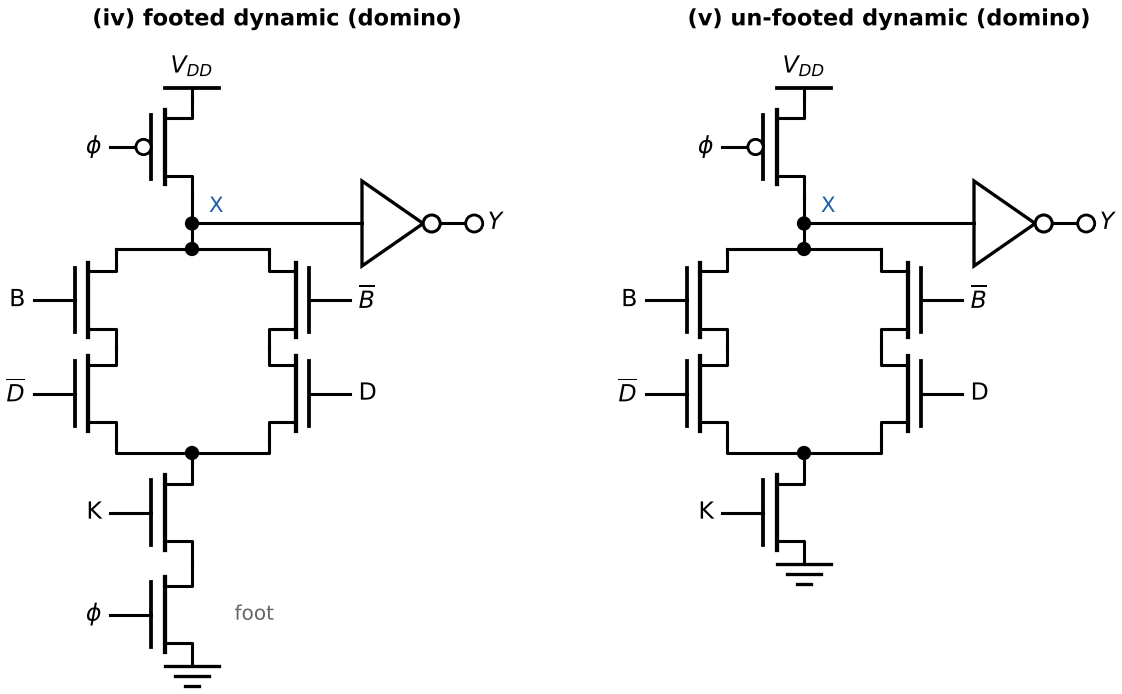


Figure 22: Footed and un-footed dynamic (domino) gates: precharge pMOS, same network, foot nMOS only in (iv)

Operation: $\phi = 0$: X precharges to 1, $Y = 0$. $\phi = 1$: X discharges if $K(B\overline{D} + \overline{B}D) = 1$, and the inverter gives $Y = 1$. In (v) the foot is removed; K, B, $\overline{B}$, D, $\overline{D}$ must then be held at 0 during precharge.

3. Keyword Explanation

1. Minterm

↩ back

An AND term containing every input once, true or complemented; it is 1 in exactly one row of the truth table.

2. Depletion-mode transistor

↩ back

A transistor with a negative threshold (for nMOS), so it is on at $V_{GS} = 0$; used as a load in NMOS logic.

4. Notes and Remarks

- **Simpler form:** $Y = K(B \oplus D)$, so the alarm could also be an XOR gate followed by an AND gate. The single complex gate above is smaller and faster.
- **Monotonic inputs for dynamic logic:** $\overline{B}$ and $\overline{D}$ come from static inverters. In a real domino chain they must already be stable before evaluation starts (here they are primary inputs, so they are).

3.(b) For the logic function $Y = \overline{(A + BC) \cdot D}$, choose transistor widths to achieve equal rise and fall resistance as a unit inverter. Assume, $\mu_n = 3\mu_p$ Marks: 4

1. Theory

1. A transistor as a resistor (Core Idea)

An "on" transistor behaves like a resistor whose value is **inversely proportional to its width**: twice as wide means half the resistance. Its mobility matters too: here electrons move $\mu_n/\mu_p = 3$ times faster than holes, so a pMOS has 3 times the resistance of an nMOS of the same width.

2. The unit inverter

Let an nMOS of width 1 have resistance R. For equal rise and fall resistance, the pMOS must be 3 times wider. So the **unit inverter** is nMOS width 1, pMOS width 3, and both edges see resistance R.

3. Sizing a gate

For a complex gate, every path from the output to ground (and from the output to V_{DD}) that can conduct alone must have resistance **no more than** R, the same as the unit inverter:

- k transistors in **series** add their resistances, so each must be about k times wider;
- transistors in **parallel** are sized for the worst case, which is *one* of them conducting alone.

2. Answer

Function: $Y = \overline{(A + BC) \cdot D}$. Unit inverter: nMOS 1, pMOS 3 (since $\mu_n = 3\mu_p$).

Pull-down network (nMOS)

$(A + BC) \cdot D$: **D** in series with [A in parallel with (B in series with C)].

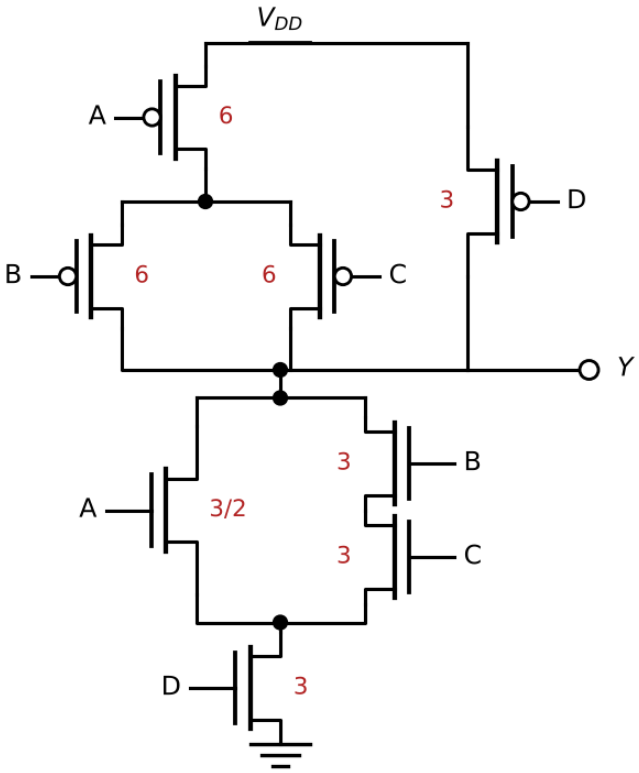
- **Path D–B–C:** three transistors in series, each $R/3$, so $D = B = C = 3$.
- **Path D–A:** D already gives $R/3$, so A may give $2R/3$: $\frac{1}{A} + \frac{1}{3} = 1 \Rightarrow A = \frac{3}{2}$.

Pull-up network (pMOS, dual)

D_p in parallel with [A_p in series with (B_p in parallel with C_p)]. A pMOS of width w has resistance $3R/w$.

- **Path D_p alone:** one transistor with resistance R: $D_p = 3$.
- **Path A_p–B_p (or A_p–C_p):** two in series, each $R/2$: $A_p = B_p = C_p = 2 \times 3 = 6$.

Free shared copy · Nur Mohammad



Widths (red) in units of the unit inverter's nMOS; unit inverter = nMOS 1, pMOS 3.

Figure 23: Sized gate: nMOS A = 3/2, B = C = D = 3; pMOS D = 3, A = B = C = 6

nMOS: $A = \frac{3}{2},\ B = C = D = 3;$	pMOS: $A = B = C = 6,\ D = 3$

(widths in units of the unit inverter's nMOS width)

3. Keyword Explanation

1. Unit inverter

↩ back

The reference inverter (nMOS width 1, pMOS width μ_n/μ_p) whose rise and fall resistances are both R.

4. Notes and Remarks

- **Rounded sizing:** many designers size A as 2 or 3 to keep a regular layout; the gate is then slightly faster than required at the cost of a little more input capacitance on A.
- **Input capacitance (logical effort):** input A sees $\frac{3}{2} + 6 = 7.5$, B and C see $3 + 6 = 9$, D sees $3 + 3 = 6$, against 4 for the unit inverter.

3.(c) Show that the dynamic power dissipation of a CMOS inverter varies directly with square of the power supply voltage whereas the delay of the circuit varies inversely with the power supply voltage. Marks: 8

1. Theory

1. Energy stored in a capacitor (Core Idea)

A capacitor C charged to voltage V holds charge $Q = CV$ and stored energy $\frac{1}{2}CV^2$. A CMOS gate output is such a capacitor (the **load capacitance** C_L: wires plus the gates it drives). Each time the output switches, C_L is charged or discharged. This takes energy (power) and time (delay).

2. How fast a transistor can charge it

The transistor current sets the speed. In saturation (long-channel model) the current is

$$I_{sat} = \frac{\beta}{2}(V_{DD} - V_t)^2$$

where $\beta = \mu C_{ox} W/L$ and V_t is the threshold voltage. More V_{DD} gives a much larger current, but the charge to move ($C_L V_{DD}$) grows only linearly.

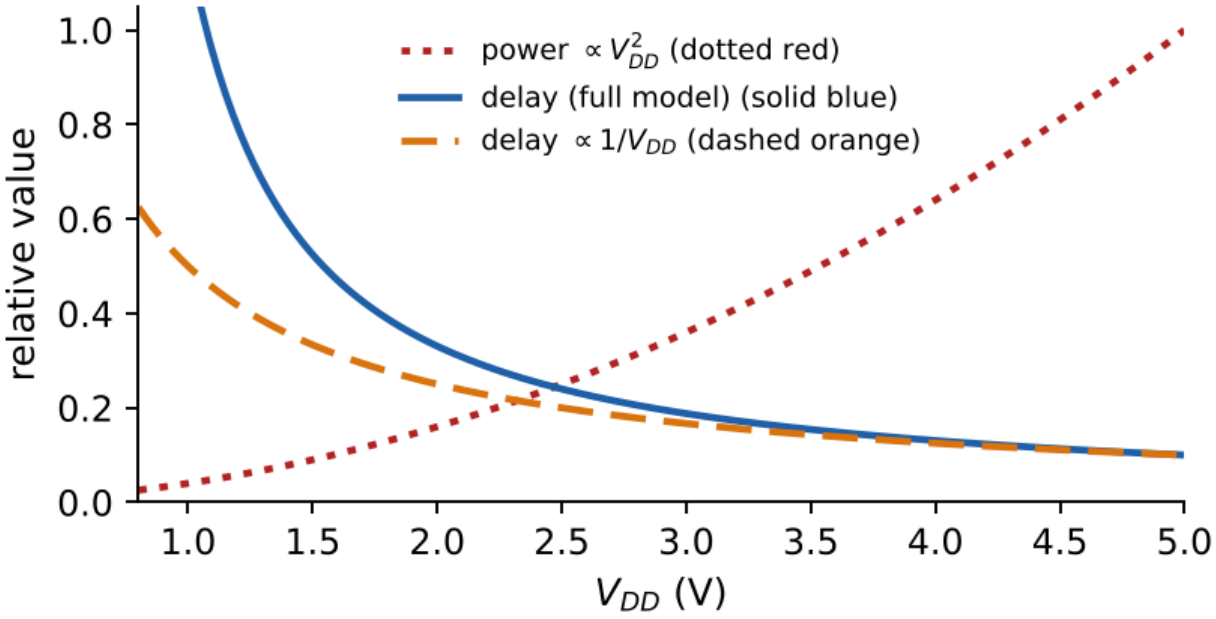


Figure 24: Dynamic power grows as VDD squared (dotted red) while delay falls roughly as 1/VDD (dashed orange); the full model (solid blue) rises steeply near Vt

Reading the figure. The dotted red curve is $P \propto V_{DD}^2$, normalised to 1 at 5 V. The dashed orange curve is delay $\propto 1/V_{DD}$; the solid blue curve is the full delay $\propto V_{DD}/(V_{DD} - V_t)^2$ for $V_t = 0.4$ V. Far above V_t the two delay curves agree; near V_t the delay grows much faster. Lowering V_{DD} saves power fast but costs speed.

2. Answer

Part 1: Dynamic power $\propto V_{DD}^2$. Consider a CMOS inverter driving C_L (see the figure in 2(d)).

Step 1 (Basic): Charge and energy of a capacitor

A capacitor holds $q = C_L v$, so a change dv needs the charge $dq = C_L\, dv$, and the current is $i = C_L\, dv/dt$. Moving dq through a voltage V costs the energy $V\, dq$.

Step 2: Charging (output 0 → 1)

The pMOS connects C_L to V_{DD}. The charge that flows from the supply is $Q = C_L V_{DD}$, all at voltage V_{DD}, so the energy taken from the supply is

$$E_{supply} = \int_0^\infty V_{DD}\, i_{DD}(t)\, dt = V_{DD} \int_0^{V_{DD}} C_L\, dv_{out} = C_L V_{DD}^2$$

The energy stored in the capacitor at the end is

$$E_C = \int_0^\infty v_{out}\, i_{DD}(t)\, dt = \int_0^{V_{DD}} C_L\, v_{out}\, dv_{out} = \tfrac{1}{2} C_L V_{DD}^2$$

So $\frac{1}{2} C_L V_{DD}^2$ is lost as heat in the pMOS while charging.

Free shared copy · Nur Mohammad

Step 3: Discharging (output 1 → 0)

The nMOS empties C_L; the stored $\frac{1}{2}C_L V_{DD}^2$ is lost as heat in the nMOS. Nothing is taken from the supply.

Step 4: Average power

So every full cycle of the output costs $C_L V_{DD}^2$ from the supply. If the output makes a $0 \to 1$ transition in a fraction α of the cycles of a clock at frequency f:

$$\boxed{P_{dynamic} = \alpha\, C_L\, V_{DD}^2\, f \;\propto\; V_{DD}^2}$$

The energy does not depend on the transistor sizes or the speed of switching, only on C_L and V_{DD}.

Part 2: Delay $\propto 1/V_{DD}$.

Step 5: The discharging current

Take the falling output: C_L starts at V_{DD} and the delay is the time to reach $V_{DD}/2$. During this time the nMOS ($V_{GS} = V_{DD}$) stays in saturation, carrying about

$$I_{sat} = \frac{\beta_n}{2}(V_{DD} - V_t)^2$$

Step 6: The delay

A constant current discharges a capacitor at a constant rate (Step 1), $I = C_L\, \Delta V/\Delta t$, so

$$t_{pd} = \frac{C_L\,(V_{DD}/2)}{I_{sat}} = \frac{C_L\, V_{DD}/2}{\frac{\beta_n}{2}(V_{DD} - V_t)^2} = \frac{C_L\, V_{DD}}{\beta_n\,(V_{DD} - V_t)^2}$$

When $V_{DD} \gg V_t$, $(V_{DD} - V_t)^2 \approx V_{DD}^2$, so

$$\boxed{t_{pd} \approx \frac{C_L}{\beta_n\, V_{DD}} \;\propto\; \frac{1}{V_{DD}}}$$

The same holds for the rising edge with β_p.

Steps to remember

Supply gives $Q = C_L V_{DD}$ at V_{DD}: $C_L V_{DD}^2$ per cycle → half stored, half lost, then the stored half lost on discharge → $P = \alpha C_L V_{DD}^2 f$ → saturated nMOS current $\frac{\beta}{2}(V_{DD} - V_t)^2$ → $t_{pd} = C_L(V_{DD}/2)/I_{sat}$ → $\approx C_L/(\beta V_{DD})$.

Interpretation: doubling V_{DD} roughly halves the delay but quadruples the dynamic power. The **energy-delay product** $\propto V_{DD}^2 \times \frac{1}{V_{DD}} = V_{DD}$ improves at lower voltage, which is why low-power chips run at the lowest V_{DD} that still meets the speed target.

3. Keyword Explanation

1. Load capacitance C_L

↩ back

The total capacitance at a gate's output: its own diffusion capacitance, the wire, and the gate capacitance of the gates it drives.

2. Propagation delay t_{pd}

↩ back

The time from the input crossing 50 % to the output crossing 50 %.

Free shared copy · Nur Mohammad

4. Notes and Remarks

- **Velocity saturation:** in short-channel transistors $I_{sat} \propto (V_{DD} - V_t)$ rather than squared, so $t_{pd} \propto V_{DD}/(V_{DD} - V_t)$; the delay then depends on V_{DD} more weakly, but the trend is the same.
- **Near threshold** the approximation fails: as $V_{DD} \to V_t$ the delay grows very fast (solid blue curve in the figure).

3.(d) Explain the inverter width sizing for favoring the rising edge, falling edge, or achieving an unskewed inverter. Marks: 7

1. Theory

1. Rising and falling edges (Core Idea)

The **rising** edge of an inverter's output is produced by the pMOS (pull-up), the **falling** edge by the nMOS (pull-down). Each edge is fast when its transistor is strong (wide). Making one transistor relatively wider speeds up its edge, at the cost of the other edge or of input capacitance. A gate sized to favour one edge is called **skewed**.

2. Logical effort of each edge

With $\mu_n = 2\mu_p$, the **unskewed** reference inverter has pMOS 2 and nMOS 1: both edges have the same resistance R, and the input capacitance is 3. The logical effort of an edge compares a gate with this reference **delivering the same current on that edge**:

$$g_{edge} = \frac{\text{input capacitance of the gate, scaled to give the unit current on that edge}}{3}$$

The beta ratio and its effect on noise margins are discussed in 2(b).

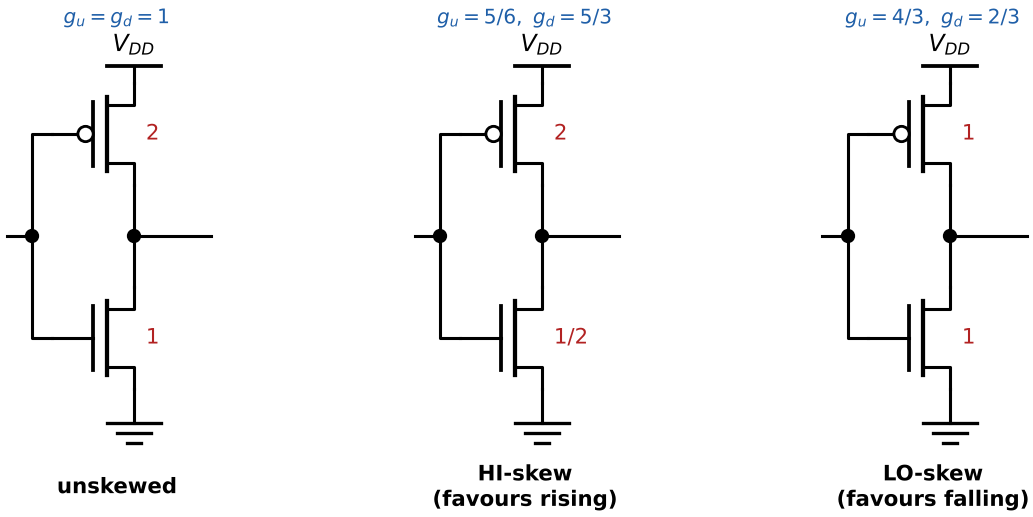


Figure 25: Unskewed, HI-skew and LO-skew inverters with their widths and the logical effort of each edge

2. Answer

Widths are in units where the unit nMOS is 1 and $\mu_n = 2\mu_p$; g_u and g_d are the logical efforts of the rising (up) and falling (down) output edges.

1. Unskewed inverter

pMOS 2, nMOS 1. The pull-up and pull-down resistances are equal, so rise and fall delays are equal: $g_u = g_d = 1$. The switching threshold is near $V_{DD}/2$. This is the reference used in logical effort.

Free shared copy · Nur Mohammad

2. HI-skew inverter (favours the rising edge)

Keep the pMOS at 2 and **shrink the nMOS** to $\frac{1}{2}$. Input capacitance = 2.5.

- **Rising:** the pMOS is the same as the unit inverter's, so the same current comes from less input capacitance: $g_u = \dfrac{2.5}{3} = \dfrac{5}{6}$.
- **Falling:** the nMOS is half as strong; to deliver the unit current the gate must be scaled by 2, giving capacitance 5: $g_d = \dfrac{5}{3}$.

The rising output is faster, the falling output slower; V_M moves up. Used when the rising edge is critical (for example after a dynamic gate in domino logic, whose output only rises).

3. LO-skew inverter (favours the falling edge)

Keep the nMOS at 1 and **shrink the pMOS** to 1. Input capacitance = 2.

- **Falling:** same nMOS as the unit inverter: $g_d = \dfrac{2}{3}$.
- **Rising:** the pMOS is half as strong; scale by 2 to capacitance 4: $g_u = \dfrac{4}{3}$.

The falling output is faster; V_M moves down.

Inverter	pMOS	nMOS	g_u	g_d	g_{avg}
Unskewed	2	1	1	1	1
HI-skew	2	1/2	5/6	5/3	5/4
LO-skew	1	1	4/3	2/3	1

Table 8: Skewed inverter sizes and logical efforts

Interpretation: skewing makes the important edge faster by shrinking the transistor of the unimportant edge; the input capacitance falls, so the previous stage is loaded less. The cost is a slower other edge and a smaller noise margin on one side.

3. Keyword Explanation

1. Skewed gate

↩ back

A gate whose pull-up and pull-down strengths are deliberately unequal, to favour one output transition.

2. g_u, g_d

↩ back

Logical effort for the rising (up) and falling (down) output transition.

4. Notes and Remarks

- **P/N ratio for minimum average delay:** if both edges matter equally, a pMOS somewhat smaller than 2× (about $\sqrt{2}$ times the nMOS) minimises the average delay, because the smaller pMOS loads the previous stage less. The unskewed 2 : 1 choice gives equal edges, not the smallest average.

5. Final Concise Answer

(a) $Y = KB\overline{D} + K\overline{B}D = K(B \oplus D)$ (truth-table rows 101 and 110). All five styles share the nMOS network $K \cdot (B\overline{D} + \overline{B}D)$ under node $Z = \overline{Y}$, followed by an inverter: static CMOS adds the dual pMOS network; NMOS uses a depletion load; pseudo-NMOS a grounded-gate pMOS; footed dynamic a clocked precharge pMOS and clocked foot nMOS; un-footed dynamic the same without the foot.
(b) With unit inverter n 1 / p 3: nMOS $A = 3/2$, $B = C = D = 3$; pMOS $A = B = C = 6$, $D = 3$.
(c) Each output cycle draws $C_L V_{DD}^2$ from the supply, so $P = \alpha C_L V_{DD}^2 f \propto V_{DD}^2$. Discharging C_L by $V_{DD}/2$ with $I_{sat} = \frac{\beta}{2}(V_{DD} - V_t)^2$ gives $t_{pd} = \frac{C_L V_{DD}}{\beta(V_{DD}-V_t)^2} \approx \frac{C_L}{\beta V_{DD}} \propto 1/V_{DD}$.

Free shared copy · Nur Mohammad

(d) Unskewed: p 2, n 1, $g_u = g_d = 1$. HI-skew (fast rise): p 2, n 1/2, $g_u = 5/6$, $g_d = 5/3$. LO-skew (fast fall): p 1, n 1, $g_u = 4/3$, $g_d = 2/3$.

Free shared copy · Nur Mohammad

4. (a) In the following circuit of Figure 4(a), determine whether there is any set up or hold time violation. Marks: 10

   (b) Show the circuit diagram of an asynchronous resettable Flip-Flop and explain its operation utilizing a timing diagram. Marks: 6

   (c) Suppose you have written Verilog code to operate a microprocessor. What will be the upcoming steps till its fabrication? Briefly explain your answer. Marks: 6

   (d) Implement a 2-input AND gate using both transmission gates and pass-transistor logic. Marks: 3

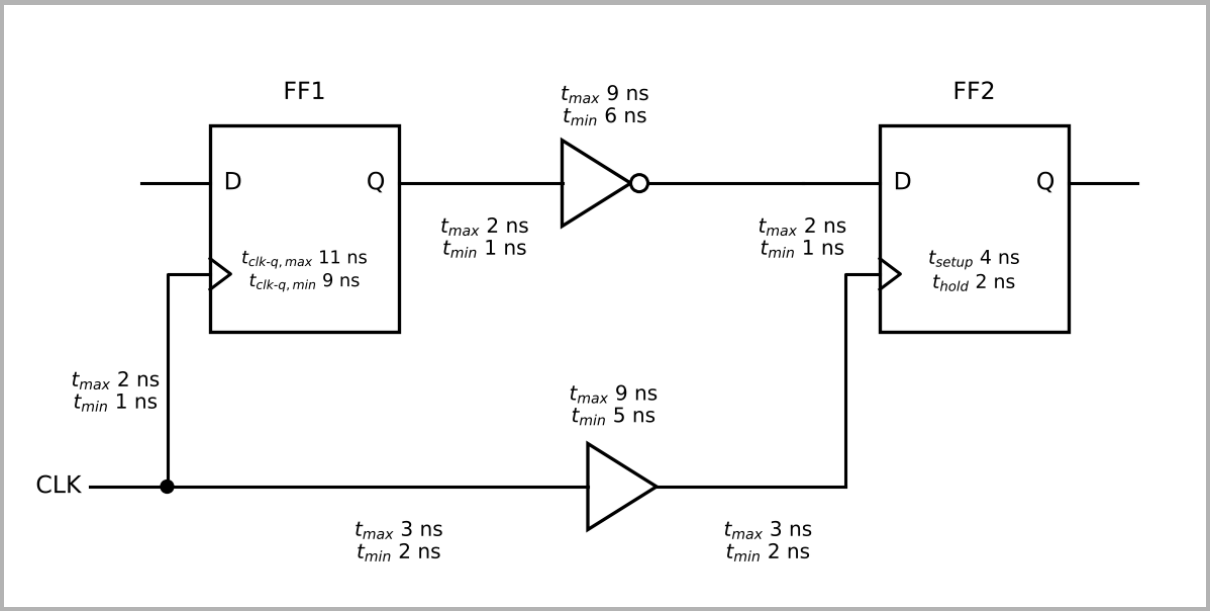


Figure 26: Figure 4(a) of the paper (redrawn)

Free shared copy · Nur Mohammad

4.(a) In the following circuit of Figure 4(a), determine whether there is any set up or hold time violation. Marks: 10

1. Theory

1. What does a flip-flop need? (Core Idea)

A **flip-flop** copies its input D to its output Q at the moment the clock rises (the **clock edge**), and holds Q until the next edge. For this copy to be reliable, D must be **steady for a short time around the edge**:

- **Setup time t_{setup}:** D must be stable at least this long *before* the clock edge.
- **Hold time t_{hold}:** D must stay stable at least this long *after* the clock edge.

After the edge, the new value appears at Q after the **clock-to-Q delay** $t_{clk\text{-}q}$.

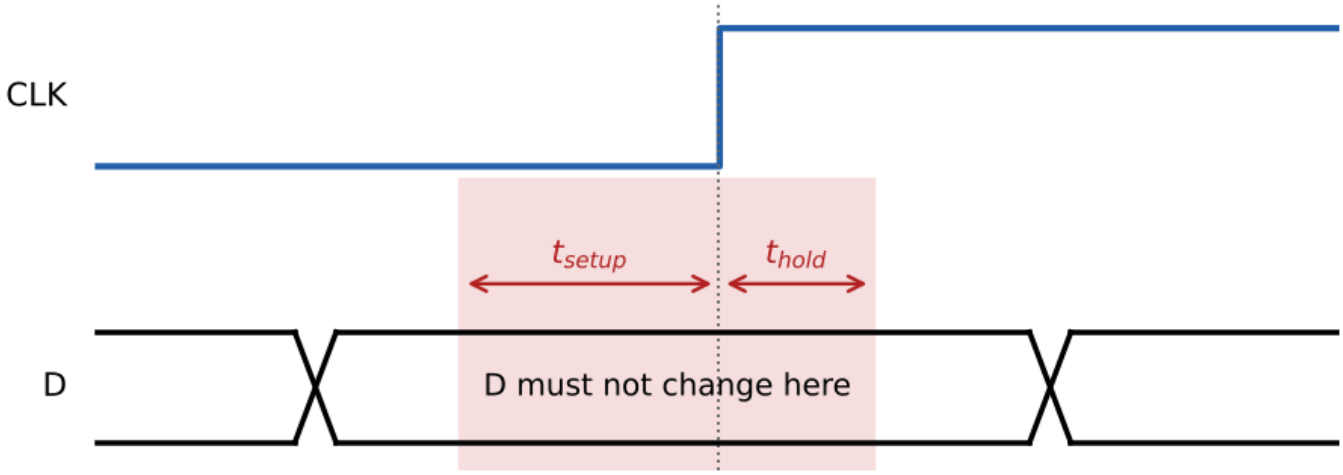


Figure 27: Setup and hold window: D must not change in the shaded interval around the clock edge

Reading the figure. The clock (top) rises at the dotted line. The shaded window runs from t_{setup} before the edge to t_{hold} after it. D (bottom) may change before or after the window, but not inside it.

2. Two flip-flops and the logic between them

In a pipeline, FF1 **launches** data at a clock edge; the data passes through logic and wires; FF2 **captures** it.

- **Setup check (too slow?):** the latest data, launched at one edge, must arrive at FF2 at least t_{setup} before the **next** clock edge at FF2. This sets the minimum clock period.
- **Hold check (too fast?):** the earliest data, launched at an edge, must not reach FF2 before FF2 has finished capturing the *same* edge, that is before (clock arrival at FF2) + t_{hold}. This does not depend on the clock period.

3. Clock skew

The clock does not reach every flip-flop at the same time: it goes through wires and buffers. The difference in arrival times is the **clock skew**. When the capture clock arrives late, setup becomes easier (more time) but hold becomes harder.

Free shared copy · Nur Mohammad

4. Which delays to use (worst cases)

Check	Data path	Launch clock	Capture clock
Setup	maximum delays	latest (max)	earliest (min)
Hold	minimum delays	earliest (min)	latest (max)

Table 9: Worst-case corners for timing checks

All times below are measured from the clock edge at the CLK source.

2. Answer

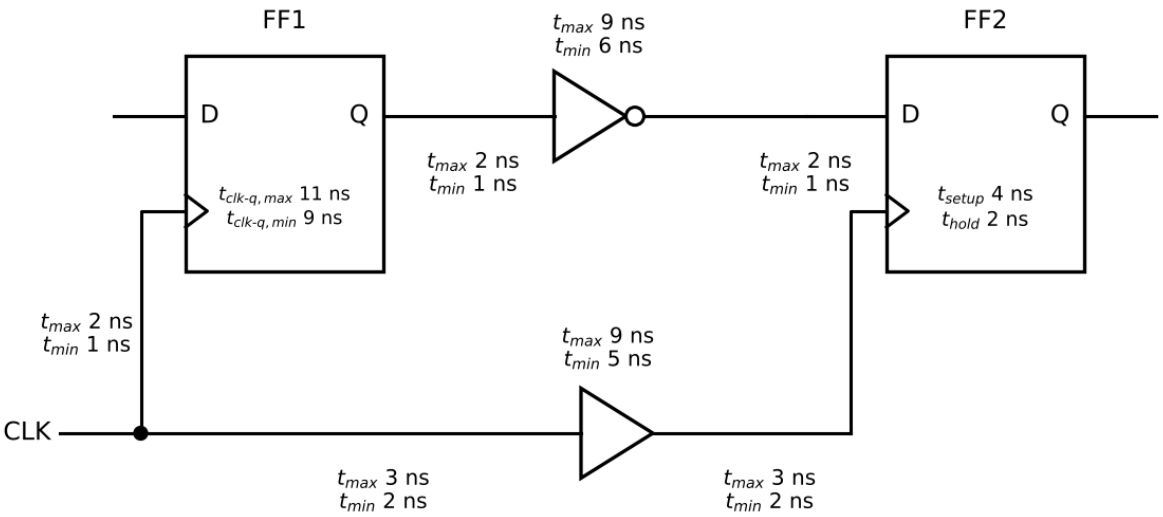


Figure 28: Repeated from the question: Figure 4(a) of the paper (redrawn)

Given (Figure 4(a)):

- Clock to FF1: $t_{max} = 2$ ns, $t_{min} = 1$ ns
- Clock to FF2: wire 3/2 ns, buffer 9/5 ns, wire 3/2 ns (max/min)
- FF1: $t_{clk\text{-}q,max} = 11$ ns, $t_{clk\text{-}q,min} = 9$ ns
- Data path: wire 2/1 ns, inverter 9/6 ns, wire 2/1 ns (max/min)
- FF2: $t_{setup} = 4$ ns, $t_{hold} = 2$ ns

Assumption: the paper gives no clock period T, so the setup check gives the smallest T that works.

Step 1: Clock arrival times

$$t_{clk1,max} = 2 \text{ ns}, \qquad t_{clk1,min} = 1 \text{ ns}$$

$$t_{clk2,max} = 3 + 9 + 3 = 15 \text{ ns}, \qquad t_{clk2,min} = 2 + 5 + 2 = 9 \text{ ns}$$

The capture clock arrives 7 to 14 ns *after* the launch clock (positive skew).

Step 2: Data arrival at FF2's D input

$$t_{data,max} = t_{clk1,max} + t_{clk\text{-}q,max} + (2 + 9 + 2) = 2 + 11 + 13 = 26 \text{ ns}$$

$$t_{data,min} = t_{clk1,min} + t_{clk\text{-}q,min} + (1 + 6 + 1) = 1 + 9 + 8 = 18 \text{ ns}$$

Free shared copy · Nur Mohammad

Step 3: Hold check

The earliest new data must arrive after FF2 has finished capturing the same edge:

$$t_{data,min} \geq t_{clk2,max} + t_{hold}$$

$$18 \text{ ns} \geq 15 + 2 = 17 \text{ ns} \quad \checkmark$$

$$\boxed{\text{Hold slack} = 18 - 17 = +1 \text{ ns: no hold violation}}$$

The margin is small: the late capture clock (up to 15 ns) almost lets the new data race through.

Step 4: Setup check

The latest data must arrive t_{setup} before the next edge at FF2, which comes at $T + t_{clk2,min}$ at the earliest:

$$t_{data,max} + t_{setup} \leq T + t_{clk2,min}$$

$$26 + 4 \leq T + 9 \quad \Rightarrow \quad T \geq 21 \text{ ns}$$

$$\boxed{T_{min} = 21 \text{ ns}, \quad f_{max} = \frac{1}{21 \text{ ns}} = 47.6 \text{ MHz}}$$

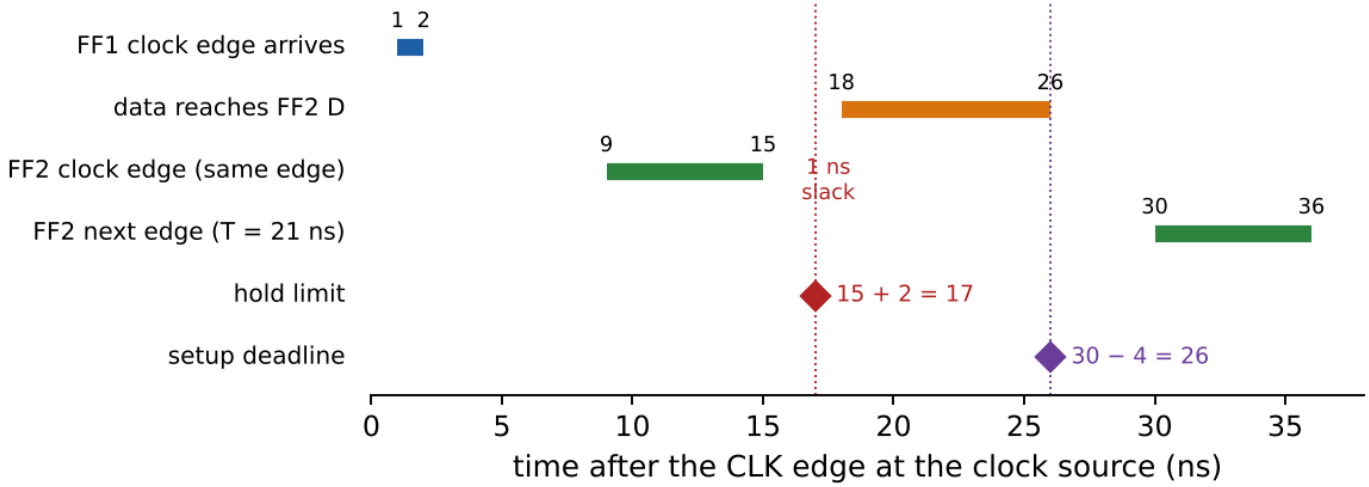


Figure 29: Timeline of the arrivals: data at FF2 between 18 and 26 ns, after the hold limit of 17 ns and before the setup deadline of 26 ns for T = 21 ns

Reading the figure. The thick orange bar is the data arrival window at FF2 (18–26 ns). The red diamond (17 ns) is when FF2 has finished holding the old data, so the data arrives 1 ns after it. The purple diamond (26 ns) is the setup deadline when $T = 21$ ns; any shorter period moves it left of the data's latest arrival.

Final Answer:

- **Hold:** no violation (slack +1 ns), whatever the clock period.
- **Setup:** no violation as long as $T \geq 21$ ns ($f \leq 47.6$ MHz); a faster clock would violate setup (for example $T = 20$ ns gives slack -1 ns).

Free shared copy · Nur Mohammad

3. Keyword Explanation

1. Slack

↩ back

The margin by which a timing requirement is met; negative slack is a violation.

2. Clock-to-Q delay

↩ back

The time from the clock edge to the new value at the flip-flop's output.

4. Notes and Remarks

- **Without the skew:** if FF2's clock arrived together with FF1's (1–2 ns), setup would need $T \geq 26 + 4 - 1 = 29$ ns. Here the late capture clock "lends" time to the slow path, which is why T_{min} is only 21 ns, but it eats into the hold margin.
- **Fixing a hold violation** needs more minimum delay in the data path (buffers) or less clock skew; changing the clock period never fixes hold.

4.(b) Show the circuit diagram of an asynchronous resettable Flip-Flop and explain its operation utilizing a timing diagram. Marks: 6

1. Theory

1. Latches and flip-flops (Core Idea)

A **latch** is transparent (output follows input) while its clock is at one level and holds its value while the clock is at the other level. It holds the value in a loop of two inverters feeding each other.

A **flip-flop** is two latches in a row, a **master** and a **slave**, clocked in opposite phases. For a positive-edge flip-flop, the master is transparent while $clk = 0$ and the slave while $clk = 1$. At the rising edge the master closes (holding D) and the slave opens (passing it to Q), so Q changes only at the rising edge.

2. Reset

A **reset** forces Q to 0 (for example at power-up).

- **Synchronous reset:** reset acts only at the next clock edge (it is just AND-ed into D).
- **Asynchronous reset:** reset forces Q to 0 **immediately**, whatever the clock is doing.

To make reset asynchronous, the inverters in *both* storage loops are replaced by gates with a reset input: a NAND forces its output to 1 when its other input is 0; a NOR forces its output to 0 when its other input is 1. The forced values must agree with a stored 0, so the loop keeps them after reset is released.

Free shared copy · Nur Mohammad

2. Answer

Circuit: a positive-edge master-slave flip-flop built with transmission gates; the master's forward inverter is a NAND with input $\overline{R}$, and the slave's forward inverter is a NOR with input R.

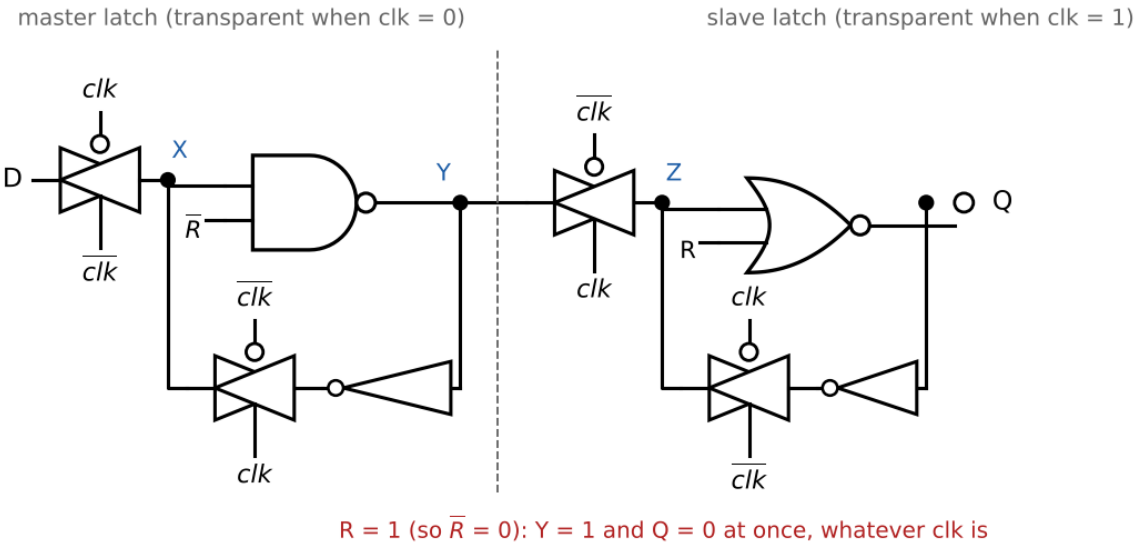


Figure 30: Asynchronous resettable flip-flop: NAND (with R-bar) in the master loop and NOR (with R) in the slave loop

Normal operation ($R = 0$, $\overline{R} = 1$): the NAND and the NOR act as inverters.

- $clk = 0$: the input transmission gate is on, so the master follows D ($X = D$, $Y = \overline{D}$). The slave's input gate is off, and the slave loop holds Q.
- $clk = 1$: the master closes its loop and holds; the slave gate opens and $Z = Y = \overline{D}$, so $Q = \overline{Z} = D$. Q takes the value D had **at the rising edge**.

Reset ($R = 1$): the NAND gets $\overline{R} = 0$, so $Y = 1$ at once; the NOR gets $R = 1$, so $Q = 0$ at once. Both happen without any clock edge. These values are the same as a stored 0 ($X = 0$, $Y = 1$, $Z = 1$, $Q = 0$), so when R returns to 0 the flip-flop keeps $Q = 0$ until the next rising edge loads D.

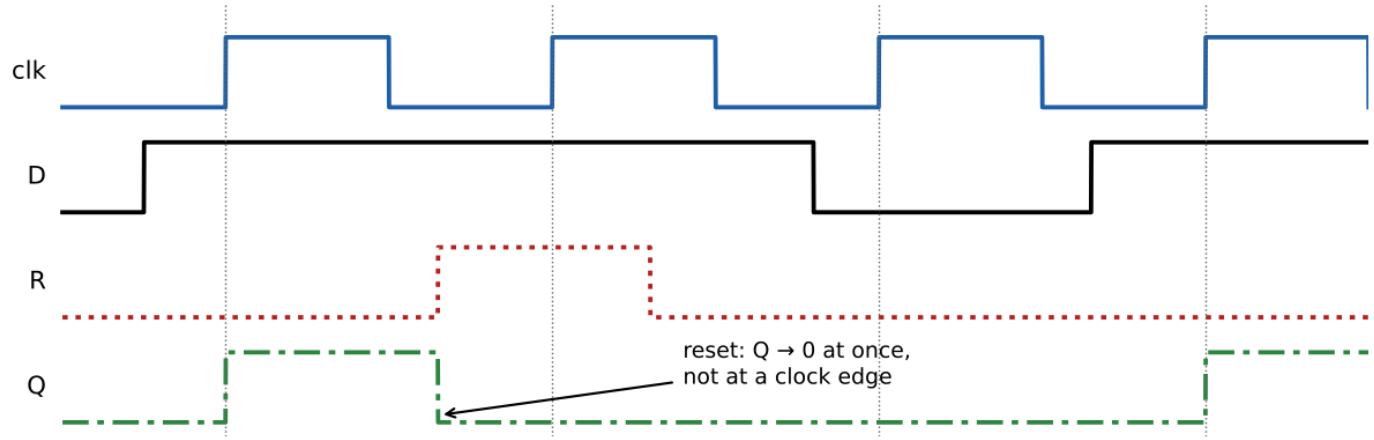


Figure 31: Timing diagram: Q follows D at rising clock edges; when R goes high, Q falls immediately, between edges, and the edge during reset is ignored

Reading the timing diagram. At the first rising edge D = 1, so Q becomes 1. R goes high in the middle of a clock cycle and Q falls to 0 at once, not at a clock edge. The edge that comes while R is high is ignored. After R is released, Q stays 0; at the edge where D = 0 it stays 0, and at the last edge D = 1 is loaded again.

3. Keyword Explanation

1. Transmission gate

↩ back

An nMOS and a pMOS in parallel, driven by complementary signals; a switch that passes both 0 and 1 well.

2. Master-slave flip-flop

↩ back

Two latches in series on opposite clock phases, forming an edge-triggered storage element.

4. Notes and Remarks

- **Asynchronous set** is made the same way with the gates swapped (a NOR in the master, a NAND with $\overline{S}$ in the slave), so Q is forced to 1.
- Asynchronous reset is fast to apply but its **release** must be synchronised to the clock, or different flip-flops may leave reset in different cycles.

4.(c) Suppose you have written Verilog code to operate a microprocessor. What will be the upcoming steps till its fabrication? Briefly explain your answer. Marks: 6

1. Theory

1. What Verilog code is (Core Idea)

Verilog code at the **register-transfer level** (RTL) describes what the microprocessor does: its registers, and the logic that moves and changes data between them every clock cycle. It is not yet a circuit. Many automated steps turn it into the geometric drawing of every layer of the chip (the **layout**), and then a factory turns the layout into silicon. This sequence is the **ASIC design flow**.

2. Answer

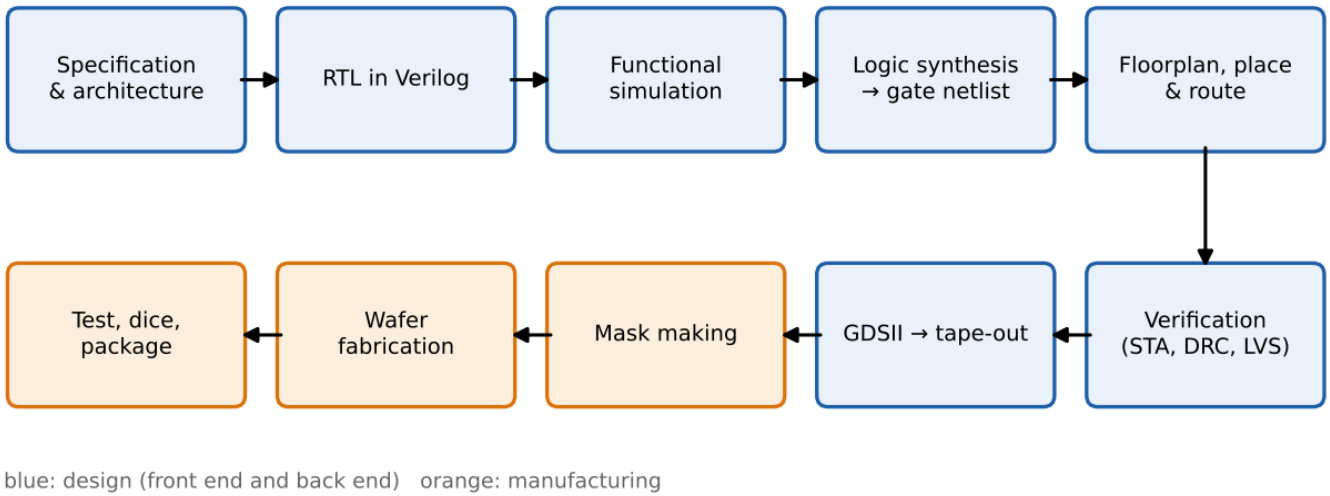


Figure 32: The design flow from Verilog code to a packaged chip

1. **Functional verification:** simulate the RTL with testbenches (and formal checks) to make sure the processor runs instructions correctly. Bugs are cheapest to fix here.
2. **Logic synthesis:** a synthesis tool maps the RTL onto the gates of a **standard-cell library** (NAND, NOR, flip-flops) under timing, area and power constraints. The result is a **gate-level netlist**. Test logic (scan chains) is usually inserted now (**design for test**).
3. **Floorplanning:** decide the chip size, where the large blocks (caches, register file) and the input/output pads go, and plan the power grid.

Free shared copy · Nur Mohammad

4. **Placement and clock-tree synthesis:** place every standard cell in rows; build a balanced clock tree so the clock reaches all flip-flops with little skew.
5. **Routing:** draw the metal wires that connect the cells.
6. **Sign-off verification:** static timing analysis (setup and hold on every path, as in 4(a)), power and IR-drop analysis, DRC (the layout obeys the manufacturing rules) and **LVS** (the layout matches the netlist).
7. **Tape-out:** the final layout is written as a **GDSII** file and sent to the foundry.
8. **Mask making:** one photomask is made for each layer.
9. **Wafer fabrication:** the foundry builds the transistors and wires layer by layer: oxidation, photolithography (5(d)), etching, ion implantation (1(e)), deposition and metallisation.
10. **Test and packaging:** each die is tested on the wafer, the wafer is cut (diced), good dies are packaged and tested again.

3. Keyword Explanation

1. Standard-cell library

↩ back

A set of pre-designed, pre-characterised gates of fixed height, which synthesis and placement tools use as building blocks.

2. DRC and LVS

↩ back

Design Rule Check (layout geometry legal for the process) and Layout Versus Schematic (layout wiring equals the intended circuit).

4.(d) Implement a 2-input AND gate using both transmission gates and pass-transistor logic. Marks: 3

1. Theory

1. Switches instead of pull-up and pull-down networks (Core Idea)

In **pass-transistor logic**, a transistor is used as a switch that *passes* an input signal to the output, instead of connecting the output to V_{DD} or ground. A function $Y = f$ is built by passing the right signal for each value of a control input.
For an AND gate: $Y = AB$ means "if $B = 1$, then $Y = A$; if $B = 0$, then $Y = 0$".

2. Weak and strong levels

- An **nMOS** passes a 0 perfectly but a 1 only up to $V_{DD} - V_t$ (it turns off as its source rises): a **weak 1**.
- A **pMOS** passes a 1 perfectly but a weak 0.
- A **transmission gate** (nMOS and pMOS in parallel, gates driven by complementary signals) passes both levels fully.

Free shared copy · Nur Mohammad

2. Answer

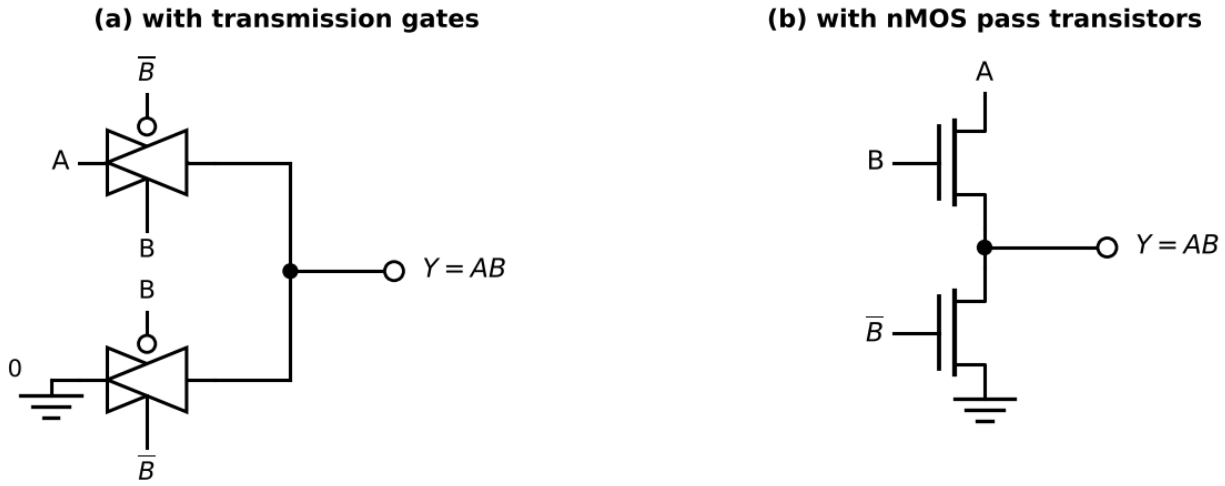


Figure 33: Two-input AND: (a) with transmission gates, (b) with nMOS pass transistors

(a) Transmission gates: the upper gate (nMOS gate B, pMOS gate $\overline{B}$) passes A to Y when $B = 1$; the lower gate (nMOS gate $\overline{B}$, pMOS gate B) passes 0 (ground) to Y when $B = 0$. So $Y = AB$ with full logic levels. 4 transistors (plus an inverter for $\overline{B}$).

(b) Pass-transistor logic: an nMOS with gate B passes A to Y when $B = 1$; an nMOS with gate $\overline{B}$ connects Y to ground when $B = 0$. 2 transistors (plus an inverter for $\overline{B}$). For $A = B = 1$ the output is only $V_{DD} - V_t$; a buffer (or a pMOS level restorer) at the output restores a full 1.

A	B	Path	Y
0	0	0 passed	0
1	0	0 passed	0
0	1	A passed	0
1	1	A passed	1

Table 10: Operation of the pass-transistor AND gate

3. Final Concise Answer

(a) Clock at FF1 1–2 ns, at FF2 9–15 ns; data at FF2 18–26 ns. Hold: $18 \geq 15 + 2 = 17$, slack +1 ns, **no hold violation**. Setup: $26 + 4 \leq T + 9$, so **no setup violation if** $T \geq 21$ **ns** ($f \leq 47.6$ MHz); a shorter period violates setup.

(b) Master-slave transmission-gate flip-flop with a NAND ($\overline{R}$) in the master loop and a NOR (R) in the slave loop; $R = 1$ forces $Y = 1$ and $Q = 0$ at once, independent of the clock, and the state stays 0 after release until the next edge.

(c) RTL verification → logic synthesis (netlist, scan) → floorplan → placement and clock tree → routing → sign-off (STA, DRC, LVS, power) → GDSII tape-out → masks → wafer fabrication → test, dicing, packaging.

(d) TG: pass A when $B = 1$, pass 0 when $B = 0$ (full swing). Pass transistor: nMOS(B) passes A, nMOS($\overline{B}$) pulls Y to 0; weak 1 of $V_{DD} - V_t$ needs restoring.

Free shared copy · Nur Mohammad

5. (a) (i) Show the circuit diagram of a 2 x 2 SRAM array which uses 6-transistor memory cell as basic storage cell. Clearly show the row select, column select, pre-charge and sense signal in your circuit.
   (ii) Explain how the READ and WRITE operation are performed showing the second row and first column cell as an example.
   (iii) Explain how sense amplifier reduces the READ time of the cell. Marks: 10

   (b) A 4-bit datapath consists of register, ALU and a shifter. Show two possible bus architectures of the system such that an addition operation of two operands stored in the register and storing the result back in the register can be computed in at most two clock cycles. Marks: 7

   (c) Explain the following: (i) positive mask, (ii) negative mask, (iii) positive photoresist and (iv) negative photoresist. Marks: 2

   (d) Describe the process sequence for patterning SiO_2 as illustrated in the figure 5(d). Marks: 6

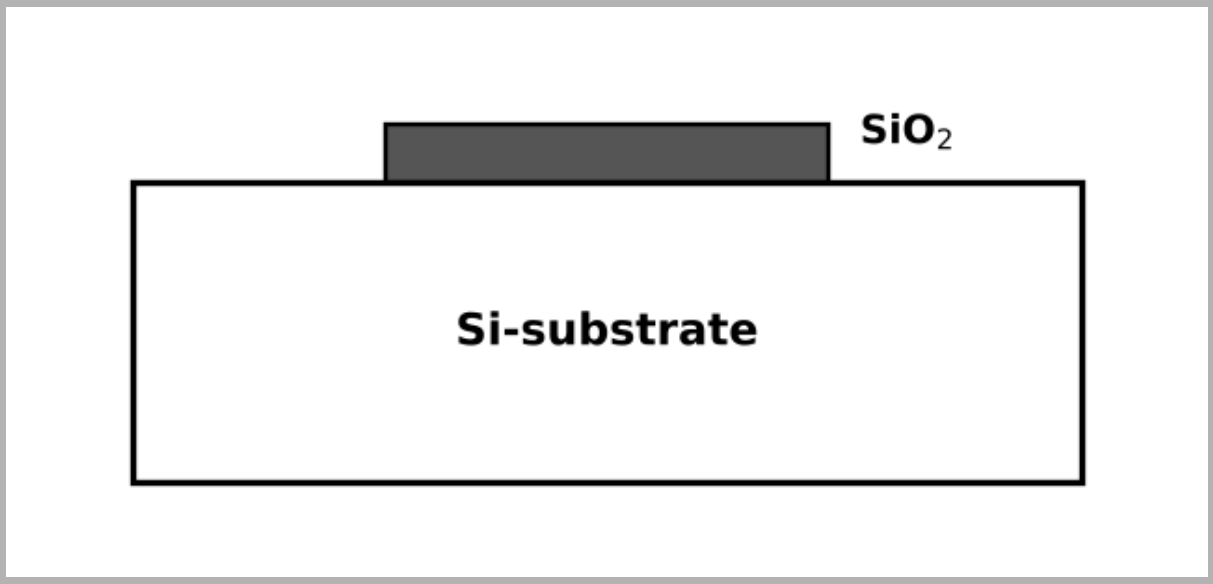


Figure 34: Figure 5(d) of the paper (redrawn)

5.(a) (i) Show the circuit diagram of a 2 x 2 SRAM array which uses 6-transistor memory cell as basic storage cell. Clearly show the row select, column select, pre-charge and sense signal in your circuit. (ii) Explain how the READ and WRITE operation are performed showing the second row and first column cell as an example. (iii) Explain how sense amplifier reduces the READ time of the cell. Marks: 10

1. Theory

1. What is SRAM? (Core Idea)

SRAM (static random-access memory) stores each bit in a small loop of two inverters. Each inverter's output drives the other's input, so the loop has two stable states: $Q = 1, \overline{Q} = 0$ or $Q = 0, \overline{Q} = 1$. It keeps its value as long as power is on ("static": no refresh needed, unlike DRAM).

Free shared copy · Nur Mohammad

2. The 6-transistor cell

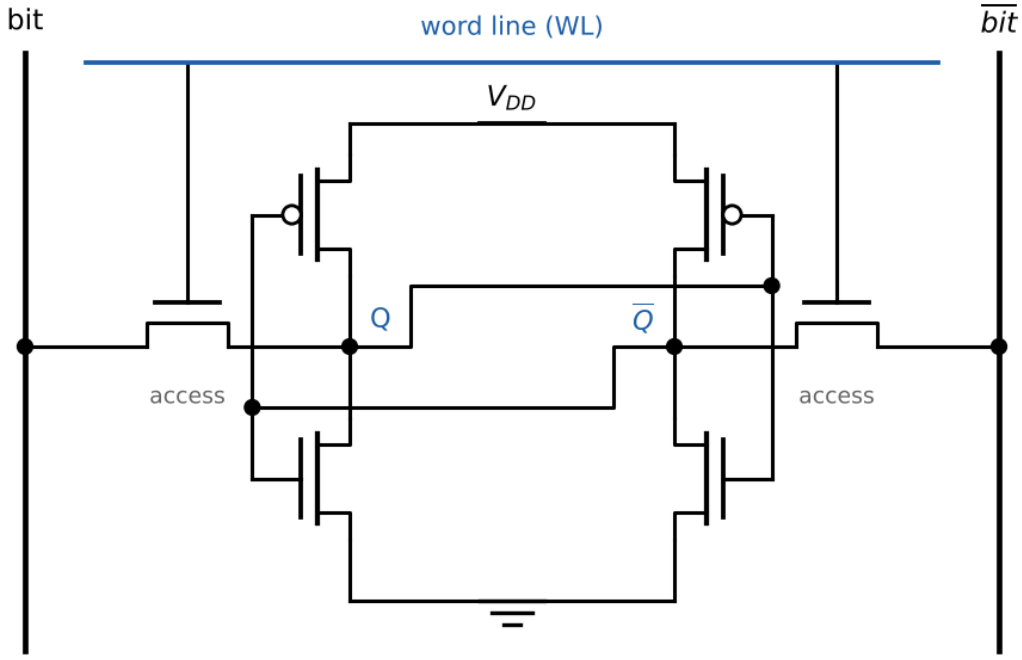


Figure 35: The 6T SRAM cell: two cross-coupled inverters (4 transistors) and two nMOS access transistors to the bit lines

Reading the figure. The two inverters (a pMOS over an nMOS each) are cross-coupled: the gates of the left inverter are driven by $\overline{Q}$, the gates of the right inverter by Q. Two **access transistors** (nMOS) connect Q to the **bit line** and $\overline{Q}$ to the complementary bit line $\overline{bit}$. Their gates are driven by the **word line** (WL). When WL is low, the cell is cut off and just holds its value; when WL is high, the cell is connected to the bit lines to be read or written.

3. The array

Cells are arranged in rows and columns:

- all cells of a **row** share one word line, driven by the **row decoder** (row select);
- all cells of a **column** share a pair of bit lines;
- **precharge** pMOS transistors pull every bit line to V_{DD} before each access;
- **column select** (column multiplexer) transistors connect the chosen column's bit lines to the shared data lines;
- a **sense amplifier** reads the small voltage difference between the data lines, and a **write driver** forces data onto them.

4. Why the bit lines are slow

A bit line runs past every cell of the column and has a large capacitance. The cell's transistors are tiny (to keep the cell small), so its current is small. Discharging the bit line by the full V_{DD} would take a long time ($t = C\,\Delta V / I$).

2. Answer

Free shared copy · Nur Mohammad

(i) Circuit of the 2 × 2 array

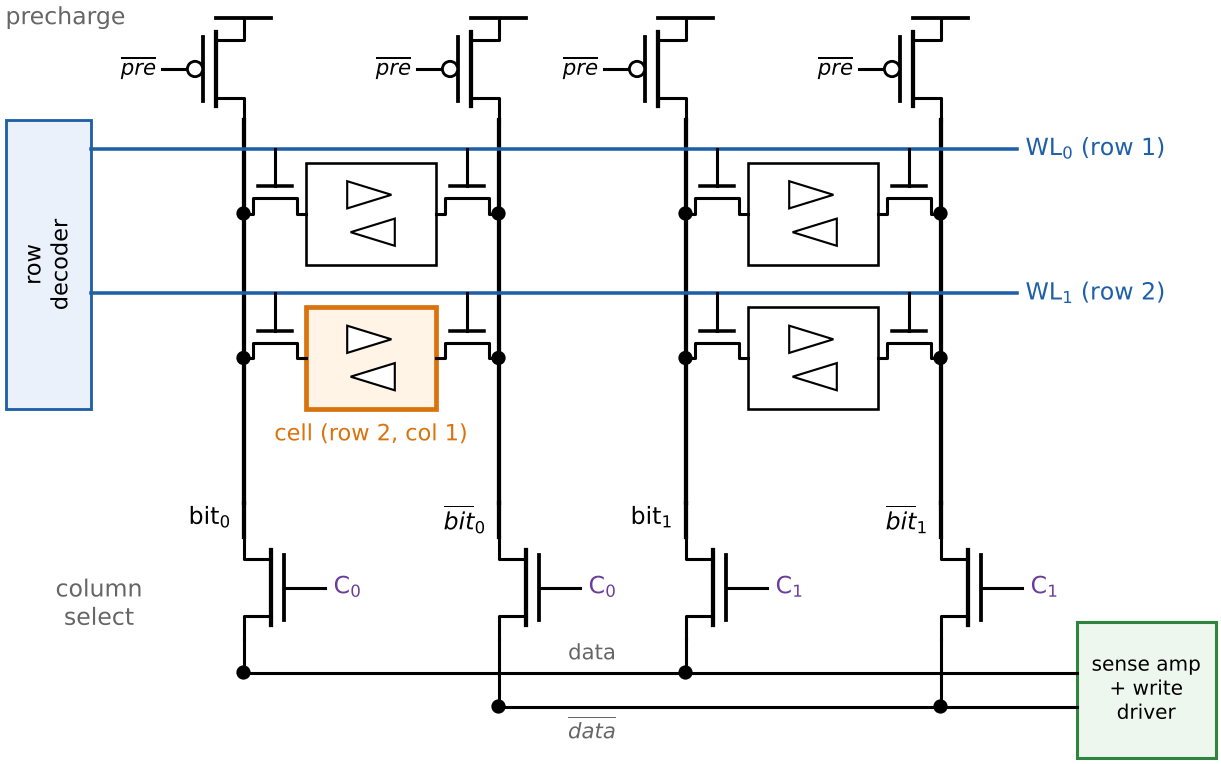


Figure 36: 2 x 2 SRAM array: precharge pMOS on every bit line, word lines WL0 and WL1 from the row decoder, column-select transistors C0 and C1, shared data lines to the sense amplifier and write driver; the cell of row 2, column 1 is highlighted

- **Row select:** WL_0 (row 1) and WL_1 (row 2), from the row decoder.
- **Column select:** C_0 (column 1) and C_1 (column 2) drive the nMOS pass transistors that connect bit_j, $\overline{bit}_j$ to data, $\overline{data}$.
- **Precharge:** pMOS on each bit line, on when $\overline{pre} = 0$.
- **Sense:** the sense amplifier, enabled by SAE, on the data lines, with the write driver beside it.

(ii) READ and WRITE of the cell in row 2, column 1

Suppose the cell stores $Q = 0$, $\overline{Q} = 1$.

READ:

1. **Precharge:** $\overline{pre} = 0$; bit_0 and $\overline{bit}_0$ (and all other bit lines) are charged to V_{DD}. Then precharge turns off.
2. **Select:** the row decoder raises $\mathbf{WL}_1$; the column decoder raises $\mathbf{C}_0$. Only the cell of row 2, column 1 is connected to the data lines.
3. **Develop:** on the side where $Q = 0$, current flows from bit_0 through the access transistor and the cell's nMOS to ground, so bit_0 starts to fall. $\overline{bit}_0$ stays at V_{DD} (its side stores 1).
4. **Sense:** when the difference reaches about 0.1–0.2 V, **SAE** is raised; the sense amplifier amplifies it to full levels: data = 0. WL_1 falls and the bit lines are precharged again.

(Cells in row 2, column 2 also connect to their bit lines but are not selected by C_1; their bit lines are simply precharged again.)

WRITE (write 1, so Q must become 1):

1. **Precharge** the bit lines as before.
2. The **write driver** drives data = 1, $\overline{data} = 0$; $\mathbf{C}_0$ connects them to $\mathrm{bit}_0 = 1$, $\overline{bit}_0 = 0$.

Free shared copy · Nur Mohammad

3. **WL**$_1$ rises. $\overline{bit}_0 = 0$ pulls $\overline{Q}$ down through the access transistor, which is made stronger than the cell's pMOS. When $\overline{Q}$ falls below the switching threshold, the inverter driven by $\overline{Q}$ raises Q to 1, and Q = 1 in turn holds $\overline{Q}$ at 0: the cell flips.
4. WL$_1$ falls; the cell keeps $Q = 1$.

Sizing rules that make this work: for a safe read, the cell's pull-down nMOS is stronger than the access transistor (so the read does not flip the cell); for a write, the access transistor is stronger than the cell's pull-up pMOS.

(iii) How the sense amplifier reduces the READ time

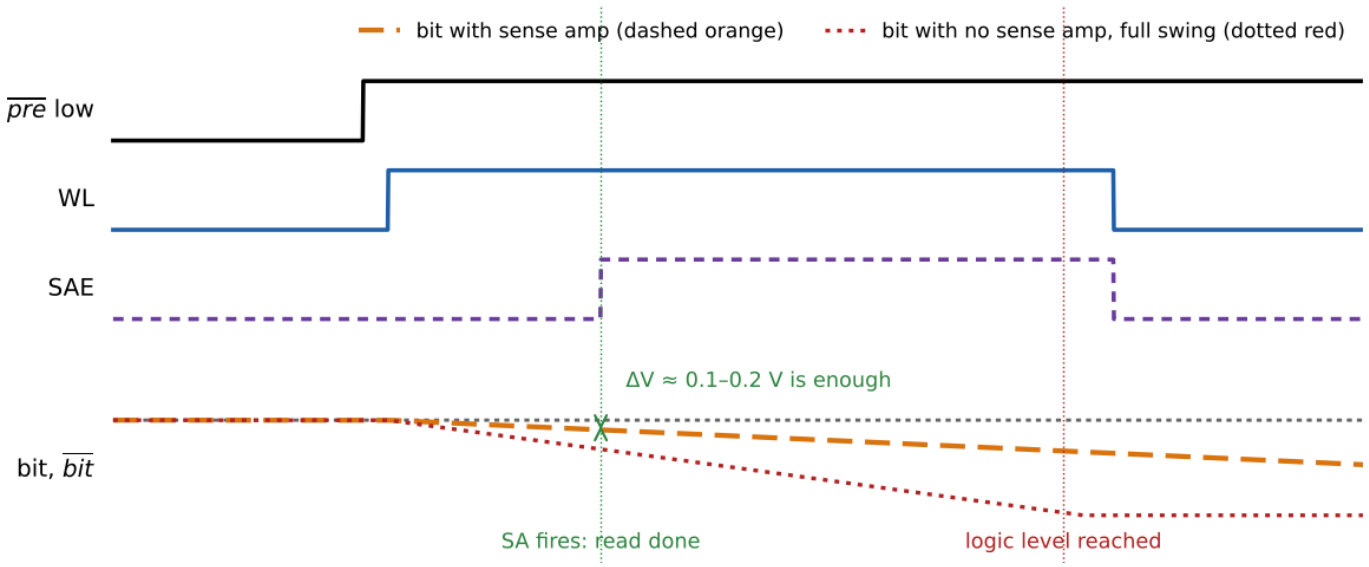


Figure 37: Read timing: with a sense amplifier the read ends when the bit lines differ by about 0.1–0.2 V (dashed orange); without one, the bit line must fall to a logic level (dotted red)

The time for the cell current I_{cell} to discharge a bit line of capacitance C_{bit} by ΔV is

$$t = \frac{C_{bit}\,\Delta V}{I_{cell}}$$

Without a sense amplifier the bit line must swing about $V_{DD}/2$ before a logic gate reads it correctly. A sense amplifier is a sensitive differential amplifier: it detects a **small** difference $\Delta V \approx 0.1$–0.2 V between bit and $\overline{bit}$ and amplifies it quickly to a full logic level. With $V_{DD} = 1.8$ V, for example, ΔV drops from 0.9 V to 0.15 V, so the bit-line part of the read becomes about **6 times faster**.

Free shared copy · Nur Mohammad

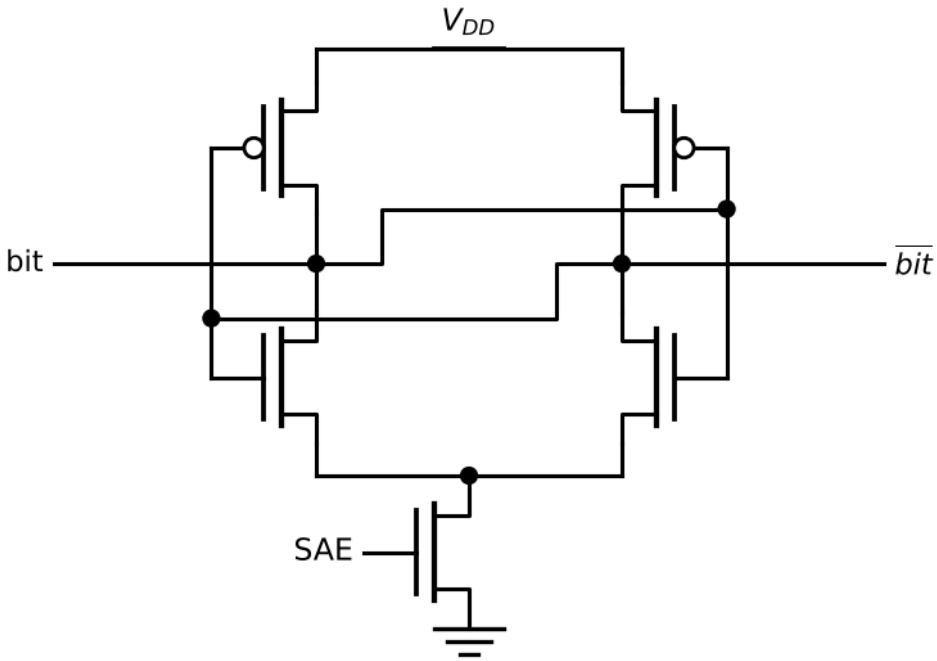


Figure 38: Cross-coupled latch sense amplifier: when SAE rises, the side that is slightly lower is pulled to 0 and the other to 1

The latch type shown uses two cross-coupled inverters (positive feedback): when SAE turns on the bottom nMOS, the slightly lower input is pulled to 0 and the higher to 1 within a short time. A smaller swing also saves the power of charging the bit lines back.

3. Keyword Explanation

1. Word line and bit line

↩ back

The word line selects a row of cells (gates of the access transistors); the bit lines carry the data in and out of the selected cells of a column.

2. Precharge

↩ back

Charging the bit lines to V_{DD} before each access, so that a read only needs to discharge one of them.

3. Sense amplifier

↩ back

An amplifier that turns a small differential voltage on the bit lines into a full-swing logic output.

4. Notes and Remarks

- **Read stability:** the read must not flip the cell. If the access transistor were stronger than the cell's nMOS, the node storing 0 would rise too far and could flip the cell. Typically the cell nMOS is about 1.5–2 times stronger than the access transistor.
- **Differential bit lines** cancel common noise, which is one reason SRAM uses both bit and $\overline{bit}$.

5.(b) A 4-bit datapath consists of register, ALU and a shifter. Show two possible bus architectures of the system such that an addition operation of two operands stored in the register and storing the result back in the register can be computed in at most two clock cycles. Marks: 7

Free shared copy · Nur Mohammad

1. Theory

1. Datapath and buses (Core Idea)

A **datapath** is the part of a processor that holds and processes data: a **register file** (stores operands), an **ALU** (adds, subtracts, …) and a **shifter**. The blocks are joined by **buses**: shared groups of wires (here 4 wires for 4-bit data). In one clock cycle a bus can carry *one* value from one source to one or more destinations.

2. Counting transfers

The operation "$R_c \leftarrow R_a + R_b$" needs three transfers: operand A from the register file to the ALU, operand B to the ALU, and the result back to the register file. The number of buses decides how many transfers can happen in one cycle:

- **one bus:** one transfer per cycle, so 3 cycles (too slow for the requirement);
- **two buses:** two operands together, then the result: **2 cycles**;
- **three buses:** all three in one cycle: **1 cycle**.

2. Answer

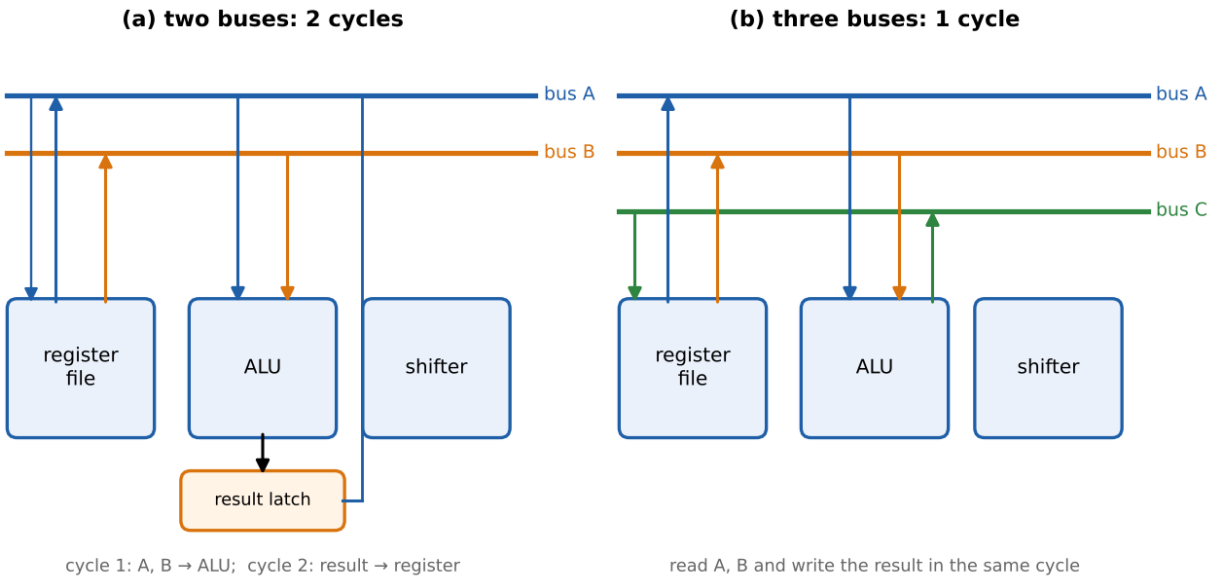


Figure 39: Two bus architectures: (a) two buses, two cycles, with a latch holding the ALU result; (b) three buses, one cycle

Architecture 1: two-bus datapath (2 cycles)

- **Hardware:** bus A and bus B. The register file has **two read ports**, one onto each bus, and one write port from bus A. The ALU takes its two inputs from bus A and bus B; its output goes into a **result latch** that can drive bus A. The shifter is on the same buses.
- **Cycle 1:** the register file puts R_a on bus A and R_b on bus B; the ALU adds them; the sum is stored in the result latch at the end of the cycle.
- **Cycle 2:** the result latch drives bus A; the register file writes it into R_c.

Two cycles: meets the "at most two cycles" requirement.

Architecture 2: three-bus datapath (1 cycle)

- **Hardware:** buses A and B carry the operands from two read ports; bus C carries the ALU (or shifter) output to a separate **write port** of the register file.
- **Cycle 1:** $R_a \rightarrow$ bus A, $R_b \rightarrow$ bus B, ALU adds, sum $\rightarrow$ bus C $\rightarrow$ written into R_c at the clock edge.

Free shared copy · Nur Mohammad

One cycle: faster, but costs a third bus (more wiring area) and a register file with 2 read ports and 1 write port.

Architecture	Buses	Register-file ports	Cycles for $R_c \leftarrow R_a + R_b$
Single bus	1	1 (read or write)	3
Two buses	2	2 read, 1 write	2
Three buses	3	2 read, 1 write (separate)	1

Table 11: Bus architectures compared

3. Keyword Explanation

1. Register file port

↩ back

A read or write access path to the register file; each port can read or write one register per cycle.

4. Notes and Remarks

- With a single bus, temporary registers at the ALU inputs (A and B latches) are needed: cycle 1 loads A, cycle 2 loads B and adds, cycle 3 writes back. That is why at least two buses are needed for the two-cycle requirement.

5.(c) Explain the following: (i) positive mask, (ii) negative mask, (iii) positive photoresist and (iv) negative photoresist. Marks: 2

1. Theory

1. Photolithography in one sentence (Core Idea)

Photolithography copies a pattern from a glass plate (the **mask**) onto the wafer, using ultraviolet (UV) light and a light-sensitive coating (the **photoresist**). It works like printing a photograph from a negative.

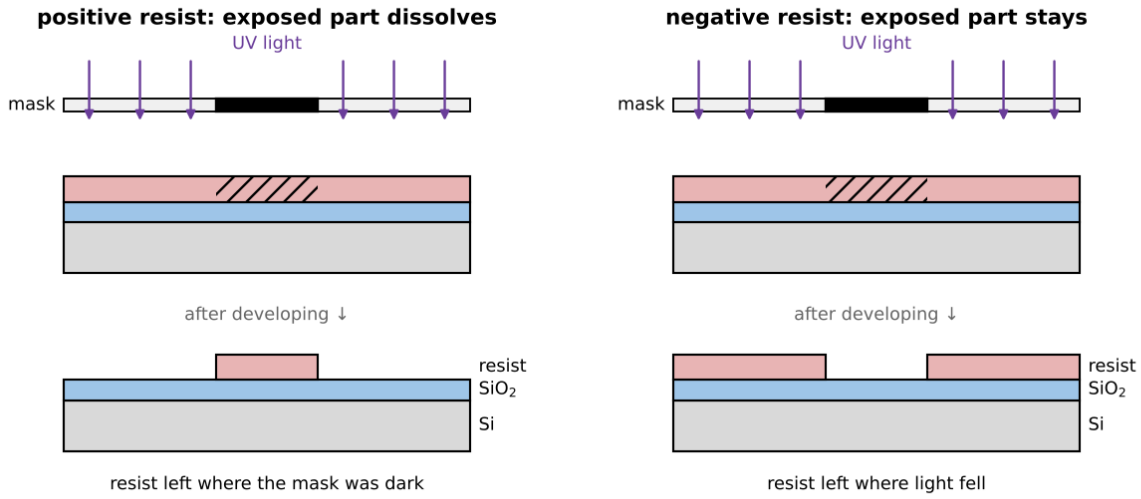


Figure 40: Positive resist keeps the pattern where the mask is dark; negative resist keeps it where light falls

Reading the figure. UV light (purple arrows) passes through the clear part of the mask and is stopped by the dark (chrome) part. The hatched resist is the exposed part. After developing, a positive

resist (left) keeps only the part that was under the dark area; a negative resist (right) keeps only the part that was exposed.

2. Answer

- **(i) Positive mask:** a mask on which the pattern to be made (the circuit feature) is **opaque** on a clear background (a clear-field mask). Light passes everywhere except through the pattern.
- **(ii) Negative mask:** the reverse: the pattern is **clear** on an opaque background (a dark-field mask). Light passes only through the pattern.
- **(iii) Positive photoresist:** exposure to UV light **breaks** its molecules, so the **exposed** parts dissolve in the developer. The resist image is the same as the opaque pattern of the mask.
- **(iv) Negative photoresist:** exposure **cross-links** (hardens) its molecules, so the exposed parts stay and the **unexposed** parts dissolve. The resist image is the reverse of the mask's opaque pattern.

3. Notes and Remarks

- **Positive resist** gives better resolution and is the standard in modern CMOS; negative resist swells in its developer, which limits the smallest feature.

5.(d) Describe the process sequence for patterning SiO_2 as illustrated in the figure 5(d). Marks: 6

1. Theory

1. What is being made? (Core Idea)

Figure 5(d) shows a silicon wafer with a small island of silicon dioxide (SiO_2) on its surface. SiO_2 islands like this are used as masks for doping, as insulation and as gate oxide. To make one, we grow oxide over the whole wafer and then remove it everywhere except where the island must stay. The tool for this is photolithography (5(c)) followed by **etching**.

2. Etching

Etching removes a material where it is not protected. **Hydrofluoric acid** (HF) dissolves SiO_2 but not silicon or photoresist, so the resist protects the oxide under it. Plasma (dry) etching does the same with less sideways undercut.

Free shared copy · Nur Mohammad

2. Answer

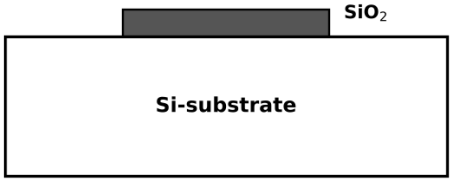


Figure 41: Repeated from the question: Figure 5(d) of the paper (redrawn)

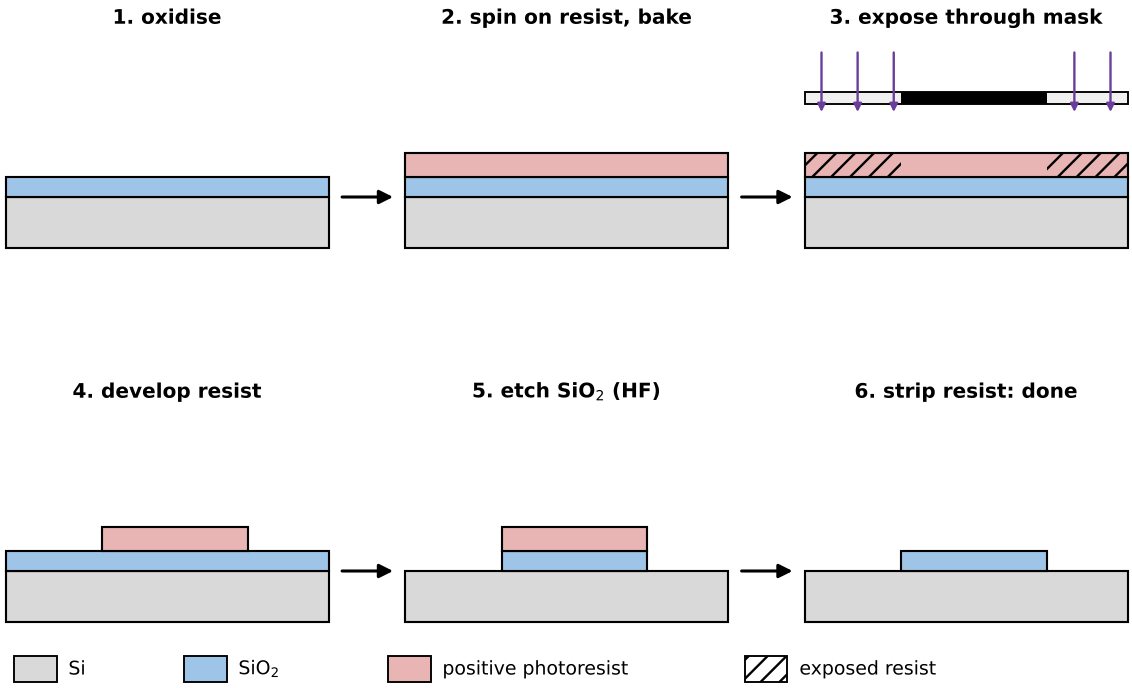


Figure 42: Process sequence for patterning SiO2: oxidise, coat resist, expose, develop, etch, strip

1. **Clean the wafer** to remove dust, organic material and native oxide.
2. **Oxidation:** heat the wafer to about 900–1200 °C in oxygen (dry) or steam (wet). A uniform SiO_2 layer grows over the whole surface ($Si + O_2 \rightarrow SiO_2$).
3. **Photoresist coating:** spin a thin, even layer of positive photoresist on the oxide, then **soft-bake** it to drive out the solvent.
4. **Mask alignment and exposure:** align the mask over the wafer; the mask is opaque over the island to be kept. Shine UV light: the resist outside the island is exposed.
5. **Development:** the developer dissolves the exposed resist. Resist now remains only over the island. **Hard-bake** it to make it tougher.
6. **Etching:** dip in (buffered) HF, or use a plasma etch. The SiO_2 not covered by resist is removed down to the silicon; the oxide under the resist stays.
7. **Resist stripping:** remove the remaining resist with a solvent (acetone) or an oxygen plasma. The SiO_2 island of Figure 5(d) is left on the silicon.

Free shared copy · Nur Mohammad

3. Keyword Explanation

1. Soft-bake and hard-bake

↩ back

Low-temperature bakes of the photoresist: before exposure to remove solvent, and after developing to harden it for etching.

2. Etch selectivity

↩ back

The ratio of etch rates of two materials; HF etches SiO_2 far faster than Si, so the etch stops at the silicon.

4. Final Concise Answer

(a) (i) 2×2 array of 6T cells; WL_0/WL_1 from the row decoder (row select), C_0/C_1 column-select pass transistors to the data lines, precharge pMOS on each bit line ($\overline{pre}$), sense amplifier (SAE) and write driver on the data lines. (ii) Read row 2, column 1: precharge, raise WL_1 and C_0, the side storing 0 discharges its bit line slightly, SAE fires. Write: driver sets $bit_0/\overline{bit}_0$, C_0 and WL_1 high, the low bit line overpowers the cell's pMOS and flips it. (iii) $t = C_{bit}\Delta V/I_{cell}$; the sense amplifier needs only $\Delta V \approx 0.1$–0.2 V instead of about $V_{DD}/2$, so the read is several times faster.

(b) Two buses: cycle 1 reads both operands and adds (result latched), cycle 2 writes back: 2 cycles. Three buses: read A, B and write the result on bus C in 1 cycle.

(c) Positive mask: opaque pattern, clear field. Negative mask: clear pattern, opaque field. Positive resist: exposed part dissolves (same image as the mask). Negative resist: exposed part hardens and stays (reverse image).

(d) Clean → thermal oxidation → spin resist and soft-bake → align mask and UV exposure → develop and hard-bake → HF (or plasma) etch of exposed SiO_2 → strip resist, leaving the SiO_2 island.

Free shared copy · Nur Mohammad

6. (a) Design a 2-to-1 multiplexer using pass transistor logic. Mention its working procedure. Marks: 4

(b) Design a 7-segment display driver in Verilog. Marks: 5

(c) You have to design a divide-by-3 counter using FSM.

i) Draw the state transition diagram of the finite state machine described.
ii) Derive the state transition table and equations for the next-state logic.
iii) Draw the complete FSM circuit. Marks: 6

(d) Write a Verilog code for the FSM described in Question 6(c). Marks: 3

(e) Sketch a dot diagram for a 2-input XOR using a PLA. Marks: 3

(f) Fill in the truth table for the flip flop shown in Figure 6(f). Is this a positive edge triggered flip flop? Marks: 4

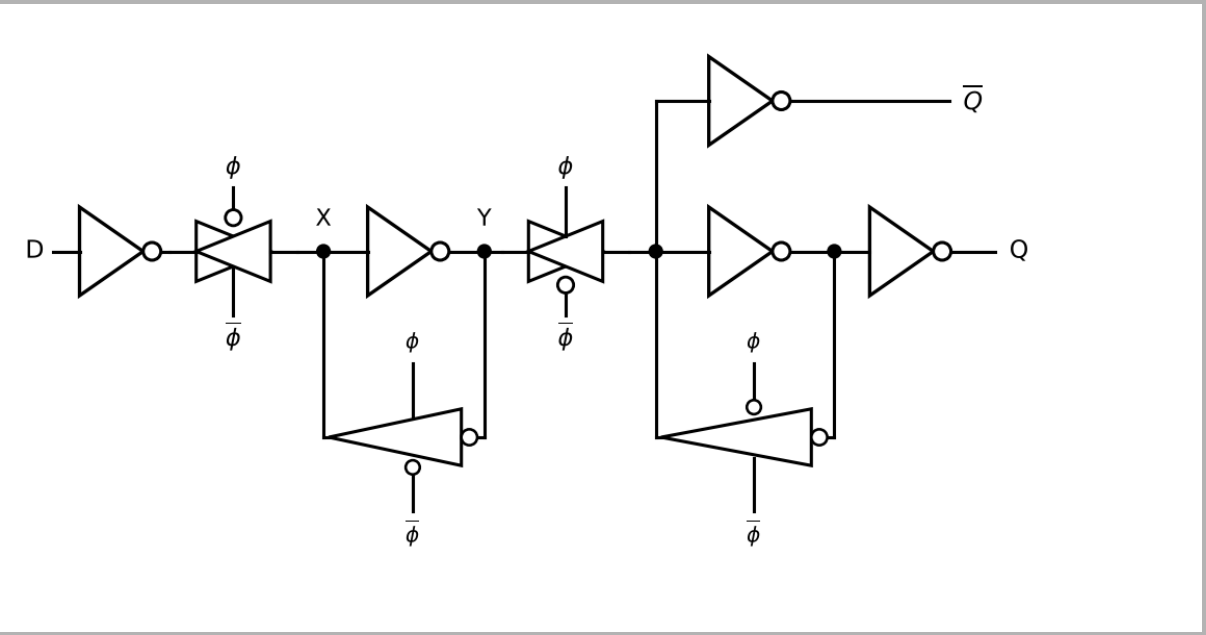


Figure 43: Figure 6(f) of the paper (redrawn)

ϕ	$\overline{\phi}$	D	X	Y	$\overline{Q}$	Q
0	1		1			1
1	0	1				
0	1	1				
1	0	1				

Figure 44: The table of Figure 6(f), as printed

Free shared copy · Nur Mohammad

6.(a) Design a 2-to-1 multiplexer using pass transistor logic. Mention its working procedure. Marks: 4

1. Theory

1. What is a multiplexer? (Core Idea)

A **multiplexer** (mux) is a data selector: it has data inputs, a select input, and one output. A 2-to-1 mux passes D_0 to the output when the select $S = 0$ and D_1 when $S = 1$:

$$Y = \overline{S}\, D_0 + S\, D_1$$

2. Why pass transistors suit a mux

Pass-transistor logic uses transistors as switches that pass a signal (4(d)). A mux is exactly "pass one of two signals", so it needs only two switches, one closed and one open at any time. An nMOS passes a weak 1 ($V_{DD} - V_t$); a transmission gate passes both levels fully.

2. Answer

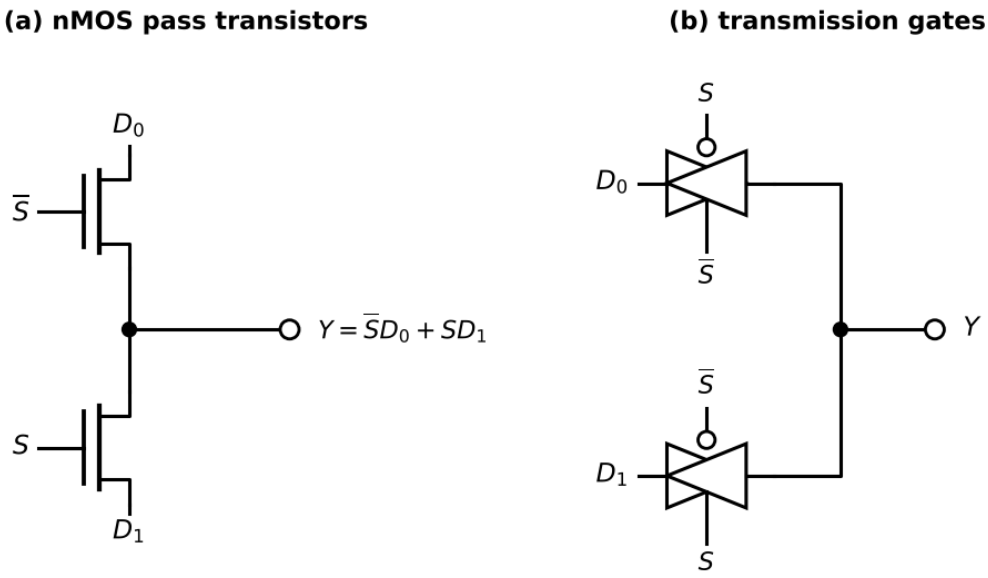


Figure 45: 2-to-1 multiplexer: (a) two nMOS pass transistors, (b) two transmission gates

Design (a), nMOS pass transistors: one nMOS with gate $\overline{S}$ between D_0 and Y, one nMOS with gate S between D_1 and Y. An inverter makes $\overline{S}$ from S.
Working procedure:

- $S = 0$ ($\overline{S} = 1$): the upper nMOS is on and the lower off, so $Y = D_0$.
- $S = 1$ ($\overline{S} = 0$): the lower nMOS is on and the upper off, so $Y = D_1$.
- Exactly one switch is on at any time, so the output is never floating and the two inputs never short together.

S	Switch on	Y
0	$\overline{S}$ transistor	D_0
1	S transistor	D_1

Table 12: Operation of the 2-to-1 multiplexer

Free shared copy · Nur Mohammad

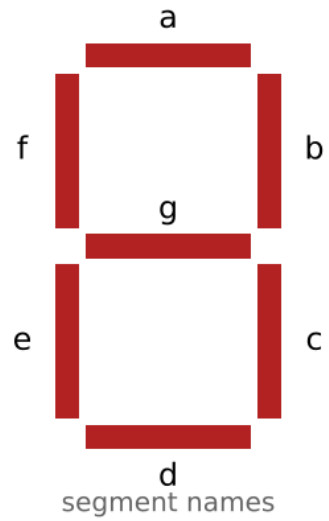
Weak 1: a 1 passed through an nMOS reaches only $V_{DD} - V_t$. Design (b) replaces each nMOS with a **transmission gate** (nMOS and pMOS in parallel, gates $S/\overline{S}$), which passes full 0 and 1 levels; or a buffer with a pMOS level restorer follows design (a).

6.(b) Design a 7-segment display driver in Verilog. Marks: 5

1. Theory

1. The 7-segment display (Core Idea)

A **7-segment display** shows a decimal digit with seven bars named a to g. Turning on the right bars draws each digit; for example 1 lights only b and c.



segments a b c d e f g (1 = on)

0: 1111110	1: 0110000
2: 1101101	3: 1111001
4: 0110011	5: 1011011
6: 1011111	7: 1110000
8: 1111111	9: 1111011

Figure 46: Segment names a to g and the segment pattern of each digit (1 = on)

2. The driver

The **driver** (decoder) takes a 4-bit binary number (0–9) and produces 7 outputs, one per segment. It is pure **combinational logic**: a lookup from the input value to a 7-bit pattern.

3. Describing it in Verilog

In Verilog, combinational logic of this lookup kind is written with `always @(*)` and a `case` statement. The outputs assigned inside an `always` block are declared `reg`, although no register is made, because every case assigns them. A `default` branch covers the inputs 10–15 so that no latch is inferred.

2. Answer

```
module sevenseg(input      [3:0] data,
                output reg [6:0] segments);   // segments[6] = a ... segments[0] = g
  always @(*)
    case (data)
      //                        abc_defg
      4'd0:    segments = 7'b111_1110;
      4'd1:    segments = 7'b011_0000;
      4'd2:    segments = 7'b110_1101;
      4'd3:    segments = 7'b111_1001;
      4'd4:    segments = 7'b011_0011;
      4'd5:    segments = 7'b101_1011;
      4'd6:    segments = 7'b101_1111;
      4'd7:    segments = 7'b111_0000;
      4'd8:    segments = 7'b111_1111;
      4'd9:    segments = 7'b111_1011;
      default: segments = 7'b000_0000;  // 10-15: blank
    endcase
endmodule
```

Listing 1: 7-segment display driver (segments in the order a b c d e f g, 1 = on)

How it works: whenever `data` changes, the `case` picks the pattern for that digit and drives the seven segment lines. Inputs 10 to 15 blank the display. The synthesis tool turns this into a block of combinational gates.

3. Notes and Remarks

- For a **common-anode** display the segments are on when driven **low**; then output `~segments` instead.

6.(c) You have to design a divide-by-3 counter using FSM. i) Draw the state transition diagram of the finite state machine described. ii) Derive the state transition table and equations for the next-state logic. iii) Draw the complete FSM circuit. Marks: 6

1. Theory

1. What is a finite state machine? (Core Idea)

A **finite state machine** (FSM) is a circuit with memory: a register (flip-flops) holds the **current state**, and combinational logic computes the **next state** from the current state (and inputs). At every clock edge the register loads the next state. The outputs depend on the state (a **Moore** machine) or on the state and the inputs (a **Mealy** machine).

Designing an FSM follows fixed steps: draw the **state transition diagram**, choose a binary code for each state, write the **state transition table**, derive the **next-state** and **output** equations, and draw the circuit.

2. A divide-by-3 counter

A **divide-by-3 counter** produces an output y that is 1 for one clock cycle out of every three. Its frequency is therefore one third of the clock frequency. It needs three states that follow each other in a ring: S0 → S1 → S2 → S0 …, with $y = 1$ only in S0. It has no inputs except clock and reset.

2. Answer

Free shared copy · Nur Mohammad

i) State transition diagram

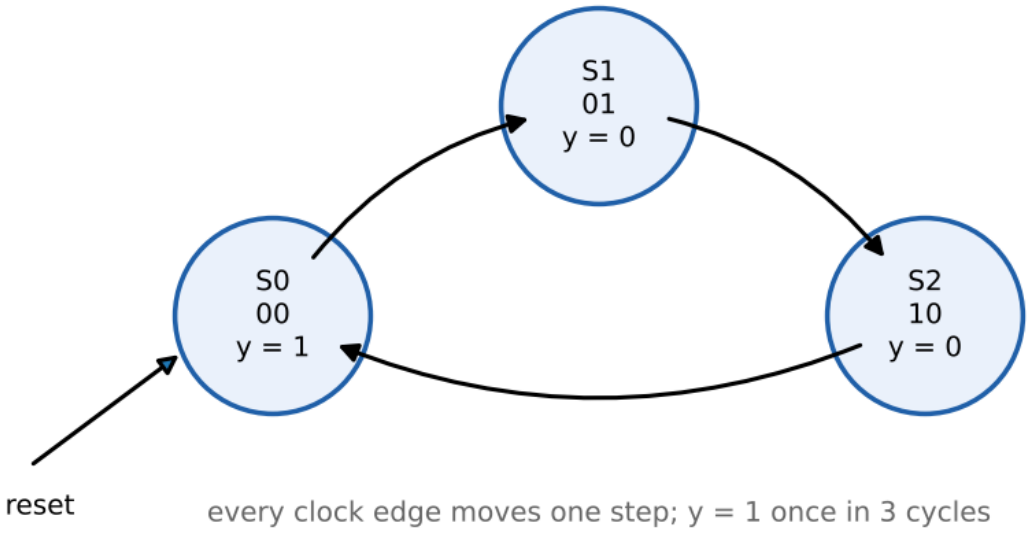


Figure 47: State transition diagram: S0 (y = 1) to S1 to S2 and back to S0 on every clock edge; reset goes to S0

ii) State transition table and next-state equations

State encoding: S0 = 00, S1 = 01, S2 = 10 (S_1S_0); 11 is unused.

Current S_1S_0	Next $S_1'S_0'$	Output y
00 (S0)	01 (S1)	1
01 (S1)	10 (S2)	0
10 (S2)	00 (S0)	0
11 (unused)	X X	X

Table 13: State transition table of the divide-by-3 counter

Next-state equations (using the unused state 11 as a don't care):

$$S_1' = \overline{S_1}\, S_0 \;\rightarrow\; S_1' = S_0$$

$$S_0' = \overline{S_1}\,\overline{S_0} = \overline{S_1 + S_0}$$

Output equation:

$$y = \overline{S_1}\,\overline{S_0}$$

S_0' and y are the same function, so one NOR gate gives both.

Free shared copy · Nur Mohammad

iii) Complete FSM circuit

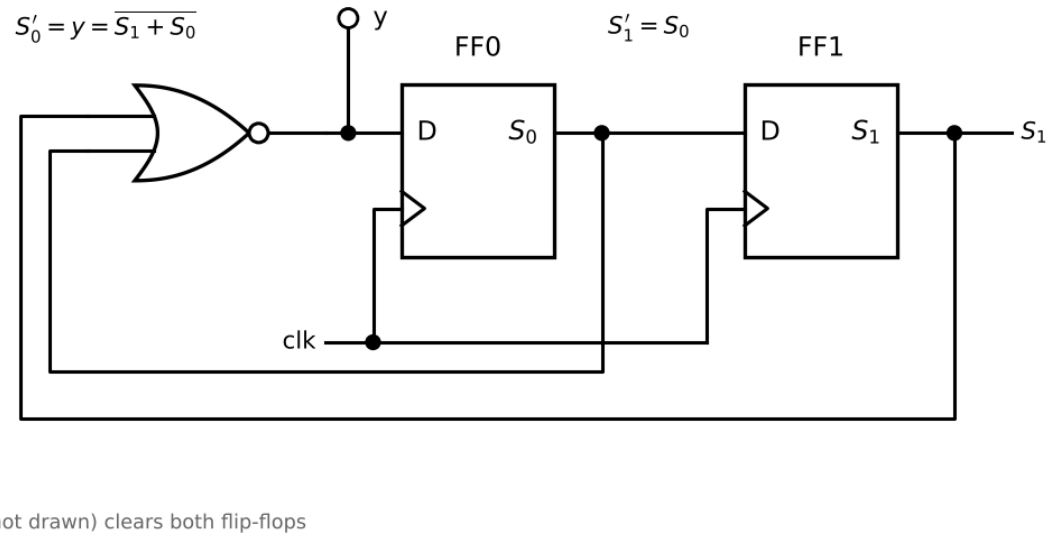


Figure 48: FSM circuit: two D flip-flops, one NOR gate gives the next value of S0 and the output y; S0 feeds FF1

Two D flip-flops hold S_0 and S_1; FF1's D input is S_0; a NOR2 of S_1 and S_0 feeds FF0's D input and is also the output y. Reset clears both flip-flops (state S0).

3. Keyword Explanation

1. Moore machine

↩ back

An FSM whose outputs depend only on the current state.

2. State encoding

↩ back

The choice of binary code for each state; it decides the next-state logic.

4. Notes and Remarks

- **Self-starting:** if the flip-flops power up in 11, the equations give $S_1'S_0' = 10$ (S2), then 00: the counter recovers by itself.

6.(d) Write a Verilog code for the FSM described in Question 6(c). Marks: 3

1. Theory

The basics

A **finite state machine (FSM)** is a circuit with a few memory bits that store its **state**. On each clock edge it moves to the next state, and its output depends on that state. A divide-by-3 counter steps through three states and gives one output pulse for every three clock cycles. **Verilog** is a language that describes such hardware in text.

1. Describing an FSM in Verilog (Core Idea)

A Verilog FSM has three parts, matching the circuit of 6(c):

- a **state register:** an `always @(posedge clk ...)` block with **non-blocking** assignments (`<=`), which makes flip-flops;
- **next-state logic:** an `always @(*)` block with a `case` on the state, which makes combinational logic;
- **output logic:** an `assign` that depends on the state.

Free shared copy · Nur Mohammad

Naming the states with `parameter` keeps the code readable.

2. Answer

```
module divideby3FSM(input  clk,
                    input  reset,
                    output y);
  reg [1:0] state, nextstate;

  parameter S0 = 2'b00;
  parameter S1 = 2'b01;
  parameter S2 = 2'b10;

  // state register
  always @(posedge clk, posedge reset)
    if (reset) state <= S0;
    else       state <= nextstate;

  // next-state logic
  always @(*)
    case (state)
      S0:      nextstate = S1;
      S1:      nextstate = S2;
      S2:      nextstate = S0;
      default: nextstate = S0;
    endcase

  // output logic
  assign y = (state == S0);
endmodule
```

Listing 2: Divide-by-3 counter FSM (Moore machine, asynchronous reset)

How it works: reset puts the machine in S0. On each rising clock edge the register loads `nextstate`, so the state runs S0 → S1 → S2 → S0. The output `y` is 1 only in S0, once every three cycles. The `default` branch sends the unused state 11 back to S0.

6.(e) Sketch a dot diagram for a 2-input XOR using a PLA. Marks: 3

1. Theory

1. What is a PLA? (Core Idea)

A **programmable logic array** (PLA) implements any sum-of-products function with two regular grids of wires:

- the **AND plane:** vertical lines carry every input and its complement (A, $\overline{A}$, B, $\overline{B}$); each horizontal line is a **product term**. A dot where a vertical and a horizontal line cross means "this literal is in this product";
- the **OR plane:** each output line collects (ORs) the product terms marked with a dot.

On the chip each dot is a transistor (often in pseudo-NMOS NOR-NOR form), so the PLA is a dense, regular layout that is easy to generate.

2. XOR as a sum of products

$$Y = A \oplus B = A\overline{B} + \overline{A}B$$

Two product terms are needed: $P_1 = A\overline{B}$ and $P_2 = \overline{A}B$.

Free shared copy · Nur Mohammad

2. Answer

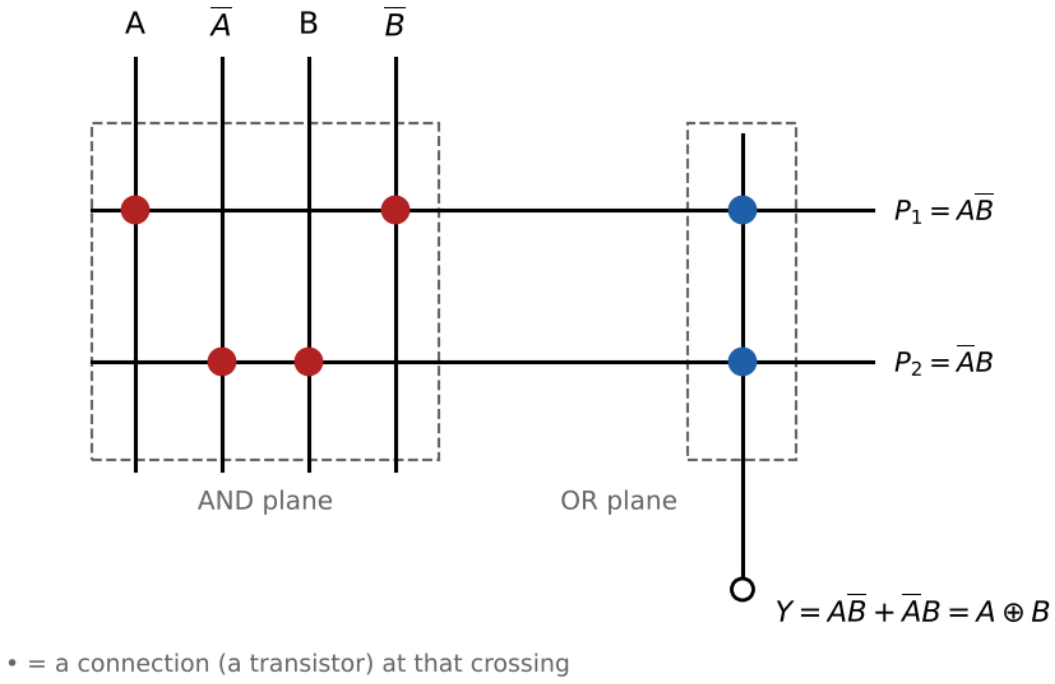


Figure 49: PLA dot diagram of a two-input XOR: dots at A and B-bar for P1, at A-bar and B for P2; both products dotted onto the output Y

- **AND plane:** row P_1 has dots on A and $\overline{B}$; row P_2 has dots on $\overline{A}$ and B.
- **OR plane:** the output column Y has dots on both rows.
- **Result:** $Y = P_1 + P_2 = A\overline{B} + \overline{A}B = A \oplus B$.

6.(f) Fill in the truth table for the flip flop shown in Figure 6(f). Is this a positive edge triggered flip flop? Marks: 4

1. Theory

1. Reading the circuit (Core Idea)

The circuit of Figure 6(f) is a master-slave flip-flop (4(b)) built from inverters, transmission gates and tristate inverters (inverters that drive their output only when enabled; otherwise they are disconnected).

- **Transmission gate rule:** the control with the bubble drives the pMOS; the gate conducts when the bubble input is 0 and the other input is 1.
- **Input stage:** D is inverted, so the master stores $X = \overline{D}$.

2. Which latch is open when?

- **First transmission gate** (bubble ϕ, other input $\overline{\phi}$): conducts when $\phi = 0$. The **master is transparent at** $\phi = 0$: $X = \overline{D}$, $Y = \overline{X} = D$. Its feedback tristate is enabled at $\phi = 1$, so the master holds while $\phi = 1$.
- **Second transmission gate** (bubble $\overline{\phi}$): conducts when $\phi = 1$. The **slave is transparent at** $\phi = 1$: its node $Z = Y$, so $\overline{Q} = \overline{Z}$ and $Q = Z = Y$. Its feedback tristate is enabled at $\phi = 0$, so the slave holds while $\phi = 0$.

Free shared copy · Nur Mohammad

So the value of D just before ϕ rises is held by the master and passed to Q while $\phi = 1$.

2. Answer

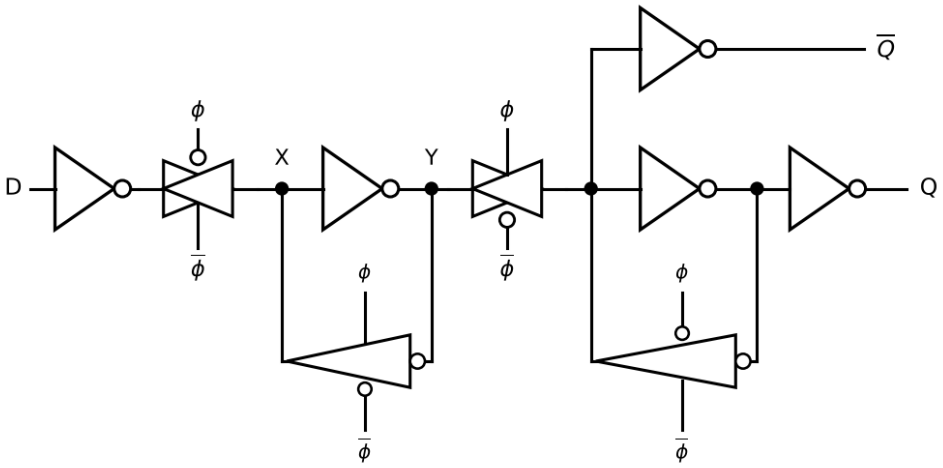


Figure 50: Repeated from the question: Figure 6(f) of the paper (redrawn)

ϕ	$\overline{\phi}$	D	X	Y	$\overline{Q}$	Q
0	1		1			1
1	0	1				
0	1	1				
1	0	1				

Figure 51: Repeated from the question: The table of Figure 6(f), as printed

Row 1 ($\phi = 0$): the master is transparent and $X = 1$, so $D = 0$ ($X = \overline{D}$) and $Y = 0$. The slave holds the old value $Q = 1$, so $\overline{Q} = 0$.
Row 2 (ϕ rises to 1, $D = 1$): the master closes and keeps $X = 1$, $Y = 0$ (D changing to 1 at the edge is too late). The slave opens: $Q = Y = 0$, $\overline{Q} = 1$.
Row 3 ($\phi = 0$, $D = 1$): the master opens: $X = \overline{D} = 0$, $Y = 1$. The slave holds $Q = 0$, $\overline{Q} = 1$.
Row 4 (ϕ rises to 1, $D = 1$): the master holds $X = 0$, $Y = 1$; the slave opens: $Q = 1$, $\overline{Q} = 0$.

ϕ	$\overline{\phi}$	**D**	**X**	**Y**	$\overline{Q}$	**Q**
0	1	0	1	0	0	1
1	0	1	1	0	1	0
0	1	1	0	1	1	0
1	0	1	0	1	0	1

Table 14: Completed truth table of Figure 6(f); D = 0 in row 1 is not printed on the paper but follows from X = 1

Free shared copy · Nur Mohammad

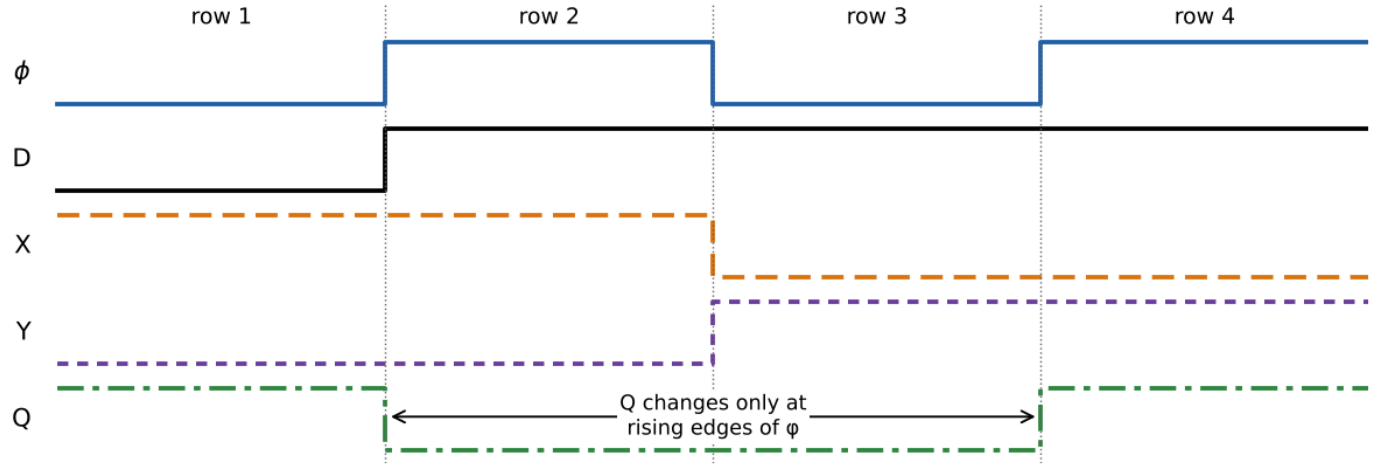


Figure 52: The four rows as waveforms: Q changes only when phi rises

Is it positive edge triggered? Yes. The master is transparent while $\phi = 0$ and the slave while $\phi = 1$, so Q can change only when ϕ goes from 0 to 1, and it takes the value D had just before that rising edge (rows 2 and 4). While ϕ stays at 1 or 0, Q does not follow D (row 3: $D = 1$ but Q stays 0).

3. Keyword Explanation

1. Tristate inverter

↩ back

An inverter with enable inputs; when disabled its output floats, so it does not fight the transmission gate that drives the same node.

2. Edge triggered

↩ back

A storage element that changes its output only at a clock edge, not during a clock level.

4. Final Concise Answer

(a) nMOS with gate $\overline{S}$ passes D_0, nMOS with gate S passes D_1, outputs joined: $Y = \overline{S}D_0 + SD_1$; exactly one switch conducts; use transmission gates for full-swing levels.

(b) `always @(*) case (data)` mapping 0–9 to the 7-bit patterns abc_defg (0: 1111110, 1: 0110000, …, 9: 1111011), default blank.

(c) S0 (00, y = 1) → S1 (01) → S2 (10) → S0; $S_1' = S_0$, $S_0' = \overline{S_1 + S_0}$, $y = \overline{S_1}\,\overline{S_0}$; two D flip-flops and one NOR2.

(d) State register `always @(posedge clk, posedge reset)`, next-state `case` (S0→S1→S2→S0), `assign y = (state == S0)`.

(e) AND plane: $P_1 = A\overline{B}$, $P_2 = \overline{A}B$; OR plane: $Y = P_1 + P_2$.

(f) Rows: (X, Y, $\overline{Q}$, Q) = (1, 0, 0, 1), (1, 0, 1, 0), (0, 1, 1, 0), (0, 1, 0, 1). Master transparent at $\phi = 0$, slave at $\phi = 1$: **yes, positive edge triggered**.

Free shared copy · Nur Mohammad

FORMULA SHEET

EEE455

VLSI Design and Testing

Formula Sheet

Session 2020–21 paper *Every formula behind the 4th Year 1st Semester Final Examination 2024*

TOPICS

1 MOS transistor — 9 formulas

2 Inverter levels and noise margins — 5 formulas

3 Delay — 10 formulas

4 Transistor sizing — 5 formulas

5 Power — 4 formulas

6 Sequential timing — 4 formulas

7 Memory and logic functions — 5 formulas

Each topic is one table: **Title** · **Equation** · **Notation** · **Reference**.

Prepared by Nur Mohammad

October 2026

BACK OF THE COVER

Before you use the sheet

How to read a row

Oxide capacitance	$C_{ox} = \frac{\varepsilon_{ox}}{t_{ox}}$	C_{ox} = gate capacitance per area (F/m^2) ε_{ox} = permittivity of SiO_2, $3.9\varepsilon_0$ (F/m) t_{ox} = oxide thickness (m)	1(a)

the name you search for

the formula, exactly as derived

every new symbol with its unit

link: opens the derivation

In the exam

- Write the formula first, then put the numbers in: the formula alone often earns a mark.
- Read the symbols in the Notation column before you substitute, and check the units.
- Not sure where a formula comes from? Its Reference opens the derivation.
- The **Key formulas** page at the end gathers the rows marked ★.
- Lost a symbol? The **Symbol index** at the end links to the row where it first appears.

Contents

1 MOS transistor 4

2 Inverter levels and noise margins 5

3 Delay 5

4 Transistor sizing 6

5 Power 7

6 Sequential timing 7

7 Memory and logic functions 8

8 Key formulas 9

9 Symbol index 10

Free shared copy · Nur Mohammad

Conventions: nMOS quantities carry the subscript n, pMOS p; for a pMOS, V_{GS}, V_{DS} and V_{Tp} are negative. Widths in logical effort are in units of the unit inverter's nMOS (unit inverter: nMOS 1, pMOS 2, from $\mu_n = 2\mu_p$) unless the question says otherwise. Delays are in units of τ; 1 FO4 $= 5\tau$. References point to the solutions of the 2020–21 paper.

Columns: Title · Equation · Notation · Ref. (reference).

1. MOS transistor:

Title	Equation	Notation	Ref.
Oxide capacitance	$C_{ox} = \dfrac{\varepsilon_{ox}}{t_{ox}}$	C_{ox} = gate capacitance per area (F/m^2) ε_{ox} = permittivity of SiO_2, $3.9\varepsilon_0$ (F/m) t_{ox} = oxide thickness (m)	1(a)
★ **Gate capacitance**	$C_0 = C_{ox}WL$	C_0 = full gate capacitance (F) W = gate width (m) L = gate length (m)	1(a)
Gate capacitance, cutoff	$C_{gb} \approx C_0,\ C_{gs} = C_{gd} = 0$	C_{gb} = gate–body capacitance (F) C_{gs} = gate–source capacitance (F) C_{gd} = gate–drain capacitance (F)	1(a)
Gate capacitance, linear	$C_{gs} = C_{gd} = \frac{1}{2}C_0,\ C_{gb} = 0$	valid for $V_{GS} > V_T$ and $V_{DS} < V_{GS} - V_T$	1(a)
Gate capacitance, saturation	$C_{gs} = \frac{2}{3}C_0,\ C_{gd} = C_{gb} = 0$	valid for $V_{DS} \geq V_{GS} - V_T$	1(a)
Device gain factor	$k = k'\dfrac{W}{L},\ k' = \mu C_{ox}$	k = device transconductance (A/V^2) k' = process transconductance (A/V^2) μ = carrier mobility (m^2/V · s)	1(b)
★ **Drain current, linear**	$I_D = \dfrac{k}{2}\left[2(V_{GS} - V_T)V_{DS} - V_{DS}^2\right]$	I_D = drain current (A) V_{GS} = gate–source voltage (V) V_{DS} = drain–source voltage (V) V_T = threshold voltage (V)	1(b)
★ **Drain current, saturation**	$I_D = \dfrac{k}{2}(V_{GS} - V_T)^2$	valid for $V_{DS} \geq V_{GS} - V_T$	1(b)
Body effect	$V_T = V_{T0} + \gamma\left(\sqrt{\lvert 2\phi_F\rvert + V_{SB}} - \sqrt{\lvert 2\phi_F\rvert}\right)$	V_{T0} = threshold at $V_{SB} = 0$ (V) γ = body-effect coefficient (V$^{1/2}$) ϕ_F = Fermi potential (V) V_{SB} = source–body voltage (V)	1(b)

Table 1: MOS transistor

2. Inverter levels and noise margins:

Title	Equation	Notation	Ref.
★ **Pseudo-NMOS output low**	$k_n\left[2(V_{DD}-V_{Tn})V_{OL}-V_{OL}^2\right]=k_p(V_{DD}-\lvert V_{Tp}\rvert)^2$	V_{Tn}, V_{Tp} = nMOS and pMOS thresholds (V) V_{OL} = output low voltage (V) V_{DD} = supply voltage (V) k_n = driver gain factor (A/V^2) k_p = load gain factor (A/V^2)	1(b)
Inverter ratio	$\frac{(W/L)_n}{(W/L)_p}=\frac{k_n}{k_p}\cdot\frac{\mu_pC_{ox}}{\mu_nC_{ox}}$	μ_n, μ_p = electron and hole mobilities (m^2/V $\cdot$ s) C_{ox} = gate capacitance per area (F/m^2) $(W/L)_n$ = driver aspect ratio $(W/L)_p$ = load aspect ratio	1(b)
★ **Switching threshold**	$V_M=\frac{V_{DD}-\lvert V_{Tp}\rvert+V_{Tn}\sqrt{\beta_n/\beta_p}}{1+\sqrt{\beta_n/\beta_p}}$	V_M = input where $V_{out}=V_{in}$ (V) β_n/β_p = beta ratio	2(b)
Noise margins	$NM_L=V_{IL}-V_{OL},\ NM_H=V_{OH}-V_{IH}$	NM_L = low noise margin (V) NM_H = high noise margin (V) V_{IL} = lower unity-gain input (V) V_{IH} = upper unity-gain input (V) V_{OH} = output high voltage (V)	2(b)
Symmetric CMOS V_{IL}	$V_{IL}=\frac{3V_{DD}+2V_T}{8},\ V_{IH}=\frac{5V_{DD}-2V_T}{8}$	V_T = threshold of either transistor (V) valid for $\beta_n=\beta_p$, $V_{Tn}=\lvert V_{Tp}\rvert=V_T$	2(b)

Table 2: Inverter levels and noise margins

3. Delay:

Title	Equation	Notation	Ref.
★ **Elmore delay**	$t_{D,i}=\sum_k R_{ik}C_k$	$t_{D,i}$ = delay from the input to node i (s) C_k = capacitance at node k (F) R_{ik} = resistance shared by the paths to i and k (Ω)	2(a)
Delay from current	$t_{pd}=\frac{C_L\,V_{DD}/2}{I_{sat}}=\frac{C_LV_{DD}}{\beta(V_{DD}-V_T)^2}$	V_{DD} = supply voltage (V) β = gain factor (A/V^2) V_T = threshold voltage (V) t_{pd} = propagation delay (s) C_L = load capacitance (F) I_{sat} = saturation current (A)	3(c)
★ **Delay vs supply**	$t_{pd}\approx\frac{C_L}{\beta V_{DD}}\propto\frac{1}{V_{DD}}$	valid for $V_{DD}\gg V_T$	3(c)
Gate delay	$d=gh+p$	d = delay of one gate (τ) g = logical effort h = electrical effort, C_{out}/C_{in} p = parasitic delay (τ)	1(d)
Logical effort, NAND and NOR	$g_{NAND\text{-}n}=\frac{n+2}{3},\ g_{NOR\text{-}n}=\frac{2n+1}{3}$	$g_{NAND\text{-}n}$, $g_{NOR\text{-}n}$ = logical efforts of n-input NAND and NOR gates n = number of inputs parasitic delay $p=n$ for both	1(d)

Free shared copy · Nur Mohammad

Title	Equation	Notation	Ref.
Path effort	$F = GBH,\ G = \prod g_i,\ B = \prod b_i,\ H = \dfrac{C_{out}}{C_{in}}$	g_i = logical effort of stage i C_{out}, C_{in} = load of the path and input capacitance of its first stage (F) F = path effort G = path logical effort B = path branching effort H = path electrical effort b_i = branching of stage i	1(d)
★ **Minimum path delay**	$\hat{D} = NF^{1/N} + P,\ \hat{f} = F^{1/N}$	$\hat{D}$ = least path delay (τ) N = number of stages P = sum of parasitic delays (τ) $\hat{f}$ = best stage effort	1(d)
Gate sizing	$C_{in,i} = \dfrac{g_i C_{out,i}}{\hat{f}}$	$C_{in,i}$ = input capacitance of stage i $C_{out,i}$ = load of stage i	1(d)
Best number of stages	$\hat{N} \approx \log_4 F$	best stage effort near 4	2(c)
FO4 delay	$d_{FO4} = 1 \cdot 4 + 1 = 5\tau$	d_{FO4} = delay of an inverter driving four copies of itself τ = delay unit of the process (s) FO4 = inverter driving 4 copies of itself	1(d)

Table 3: Delay

4. Transistor sizing:

Title	Equation	Notation	Ref.
Unit inverter	$W_p = \dfrac{\mu_n}{\mu_p} W_n$	μ_n, μ_p = electron and hole mobilities ($m^2/V \cdot s$) W_n = nMOS width W_p = pMOS width for equal rise and fall	3(b)
★ **Series stack**	$W = k\, W_{unit}$	W = width of each transistor in the stack k = transistors in series on the path W_{unit} = width in the unit inverter	3(b)
Path resistance	$\sum_{\text{path}} \dfrac{R_{unit} W_{unit}}{W_j} \leq R_{unit}$	R_{unit} = resistance of the unit transistor (Ω) W_j = width of transistor j on the path	3(b)
HI-skew inverter	$g_u = \frac{5}{6},\ g_d = \frac{5}{3}$	g_u = logical effort, rising output g_d = logical effort, falling output sizes pMOS 2, nMOS 1/2	3(d)
LO-skew inverter	$g_u = \frac{4}{3},\ g_d = \frac{2}{3}$	sizes pMOS 1, nMOS 1	3(d)

Table 4: Transistor sizing

Free shared copy · Nur Mohammad

5. Power:

Title	Equation	Notation	Ref.
Energy per output cycle	$E = C_L V_{DD}^2$	C_L = load capacitance (F) V_{DD} = supply voltage (V) E = energy taken from the supply (J)	3(c)
Energy stored	$E_C = \frac{1}{2} C_L V_{DD}^2$	E_C = energy in the charged load (J)	3(c)
★ **Dynamic power**	$P_{dynamic} = \alpha C_L V_{DD}^2 f$	$P_{dynamic}$ = switching power (W) α = activity factor f = clock frequency (Hz)	2(d)
Static power	$P_{static} = I_{static} V_{DD}$	P_{static} = static power (W) I_{static} = leakage or contention current (A)	2(d)

Table 5: Power

6. Sequential timing:

Title	Equation	Notation	Ref.
Latest data arrival	$t_{data,max} = t_{clk1,max} + t_{clk\text{-}q,max} + t_{logic,max}$	$t_{data,max}$ = latest arrival at the capturing D (s) $t_{clk1,max}$ = latest launch clock (s) $t_{clk\text{-}q,max}$ = largest clock-to-Q delay (s) $t_{logic,max}$ = largest logic and wire delay (s)	4(a)
★ **Setup constraint**	$t_{data,max} + t_{setup} \leq T + t_{clk2,min}$	t_{setup} = setup time (s) T = clock period (s) $t_{clk2,min}$ = earliest capture clock (s)	4(a)
★ **Hold constraint**	$t_{data,min} \geq t_{clk2,max} + t_{hold}$	$t_{data,min}$ = earliest arrival at the capturing D (s) $t_{clk2,max}$ = latest capture clock (s) t_{hold} = hold time (s)	4(a)
Maximum frequency	$f_{max} = \frac{1}{T_{min}}$	f_{max} = highest clock frequency (Hz) T_{min} = least period meeting setup (s)	4(a)

Table 6: Sequential timing

Free shared copy · Nur Mohammad

7. Memory and logic functions:

Title	Equation	Notation	Ref.
Bit-line read time	$t = \dfrac{C_{bit}\,\Delta V}{I_{cell}}$	t = read time (s) C_{bit} = bit-line capacitance (F) ΔV = swing needed (V) I_{cell} = cell read current (A)	5(a)
Car alarm	$Y = KB\overline{D} + K\overline{B}D = K(B \oplus D)$	Y = alarm output K = key in B = belt fastened D = door closed	3(a)
2-to-1 multiplexer	$Y = \overline{S}D_0 + SD_1$	Y = multiplexer output S = select input D_0, D_1 = data inputs	6(a)
XOR as sum of products	$A \oplus B = A\overline{B} + \overline{A}B$	A, B = inputs	6(e)
Divide-by-3 next state	$S_1' = S_0,\ S_0' = \overline{S_1 + S_0},\ y = \overline{S_1}\,\overline{S_0}$	S_1', S_0' = next-state bits S_1S_0 = state bits y = counter output	6(c)

Table 7: Memory and logic functions

Free shared copy · Nur Mohammad

8. Key formulas:

The formulas marked ★ in the sheet, on one page. Each title links back to its row.

Title	Equation
Gate capacitance	$C_0 = C_{ox} W L$
Drain current, linear	$I_D = \frac{k}{2}\left[2(V_{GS} - V_T)V_{DS} - V_{DS}^2\right]$
Drain current, saturation	$I_D = \frac{k}{2}(V_{GS} - V_T)^2$
Pseudo-NMOS output low	$k_n\left[2(V_{DD} - V_{Tn})V_{OL} - V_{OL}^2\right] = k_p(V_{DD} - \lvert V_{Tp}\rvert)^2$
Switching threshold	$V_M = \frac{V_{DD} - \lvert V_{Tp}\rvert + V_{Tn}\sqrt{\beta_n/\beta_p}}{1 + \sqrt{\beta_n/\beta_p}}$
Elmore delay	$t_{D,i} = \sum_k R_{ik} C_k$
Delay vs supply	$t_{pd} \approx \frac{C_L}{\beta V_{DD}} \propto \frac{1}{V_{DD}}$
Minimum path delay	$\hat{D} = NF^{1/N} + P,\ \hat{f} = F^{1/N}$
Series stack	$W = k\, W_{unit}$
Dynamic power	$P_{dynamic} = \alpha C_L V_{DD}^2 f$
Setup constraint	$t_{data,max} + t_{setup} \leq T + t_{clk2,min}$
Hold constraint	$t_{data,min} \geq t_{clk2,max} + t_{hold}$

Free shared copy · Nur Mohammad

9. Symbol index:

Every symbol the sheet defines, in the order it first appears. The last column links to that row.

Symbol	Meaning	First used in
C_{ox}	gate capacitance per area (F/m^2)	Oxide capacitance
ε_{ox}	permittivity of SiO_2, $3.9\varepsilon_0$ (F/m)	Oxide capacitance
t_{ox}	oxide thickness (m)	Oxide capacitance
C_0	full gate capacitance (F)	Gate capacitance
W	gate width (m)	Gate capacitance
L	gate length (m)	Gate capacitance
C_{gb}	gate–body capacitance (F)	Gate capacitance, cutoff
C_{gs}	gate–source capacitance (F)	Gate capacitance, cutoff
C_{gd}	gate–drain capacitance (F)	Gate capacitance, cutoff
k	device transconductance (A/V^2)	Device gain factor
k'	process transconductance (A/V^2)	Device gain factor
μ	carrier mobility (m^2/V $\cdot$ s)	Device gain factor
I_D	drain current (A)	Drain current, linear
V_{GS}	gate–source voltage (V)	Drain current, linear
V_{DS}	drain–source voltage (V)	Drain current, linear
V_T	threshold voltage (V)	Drain current, linear
V_{T0}	threshold at $V_{SB} = 0$ (V)	Body effect
γ	body-effect coefficient (V$^{1/2}$)	Body effect
ϕ_F	Fermi potential (V)	Body effect
V_{SB}	source–body voltage (V)	Body effect
V_{Tn}, V_{Tp}	nMOS and pMOS thresholds (V)	Pseudo-NMOS output low
V_{OL}	output low voltage (V)	Pseudo-NMOS output low
V_{DD}	supply voltage (V)	Pseudo-NMOS output low
k_n	driver gain factor (A/V^2)	Pseudo-NMOS output low
k_p	load gain factor (A/V^2)	Pseudo-NMOS output low
μ_n, μ_p	electron and hole mobilities (m^2/V $\cdot$ s)	Inverter ratio
$(W/L)_n$	driver aspect ratio	Inverter ratio
$(W/L)_p$	load aspect ratio	Inverter ratio
V_M	input where $V_{out} = V_{in}$ (V)	Switching threshold
β_n/β_p	beta ratio	Switching threshold
NM_L	low noise margin (V)	Noise margins
NM_H	high noise margin (V)	Noise margins
V_{IL}	lower unity-gain input (V)	Noise margins
V_{IH}	upper unity-gain input (V)	Noise margins
V_{OH}	output high voltage (V)	Noise margins
V_T	threshold of either transistor (V)	Symmetric CMOS V_{IL}

Free shared copy · Nur Mohammad

Symbol	Meaning	First used in
$t_{D,i}$	delay from the input to node i (s)	Elmore delay
C_k	capacitance at node k (F)	Elmore delay
R_{ik}	resistance shared by the paths to i and k (Ω)	Elmore delay
β	gain factor (A/V^2)	Delay from current
t_{pd}	propagation delay (s)	Delay from current
C_L	load capacitance (F)	Delay from current
I_{sat}	saturation current (A)	Delay from current
d	delay of one gate (τ)	Gate delay
g	logical effort	Gate delay
h	electrical effort, C_{out}/C_{in}	Gate delay
p	parasitic delay (τ)	Gate delay
$g_{NAND\text{-}n}$, $g_{NOR\text{-}n}$	logical efforts of n-input NAND and NOR gates	Logical effort, NAND and NOR
n	number of inputs	Logical effort, NAND and NOR
g_i	logical effort of stage i	Path effort
C_{out}, C_{in}	load of the path and input capacitance of its first stage (F)	Path effort
F	path effort	Path effort
G	path logical effort	Path effort
B	path branching effort	Path effort
H	path electrical effort	Path effort
b_i	branching of stage i	Path effort
$\hat{D}$	least path delay (τ)	Minimum path delay
N	number of stages	Minimum path delay
P	sum of parasitic delays (τ)	Minimum path delay
$\hat{f}$	best stage effort	Minimum path delay
$C_{in,i}$	input capacitance of stage i	Gate sizing
$C_{out,i}$	load of stage i	Gate sizing
d_{FO4}	delay of an inverter driving four copies of itself	FO4 delay
τ	delay unit of the process (s)	FO4 delay
FO4	inverter driving 4 copies of itself	FO4 delay
W_n	nMOS width	Unit inverter
W_p	pMOS width for equal rise and fall	Unit inverter
W	width of each transistor in the stack	Series stack
k	transistors in series on the path	Series stack
W_{unit}	width in the unit inverter	Series stack
R_{unit}	resistance of the unit transistor (Ω)	Path resistance
W_j	width of transistor j on the path	Path resistance
g_u	logical effort, rising output	HI-skew inverter

Free shared copy · Nur Mohammad

Symbol	Meaning	First used in
g_d	logical effort, falling output	HI-skew inverter
E	energy taken from the supply (J)	Energy per output cycle
E_C	energy in the charged load (J)	Energy stored
$P_{dynamic}$	switching power (W)	Dynamic power
α	activity factor	Dynamic power
f	clock frequency (Hz)	Dynamic power
P_{static}	static power (W)	Static power
I_{static}	leakage or contention current (A)	Static power
$t_{data,max}$	latest arrival at the capturing D (s)	Latest data arrival
$t_{clk1,max}$	latest launch clock (s)	Latest data arrival
$t_{clk\text{-}q,max}$	largest clock-to-Q delay (s)	Latest data arrival
$t_{logic,max}$	largest logic and wire delay (s)	Latest data arrival
t_{setup}	setup time (s)	Setup constraint
T	clock period (s)	Setup constraint
$t_{clk2,min}$	earliest capture clock (s)	Setup constraint
$t_{data,min}$	earliest arrival at the capturing D (s)	Hold constraint
$t_{clk2,max}$	latest capture clock (s)	Hold constraint
t_{hold}	hold time (s)	Hold constraint
f_{max}	highest clock frequency (Hz)	Maximum frequency
T_{min}	least period meeting setup (s)	Maximum frequency
t	read time (s)	Bit-line read time
C_{bit}	bit-line capacitance (F)	Bit-line read time
ΔV	swing needed (V)	Bit-line read time
I_{cell}	cell read current (A)	Bit-line read time
Y	alarm output	Car alarm
K	key in	Car alarm
B	belt fastened	Car alarm
D	door closed	Car alarm
Y	multiplexer output	2-to-1 multiplexer
S	select input	2-to-1 multiplexer
D_0, D_1	data inputs	2-to-1 multiplexer
A, B	inputs	XOR as sum of products
S_1', S_0'	next-state bits	Divide-by-3 next state
S_1S_0	state bits	Divide-by-3 next state
y	counter output	Divide-by-3 next state

Free shared copy · Nur Mohammad